Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет ИЭФ
Кафедра Экономической информатики
Дисциплина «Программирование сетевых приложений»

«К ЗАЩИТЕ ДОПУСТИТЬ»
Руководитель курсового проекта
ассистент кафедры ЭИ
_____ _Н. А. Шилов
___.___.2025

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему:

«РАЗРАБОТКА СИСТЕМЫ АВТОМАТИЗАЦИИ РАБОТЫ ШКОЛЫ ИНОСТРАННЫХ ЯЗЫКОВ»

БГУИР КП 6-05-0611-01 027 ПЗ

Выполнил студент группы 372302
Усова Виктория Александровна

_____
(подпись студента)

Курсовой проект представлен на проверку ___._____.2025

_____
(подпись студента)

Минск 2025

РЕФЕРАТ

БГУИР КП 6-05-0611-01 027 ПЗ

Усова, В. А. Разработка системы автоматизации работы школы иностранных языков / В. А. Усова. – Минск : БГУИР, 2025. – 104 с.

Пояснительная записка 104 с., 84 рис., 4 табл., 15 источников, 5 приложений

ШКОЛА, ИНОСТРАННЫЕ ЯЗЫКИ, ОБРАЗОВАТЕЛЬНЫЕ УСЛУГИ, JAVA, АВТОМАТИЗИРОВАННАЯ СИСТЕМА, КУРСЫ, СТУДЕНТЫ, ОБРАБОТКА ДАННЫХ, РАЗРАБОТКА СИСТЕМЫ, ГРУППА, АВТОМАТИЗАЦИЯ ПРОЦЕССА, СИСТЕМА ПОДАЧИ ЗАЯВОК НА ОБУЧЕНИЕ.

Целью курсовой работы является совершенствование образовательного процесса и обработки данных о предоставляемых услугах в школах иностранного языка. Программа позволяет подавать и управлять различными заявками, проходить тестирование, управлять различными данными.

Объект исследования – школы иностранных языков и подходы к автоматизации образовательного процесса.

Предмет исследования – методы и инструментальные средства разработки программного обеспечения для автоматизации учебных и административных процессов школы.

Методология проведения работы: в процессе разработки системы использованы современные подходы к обработке данных, функциональный анализ процессов, моделирование системы с помощью UML-диаграмм.

Результаты работы: рассмотрены основные бизнес-процессы предметной области. Выполнена постановка задачи и определены основные методы ее решения; в ходе моделирования системы построен ряд UML-диаграмм; разработаны модели бизнес-процессов предметной области на основе нотаций IDEF0; описаны основные алгоритмы работы программного средства; разработано руководство пользователя; выполнено тестирование программного средства, показавшее его соответствие функциональным требованиям, поставленным в задании на разработку.

Область применения результатов: в результате выполнения данного курсового проекта была разработана система, которая повысила эффективность работы сотрудников школы иностранного языка за счет автоматизации образовательного процесса, процесса записи на образовательные программы и работы с данными.

ЗАДАНИЕ
по курсовому проекту

ЗАДАНИЕ
по курсовому проекту

СОДЕРЖАНИЕ

Перечень условных обозначений, символов и терминов 6
Введение 7
1 Анализ и моделирование предметной области программного средства 9
1.1 Описание предметной области 9
1.2 Обзор функциональности аналогов программного обеспечения 13
2 Моделирование предметной области и разработка требований к программному средству 17
2.1 Анализ и формализация бизнес-процессов предметной области 17
2.2 Анализ требований к программному средству и разработка их спецификации 25
2.3 Образ предлагаемого решения 29
3 Проектирование и разработка программного средства 32
3.1 Архитектурные решения и технологии реализации программного средства. 32
3.2 Проектирование и разработка пользовательского интерфейса 34
3.3 Разработка модели данных 40
3.4 Описание статических и динамических аспектов поведения программных объектов 57
3.5 Разработка и описание алгоритмов, реализующих бизнес-логику программного средства. 60
3.6 Механизмы обеспечения информационной безопасности 63
4 Тестирование и проверка работоспособности программного средства 64
5 Руководство по установке (развертыванию) и использованию программного средства 69
5.1 Руководство по установке (развертыванию) программного средства 69
5.2 Руководство пользователя 71
5.3 Руководство студента 76
5.4 Руководство преподавателя 79
5.5 Руководство администратора 81
Заключение 92
Список использованных источников 93
Приложение А (обязательное) Отчет о проверке на заимствования в системе «Антиплагиат» 94
Приложение Б (обязательное) Листинг кода алгоритмов, реализующих основную бизнес-логику 95
Приложение В (обязательное) Листинг скрипта генерации базы данных 99
Приложение Г (справочное) Листинг кода JUnit тестов 102
Ведомость документов курсового проекта 104

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ, СИМВОЛОВ И ТЕРМИНОВ

TCP/IP	– сетевая модель передачи данных в цифровом формате описывает процесс доставки информации от отправителя к получателю. Она включает четыре уровня, на каждом из которых действуют свои правила или протоколы передачи данных.
БД (база данных)	– систематизированная совокупность материалов, оформленных в объективной форме, которая позволяет находить и обрабатывать эти данные с использованием электронных вычислительных машин.
MySQL Workbench	– инструмент для визуального проектирования баз данных, объединяющий функции проектирования, моделирования, создания и эксплуатации баз данных в единую интегрированную среду для работы с СУБД MySQL.
Java	– это объектно-ориентированный язык программирования общего назначения, для создания платформонезависимых приложений. Программы на Java компилируются в байт-код, который выполняется на любой системе, имеющей виртуальную машину Java (JVM), обеспечивая высокую портативность, безопасность и масштабируемость.
GUI (Graphical User Interface)	– это графический пользовательский интерфейс, предоставляющий пользователю удобный способ взаимодействия с программным обеспечением через визуальные элементы, такие как окна, кнопки, меню, значки и другие компоненты. В отличие от текстового интерфейса, GUI позволяет выполнять действия с помощью мыши, клавиатуры или сенсорного экрана.
JavaFX	– это платформа для разработки графических пользовательских интерфейсов (GUI) на языке Java. Она предоставляет набор инструментов для создания современных, кросс-платформенных приложений с поддержкой элементов управления, графики, анимации, мультимедиа и взаимодействия с веб-контентом. JavaFX поддерживает модульный подход и интеграцию с FXML для упрощения проектирования интерфейсов.

ВВЕДЕНИЕ

Информационные технологии (ИТ) и английский язык играют ключевую роль в современном мире, оказывая значительное влияние на различные сферы жизни, от бизнеса и науки до образования и личной коммуникации. Эти два фактора неразрывно связаны с процессами глобализации и цифровизации, которые стремительно трансформируют общество. В условиях растущей конкуренции и постоянных изменений школы иностранных языков также сталкиваются с необходимостью адаптации и внедрения передовых технологий для обеспечения более высокого уровня образовательных услуг.

Значение информационных технологий в современных условиях заключается в их способности оптимизировать и автоматизировать процессы, тем самым повышая эффективность управления и взаимодействия с клиентами. В образовательной сфере ИТ играют ключевую роль, предлагая новые возможности для обучения и управления. Автоматизация процессов, таких как запись студентов, ведение расписания и учет успеваемости, позволяет снизить нагрузку на сотрудников и повысить качество обслуживания. Для школы иностранных языков автоматизация становится особенно важной, так как процесс обучения требует точного планирования, своевременной коммуникации и контроля за учебным процессом.

Кроме того, внедрение информационных технологий позволяет не только улучшить организационные процессы, но и расширить возможности дистанционного обучения. Онлайн-платформы и виртуальные классы открывают доступ к учебным материалам в любое время и из любой точки мира, что особенно важно для студентов с плотным графиком или находящихся в разных географических локациях. Такие решения дают возможность школам иностранных языков привлекать студентов из разных стран, тем самым увеличивая охват аудитории и повышая конкурентоспособность на глобальном рынке образовательных услуг. Это делает ИТ-инструменты незаменимыми для современных образовательных учреждений, стремящихся соответствовать требованиям цифровой эпохи.

Одним из главных аспектов работы школ иностранных языков является преподавание английского языка, который давно стал глобальным средством общения и необходимым навыком для карьерного роста. Знание английского открывает доступ к международным образовательным ресурсам и возможностям трудоустройства, что делает его востребованным среди студентов всех возрастов. Школы, внедряющие ИТ-решения для автоматизации своей работы, могут предложить своим студентам более удобные и эффективные способы обучения, соответствующие потребностям современного мира.

Объектом исследования являются школы иностранных языков, а предметом – методы и инструментальные средства разработки программного обеспечения для автоматизации учебных и административных процессов школы.

Цель данного курсового проекта заключается в разработке системы автоматизации для школы иностранных языков. Эта система позволит оптимизировать управление учебными процессами, улучшить взаимодействие с учениками, повысить качество образовательных услуг и сделать обучение более доступным и удобным. Внедрение таких технологий повысит конкурентоспособность школы и позволит ей соответствовать высоким требованиям современного общества, где владение английским языком и использование информационных технологий становятся неотъемлемыми элементами успешного развития.

Для достижения цели по автоматизации работы школы иностранных языков необходимо выполнить следующие задачи:

− провести анализ текущих процессов и выявить проблемы в управлении учебным процессом, расписанием, регистрацией студентов и взаимодействием с преподавателями;

− разработать функциональные требования и модели системы управления учебным процессом, которые включают в себя автоматизацию записи студентов, ведение расписания;

− создать базу данных для хранения информации о студентах, курсах, преподавателях и учебных материалах, а также обеспечить ее структурированность и удобство использования;

− спроектировать и реализовать приложение, который позволит удобно управлять учебным процессом, регистрировать студентов на курсы, планировать расписание и учитывать успеваемость, а также рассчитывать ключевые показатели эффективности учебного процесса;

− провести тестирование разработанного программного продукта для проверки его функциональности, надежности и производительности;

− обеспечить безопасность данных и удобство использования системы для администраторов, преподавателей и студентов, чтобы минимизировать риски утечки информации и облегчить работу пользователей.

Разрабатываемое программное обеспечение должно значительно улучшить процесс управления расписанием, регистрации студентов и учета успеваемости, а также упростить взаимодействие между учениками и преподавателями в рамках учебного процесса.

Курсовой проект был выполнен полностью самостоятельно с соблюдением всех требований, предъявляемых к академическим работам. Для проверки оригинальности текста проект прошел проверку в системе «Антиплагиат», которая подтвердила высокий уровень уникальности работы. Процент оригинальности составил 98,57%, что соответствует установленным критериям и подтверждает, что работа содержит минимальное количество заимствований.

Все заимствованные материалы представлены в виде корректных цитирований, сопровожденных ссылками на соответствующие публикации, которые указаны в разделе «Список использованных источников».

1 АНАЛИЗ И МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ ПРОГРАММНОГО СРЕДСТВА

1.1 Описание предметной области

Школа иностранных языков – это учебное заведение, специализирующееся на преподавании различных иностранных языков с целью совершенствования навыков устной и письменной речи, понимания иноязычных текстов и понимания культуры. Деятельность таких школ обычно основывается на следующих принципах и задачах:

1 Базовое языковое образование: языковые школы обеспечивают качественное обучение основам выбранного языка, включая грамматику, лексику, фонетику и орфографию. На этом этапе у студентов формируются базовые навыки общения на иностранном языке.

2 Развитие коммуникативной компетенции: одной из важнейших задач школы является развитие способности свободно общаться на иностранном языке. Для этого проводятся различные уроки и практические занятия, направленные на развитие навыков аудирования, говорения, чтения и письма.

3 Подготовка к экзаменам: многие языковые школы предлагают подготовку к международным экзаменам, таким как TOEFL, IELTS и DELE на изучаемом языке. Это дает студентам официальное подтверждение их знаний и повышает их шансы на поступление в зарубежные вузы и продвижение по карьерной лестнице.

4 Персонализация обучения: языковые школы стремятся предложить индивидуальные методы обучения, учитывая уровень знаний, потребности и цели каждого студента. Для этого используются различные методы и технологии, а также индивидуальные занятия с преподавателями.

5 Приобщение к культуре страны изучаемого языка: языковые школы часто организуют мероприятия, направленные на погружение студентов в культуру страны изучаемого языка. К ним можно отнести культурные мероприятия, встречи с носителями языка и экскурсии.

Школы иностранных языков играют важную роль в развитии языковых навыков и межкультурного взаимопонимания студентов, их личностном развитии и успешной интеграции в международное сообщество.

Ни для кого не секрет, что знание иностранного языка, а лучше нескольких, является шикарной перспективой для дальнейшей самореализации человека: профессиональной, творческой и т.п. Поэтому большинство современных родителей, понимая значимость владения иностранными языками, уделяют особое внимание тому, чтобы их дети с малых лет начали изучать иностранный язык.

На сегодняшний день существует масса языковых школ, где обучение проходят не только дети, но и взрослые [1]. На диаграмме, рисунок 1.1, представлены наиболее популярные иностранные языки для изучения.

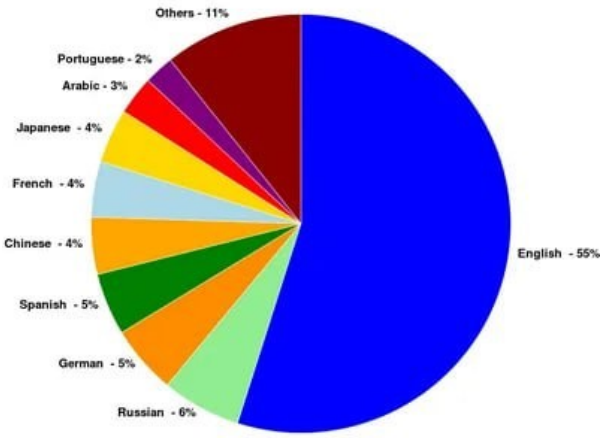


Рисунок 1.1 – Диаграмма наиболее популярных языков для изучения.

Самыми популярными языками являются английский, русский и немецкий языки. Английский и русский языки изучаются как язык международной коммуникации, а немецкий язык – для обучения в техникумах и университетах.

На рисунке 1.2 показана польза от изучения иностранного языка.



Рисунок 1.2 – Польза от изучения иностранного языка

Помимо того, что иностранный язык помогает в карьерной самореализации, необходим для обучения в иностранных учебных заведениях и трудоустройства за границей, знание иностранного языка способствует развитию мозга человека.

Учет деятельности школы иностранных языков включает в себя несколько ключевых аспектов.

1 Учет учебного процесса.

Планирование курсов. Определение расписания занятий, форматов (групповые, индивидуальные) и уровней (начальный, средний, продвинутый). Разработка учебных планов и программ для различных курсов.

Учет посещаемости. Ведение журналов посещаемости для отслеживания присутствия студентов на занятиях. Анализ причин пропусков и разработка мер по их снижению.

Оценка успеваемости. Проведение регулярных тестов и контрольных работ для оценки прогресса студентов. Ведение базы данных с результатами успеваемости и обратной связью от преподавателей.

2 Финансовый учет.

Доходы. Оплата за курсы: учет поступлений от студентов за обучение. Важно фиксировать даты оплаты, суммы и способы оплаты (наличные, банковские карты, электронные платежи). Дополнительные услуги: учет доходов от дополнительных услуг, таких как индивидуальные занятия, подготовка к экзаменам, семинары и мастер-классы.

Расходы. Заработная плата: учет выплат преподавателям, административному персоналу и другим сотрудникам. Аренда помещения: фиксация расходов на аренду классов и офисных помещений. Коммунальные услуги: учет затрат на электроэнергию, воду, интернет и другие услуги. Реклама и маркетинг: расходы на продвижение школы, включая онлайн-рекламу, печатные материалы и мероприятия. Учебные материалы: закупка учебников, методических пособий, канцелярских товаров.

Финансовая отчетность. Подготовка ежемесячных и годовых отчетов о доходах и расходах. Анализ финансовых показателей для оценки рентабельности и эффективности работы школы.

3 Управление кадрами.

Набор преподавателей. Процесс подбора квалифицированных специалистов: размещение вакансий, проведение собеседований, проверка рекомендаций. Оценка квалификации и опыта преподавателей.

Обучение и развитие. Организация тренингов и семинаров для повышения квалификации преподавателей. Создание системы наставничества для новых сотрудников.

Учет рабочего времени. Ведение учета рабочего времени преподавателей и административного персонала. Определение коэффициентов нагрузки для преподавателей в зависимости от количества часов занятий.

4 Маркетинг и реклама.

Анализ рынка. Исследование конкурентов и потребностей целевой аудитории. Определение уникальных предложений (USP) школы.

Рекламные кампании. Разработка рекламных материалов (брошюры, флаеры, баннеры). Использование онлайн-платформ (социальные сети, сайт школы) для привлечения студентов.

Обратная связь. Сбор отзывов от студентов о качестве обучения и уровне сервиса. Проведение опросов для улучшения учебного процесса и удовлетворенности клиентов.

5 Административный учет.

Документация. Ведение необходимых документов: договоров с учениками, актов выполненных работ, счетов-фактур. Хранение документации в соответствии с законодательными требованиями.

Управление качеством. Разработка стандартов качества обучения и обслуживания. Проведение регулярных проверок и аудитов для обеспечения соответствия стандартам.

Процесс управления учебным процессом и взаимодействия с учащимися в школах иностранных языков является важным аспектом организации работы учебного заведения, обеспечивая поддержку студентов и стимулируя их к достижению высоких академических результатов. Этот процесс включает сбор, обработку и анализ данных о студентах, их успеваемости, посещаемости и других показателях, что требует использования современных технологий и эффективных методов управления информацией. Также учитываются различные категории студентов, включая тех, кто учится по специальным программам, занимается индивидуально или посещает группы с разным уровнем подготовки, что позволяет более гибко подходить к организации учебного процесса.

Автоматизация этого процесса не только снижает нагрузку на сотрудников школы, но и обеспечивает прозрачность и доступность информации для всех участников образовательного процесса – как для студентов, так и для преподавателей. Основная цель автоматизации – обеспечить эффективное управление расписанием, регистрацией студентов и учетом их успеваемости, что способствует повышению качества образовательных услуг и созданию комфортных условий для обучения.

Подсистема автоматизированного управления учебным процессом должна соответствовать следующим требованиям:

− обеспечивать точное и своевременное ведение расписания и учет успеваемости студентов;

− учитывать все необходимые данные, включая посещаемость, результаты тестов и контрольных работ;

− минимизировать ошибки при обработке данных;

− поддерживать взаимодействие с различными подразделениями школы, включая учебные отделы и администрацию;

− обеспечивать прозрачность процесса управления учебным процессом и удобство использования как для сотрудников школы, так и для студентов.

На сегодняшний день специализированное программное обеспечение для автоматизации работы школ иностранных языков представлено ограниченно, при этом основное внимание уделяется системам управления расписанием и успеваемостью студентов. Разрабатываемая система должна обеспечить максимальную полноту автоматизации, включая учет академической успеваемости, посещаемости и управление расписанием, что позволит повысить эффективность работы школы и снизить вероятность ошибок [2].

1.2 Обзор функциональности аналогов программного обеспечения

Для учета деятельности школы иностранных языков можно рассмотреть три популярных программных решения и выявить их недостатки, которые могут быть устранены в разработанной программе.

Суть сравнительного анализа разрабатываемой системы автоматизации заключается в исследовании функциональных возможностей, преимуществ и ограничений платформ Moodle, CRM-системы Bitrix24 и учебного центра "1С:Образование" для определения их применимости в автоматизации работы школы иностранных языков. Целью анализа является выявление ключевых тенденции в развитии образовательных технологий и определении инструментов, которые наиболее эффективны для управления учебными курсами и расписанием, взаимодействия с учениками и преподавателями, анализа академических и бизнес-показателей, автоматизации рутинных процессов (запись на курсы, оплата, отчетность).

Указанные аналоги представлены в таблице 1.1.

Таблица 1.1 – Аналоги программного средства

Название аналога	Компания-разработчик	URL официального сайта
Moodle	Moodle Pty Ltd	https://moodle.com
Bitrix24	Bitrix, Inc.	https://www.bitrix24.ru
1С:Образование	Фирма "1С"	https://education.1c.ru

1.2.1 **Moodle.** Логотип представлен на рисунке 1.3.



Рисунок 1.3 – Система дистанционного обучения Moodle

Moodle – многофункциональная СДО со всеми возможностями для обучения и аттестации слушателей курса. Условно-бесплатная версия подойдет тем, у кого есть технические навыки или есть возможность найма соответствующих специалистов. Кроме того, пользователям серверной версии нужно знать, что запустить курс быстро не получится. Установка Moodle может занять больше месяца.

С облачной версией работать проще. Серьезные технические навыки не нужны, а за работоспособность системы отвечают разработчики Moodle. Минус этой СДО в том, что она платная, но при этом может оказаться

недостаточно удобной для пользователей. Например, из-за хранения информации на иностранном сервере. Или недостаточного количества шаблонов, сложного интерфейса [3].

1.2.2 CRM-система “Bitrix24”. Логотип представлен на рисунке 1.4.

Программа Битрикс 24 объединила современные разработки в области взаимодействия с клиентами и ведения аналитики. Это гибкая и эффективная CRM (расшифровывается как Customer Relationship Management) – система, помогающая контролировать все каналы коммуникации с заказчиками и продажи. Один сервис позволяет собирать и хранить данные о клиентах, отслеживать работу менеджеров, анализировать эффективность разных каналов рекламы и решать десятки других задач [4].



Рисунок 1.4 – CRM-система “Bitrix24”

1.2.3 Учебный центр "1С:Образование".
Логотип представлен на рисунке 1.5.

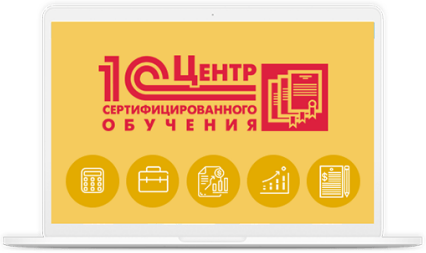


Рисунок 1.5 – Учебный центр "1С:Образование"

Учебный центр "1С:Образование" – это комплексное решение для автоматизации образовательного процесса в школах, колледжах и вузах. Основное назначение системы – организация обучения с использованием электронных образовательных ресурсов. Учителя, учащиеся и родители могут работать с системой в локальной сети школы или дистанционно через интернет. С системой можно работать как при помощи обычных компьютеров и ноутбуков, так и мобильных устройств – планшетов и смартфонов [5].

Анализ и описание программных аналогов проводятся для выявления ключевых преимуществ и недостатков существующих решений на рынке. Это позволяет оценить их функциональные возможности и ограничения, определить рыночные запросы, выявить пробелы, которые можно устранить в разрабатываемой системе для языковой школы.

Таблица 1.2 демонстрирует, как конкурирующие платформы реализуют задачи управления курсами, взаимодействия с пользователями и аналитики. На основе этого сравнения можно определить, какие функции требуют доработки или внедрения, чтобы предложить более эффективный продукт.

Таблица 1.2 – Функциональность аналогов

Характеристика	Moodle	Bitrix24	1С:Образование
Основные пользователи	Учебные заведения, корпоративные учебные центры	Бизнес-организации, учебные центры	Школы, ВУЗы, учебные центры
Типы курсов	Онлайн, гибридные	Корпоративное обучение, тренинги	Онлайн, офлайн, смешанные
Управление студентами	Зачисление, отслеживание прогресса	Управление контактами (CRM)	Полный цикл: запись, обучение, аттестация
Интеграция	Плагины (Zoom, Google), LTI	Электронная почта, календари, телефония	Интеграция с продуктами 1С, электронный дневник
Инструменты коммуникации	Форумы, чаты, блоги	Чат, видеозвонки, задачи	Форум, уведомления
Аналитика и отчетность	Базовые отчеты, плагины для расширения	Бизнес-аналитика, отчеты по эффективности	Детальная успеваемость, финансовая отчетность
Стоимость	Бесплатно (Open Source), платная поддержка	Подписка (есть бесплатный тариф)	Лицензия, зависит от конфигурации
Гибкость настройки	Высокая (открытый исходный код)	Средняя (настройка модулей)	Высокая (интеграция с 1С)
Мобильное приложение	Да (Moodle App)	Да (Bitrix24 Mobile)	Ограниченная функциональность

Углубленное исследование существующих решений (Moodle, Bitrix24, 1С:Образование) выявило, что их функциональные возможности не полностью соответствуют специфическим требованиям языковой школы. На основе результатов анализа в таблице 1.3 представлен проектируемый функционал системы автоматизации, который призван устранить выявленные пробелы, усилить слабые стороны аналогов, внедрить уникальные функции, ориентированные на нужды школы.

Это позволяет создать продукт, который не только конкурирует с существующими решениями, но и предлагает инновационные подходы для образовательных учреждений.

Таблица 1.3 – Функциональность программного средства

Функции	Пользователь	Студент	Преподаватель	Администратор
Регистрация	Да	Нет	Нет	Нет
Вход в систему	Да	Да	Да	Да
Просмотр личного кабинета	Да	Да	Да	Да
Редактирование информации в личном кабинете	Нет	Да	Да	Нет
Удаление аккаунта	Нет	Да	Да	Нет
Просмотр каталога курсов	Да	Да	Да	Да
Просмотр прайс-листа	Да	Да	Да	Да
Управление (редактирование) ценами	Нет	Нет	Нет	Да
Просмотр отзывов	Да	Да	Да	Да
Создание отзыва	Нет	Да	Да	Нет
Удаление своих отзывов	Нет	Да	Да	Нет
Удаление любых отзывов	Нет	Нет	Нет	Да
Подача заявки на курс	Нет	Да	Нет	Нет
Подача заявки на зачисление/преподавание	Да	Нет	Нет	Нет
Управление заявками (просмотр/редактирование)	Нет	Нет	Нет	Да
Прохождение тестирования	Нет	Да	Нет	Нет
Просмотр результатов тестирования	Нет	Нет	Да	Нет
Просмотр общего расписания	Нет	Да	Да	Да
Просмотр своего расписания	Нет	Да	Да	Нет
Управление расписанием	Нет	Нет	Нет	Да
Просмотр своих курсов	Нет	Нет	Да	Нет
Управление (создание/редактирование) курсами	Нет	Нет	Нет	Да
Просмотр состава своих групп	Нет	Нет	Да	Нет
Управление группами	Нет	Нет	Нет	Да
Управление пользователями	Нет	Нет	Нет	Да

Создание программы для учета деятельности школы иностранных языков требует внимательного подхода к существующим решениям. Изучение недостатков программ позволит разработать более эффективный и удобный инструмент, который окажется полезным как для преподавателей, так и для учащихся.

2 МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И РАЗРАБОТКА ТРЕБОВАНИЙ К ПРОГРАММНОМУ СРЕДСТВУ

2.1 Анализ и формализация бизнес-процессов предметной области

Для проектирования и оптимизации процесса регистрации клиентов в системе автоматизации школы иностранных языков используется BPMN-диаграмма [6]. Она визуализирует взаимодействие между клиентом и администратором, выделяет этапы обработки данных, а также точки принятия решений, что позволяет выявить узкие места и улучшить пользовательский опыт. На рисунке 2.1 представлена диаграмма процесса, отражающая логику от подачи заявки до завершения регистрации.

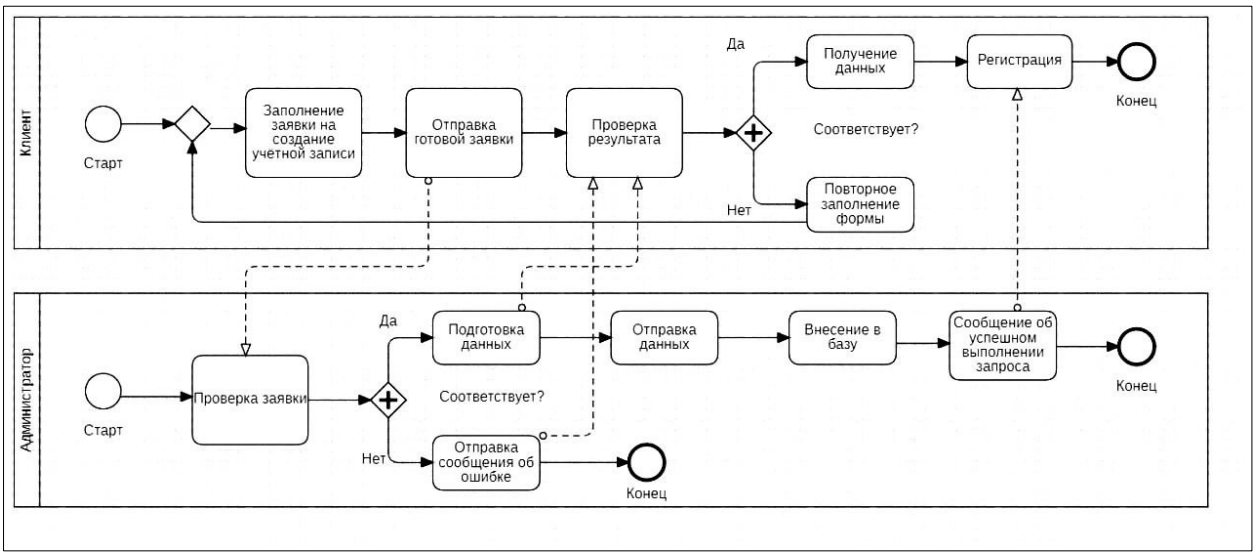


Рисунок 2.1 – BPMN диаграмма бизнес-процесса «Регистрация клиента»

Бизнес-процесс «Регистрация клиента в системе автоматизации школы иностранных языков» начинается с того, что клиент оформляет заявку на обучение, заполняя электронную форму с персональными данными (ФИО, контактная информация, уровень владения языком и предпочтения по расписанию). После заполнения клиент отправляет заявку в систему. Заявка поступает администратору, который обрабатывает её, проверяя корректность данных и наличие свободных мест в выбранной группе.

Если данные клиента соответствуют требованиям, администратор подтверждает заявку, создает учетную запись в системе и вносит информацию о клиенте в базу данных школы. В случае обнаружения ошибок (неполные данные, некорректный формат контактов, отсутствие мест в группе) администратор отправляет клиенту уведомление с указанием проблемных полей. Клиент получает запрос на исправление, корректирует форму и повторно отправляет её на проверку.

После успешной проверки администратор формирует логин и пароль для доступа к личному кабинету, после чего клиент получает уведомление об успешной регистрации с инструкциями для входа в систему. Клиент подтверждает получение данных, завершая процесс.

Финальным этапом является активация учетной записи клиента и его доступ к учебным материалам, расписанию и другим ресурсам школы.

В процессе задействованы ключевые точки взаимодействия такие как отправка заявки клиентом и обработка администратором, уведомление о результате проверки и действия клиента (исправление или завершение регистрации).

Данная BPMN-диаграмма наглядно демонстрирует:

1 Последовательность этапов –от заполнения формы до получения доступа к системе.

2 Ветвления процесса –шлюзы (Gateway) разделяют сценарии на «успешную регистрацию» и «исправление ошибок».

3 Параллельные задачи администратора –проверка данных, создание аккаунта, интеграция с базой данных.

4 Автоматизацию рутинных операций –уведомления, валидация данных.

Использование BPMN позволяет оптимизировать процесс за счет:

– сокращения времени обработки заявок через автоматическую валидацию полей;

– минимизации ошибок благодаря четким правилам проверки;

– улучшения коммуникации между клиентом и администратором через мгновенные уведомления.

Таким образом, визуализация процесса регистрации помогает школе иностранных языков повысить эффективность работы с клиентами, обеспечить прозрачность этапов и снизить нагрузку на административный персонал.

Функциональная модель – функциональная структура объекта предметной области. Она отражает производимые процессы, действия и связи между ними. Для разработки функциональной модели необходимо разбить главный процесс на несколько подпроцессов для анализа системы.

Для графического отображения всех происходящих в ИС процессов используем нотацию IDEF0 [7].

Стандарт IDEF0 – нотация графического моделирования, используемая для создания функциональной модели, отображающей структуру и функции системы, а также потоки информации и материальных объектов, связывающих эти функции. Нотация IDEF0 является одной из самых популярных нотаций моделирования бизнес-процессов.

На диаграмме верхнего уровня A0 (рисунок 2.2) представлена функциональная модель «Записать ученика на курс иностранного языка», а также определены потоки входных и выходных данных, механизмы ограничения и управления данными.

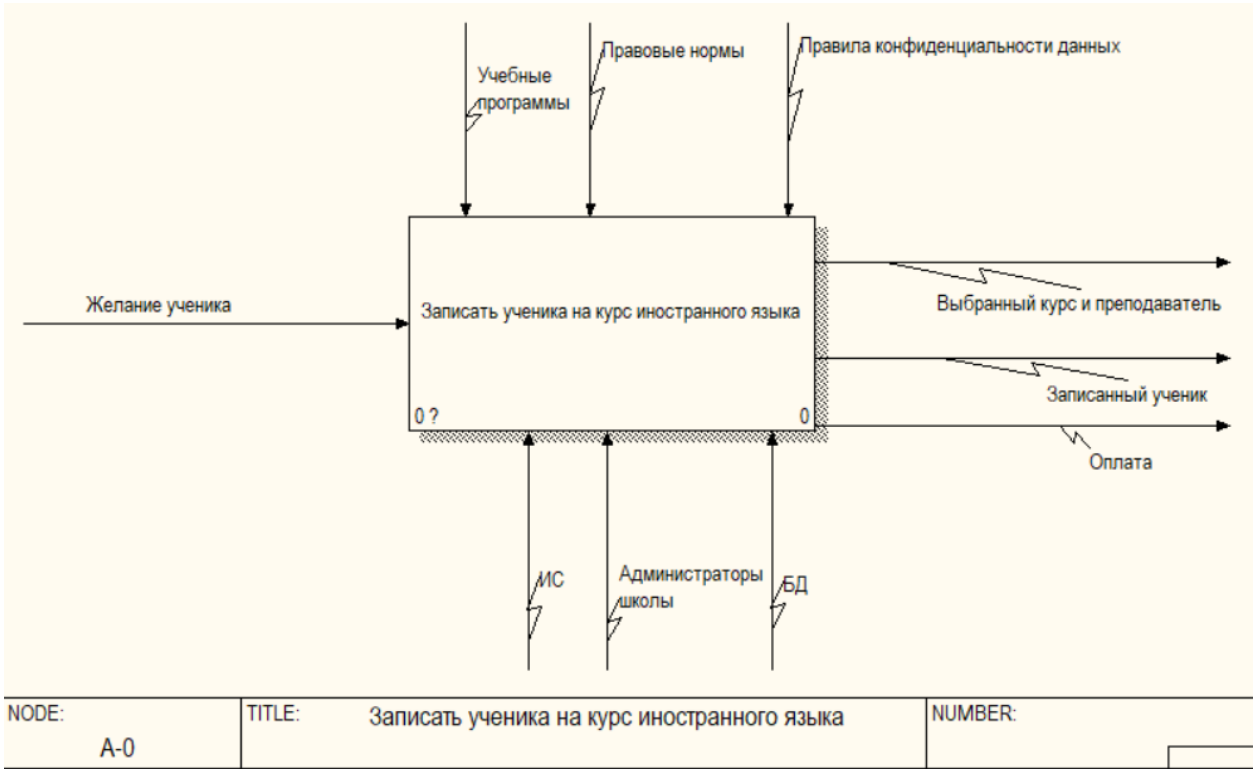


Рисунок 2.2 – Контекстная диаграмма верхнего уровня

В качестве входных параметров в данной модели выступает желание ученика обучаться. Эти данные необходимы для запуска и корректного функционирования процесса, направленного на успешную запись ученика на курс.

В результате выполнения процесса достигаются следующие выходные параметры: записанный ученик, выбранный курс и назначенный преподаватель, а также оплата, что обеспечивает полную готовность ученика к началу учебного процесса. Для более глубокого анализа и понимания структуры основного процесса была проведена декомпозиция контекстного уровня, которая позволила разделить весь процесс на четыре отдельных подпроцесса, упрощая анализ и управление каждым из них.

На рисунке 2.3 представлена декомпозиция контекстной диаграммы, где выделено четыре уровня для детального рассмотрения всех этапов выполнения задачи.

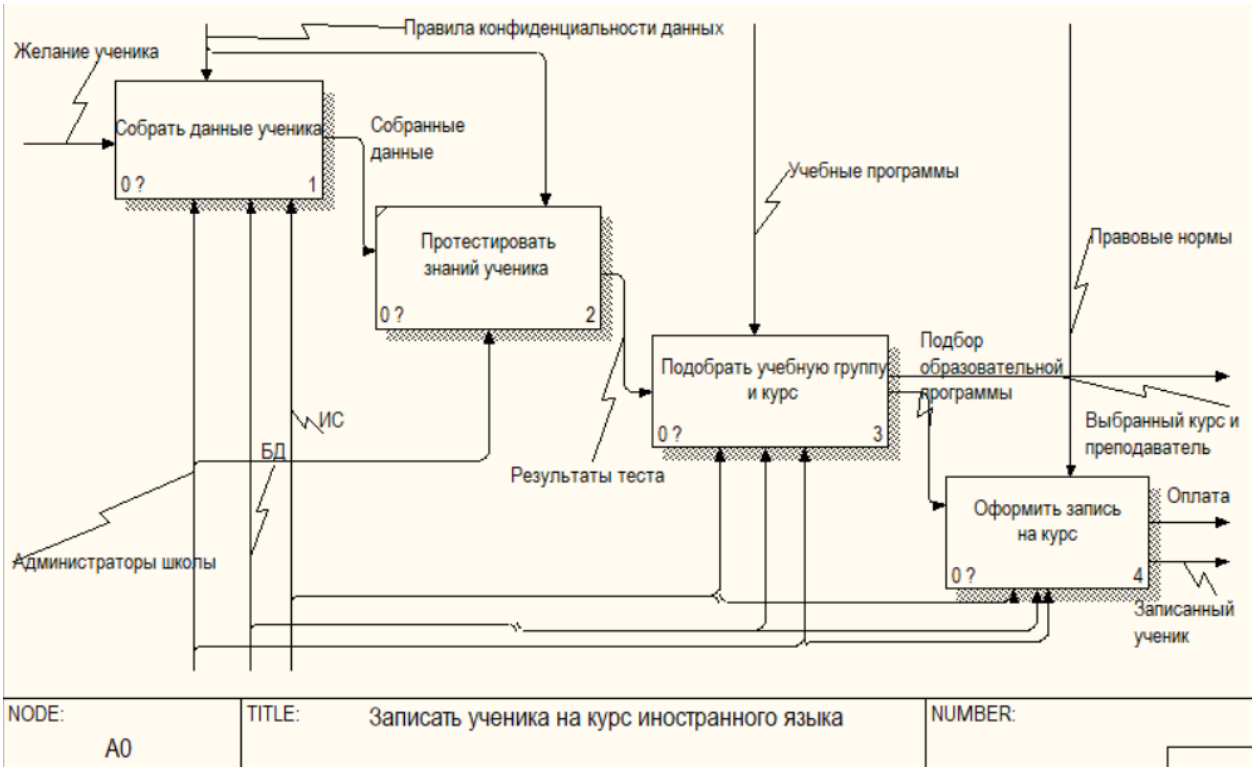


Рисунок 2.3 – Декомпозиция контекстной диаграммы верхнего уровня

На выходе была получена диаграмма, состоящая из четырёх процессов: «Собрать данные ученика», куда поступают личные данные и желание ученика; «Тестирование знаний ученика», где осуществляется проверка знаний учащегося; «Подбор учебной группы и курса», в рамках которого производится поиск и подбор учебной группы и курса на основе результатов тестирования; и «Оформление записи на курс», где после подбора группы и курса осуществляется оформление записи ученика на курс иностранного языка.

Декомпозиция процесса «Собрать данные ученика» представлена ниже на рисунке 2.4, на котором можно более детально рассмотреть шаги, необходимые для сбора всей необходимой информации.

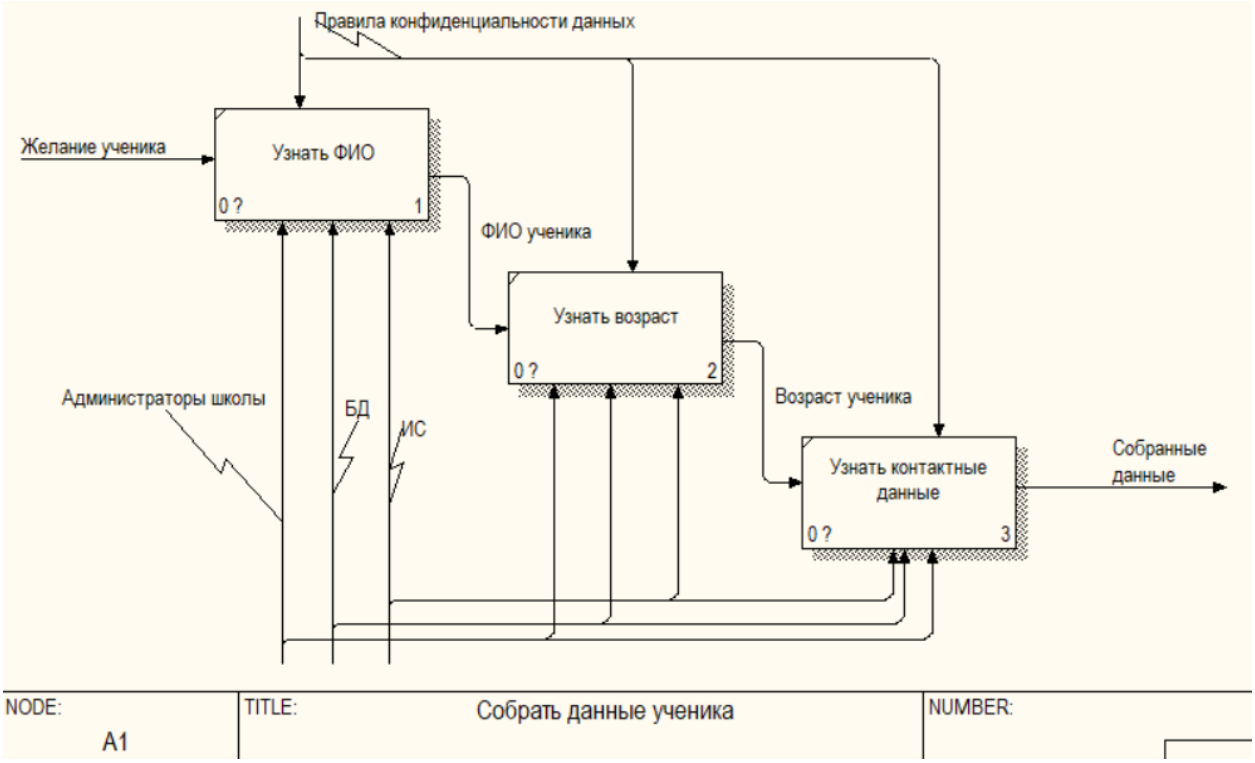


Рисунок 2.4 – Декомпозиция блока «Собрать данные ученика»

В данном блоке осуществляется сбор всех данных учащегося. Сначала происходит получение базовой информации об учащемся, такой как фамилия, имя и отчество, что позволяет идентифицировать его в системе и обеспечить индивидуальный подход. После этого собираются дополнительные данные, включая возраст и контактные данные, такие как номер телефона и адрес электронной почты.

По завершении данного блока у нас имеются готовые данные об учащемся. Этот блок является неотъемлемой частью всей схемы, так как для корректного осуществления записи на курс иностранного языка необходимо сначала провести полный сбор и подготовку всех требуемых данных, обеспечив их точность и полноту. Декомпозиция подпроцесса «Тестирование знаний ученика» представлена на рисунке 2.5, где подробно отображены этапы, направленные на оценку уровня знаний учащегося и выявление его текущих навыков, что, в свою очередь, помогает определить наиболее подходящий курс и учебную группу.

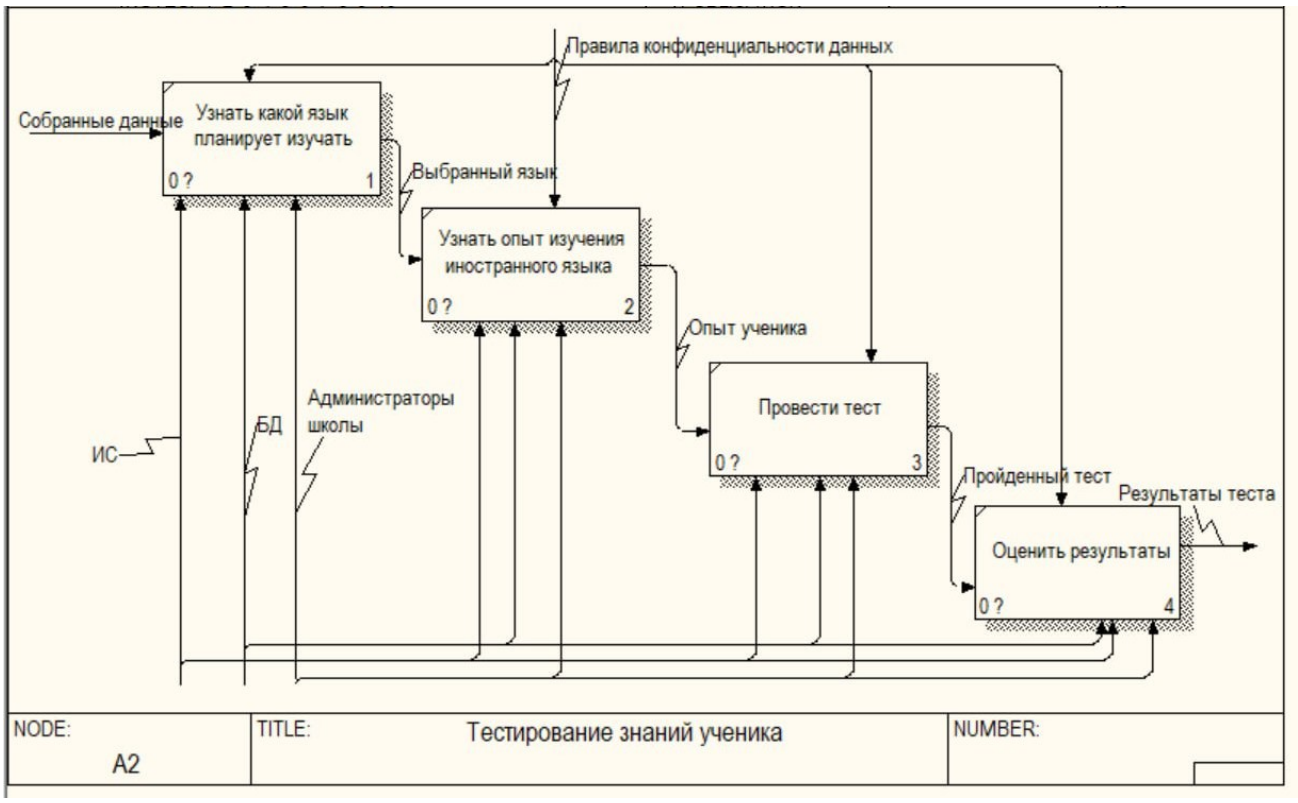


Рисунок 2.5 – Декомпозиция блока «Тестирование знаний ученика»

В данном блоке, на основе предварительно собранных данных, определяется, какой именно язык студент планирует изучать, что является важным шагом для дальнейшего процесса. После этого, на основании выбранного языка, выясняется его предыдущий опыт изучения данного языка, что помогает в корректной оценке уровня знаний. Далее осуществляется тестирование учащегося, включающее выполнение заданий различного уровня сложности, и проводится оценка его результатов. Это необходимо для определения текущего уровня владения языком и для формирования рекомендаций по учебной группе.

Результаты тестирования служат основой для последующего подбора подходящего курса, чтобы максимально соответствовать потребностям и уровню ученика. Данный блок является необходимым этапом, так как полученные результаты теста позволяют начать подбор учебной группы и курса, обеспечивая максимально эффективное обучение для каждого учащегося. Следующий этап – декомпозиция процесса «Подбор учебной группы и курса» (рисунок 2.6), где будет рассмотрен механизм определения оптимальной учебной группы и курса для каждого учащегося.

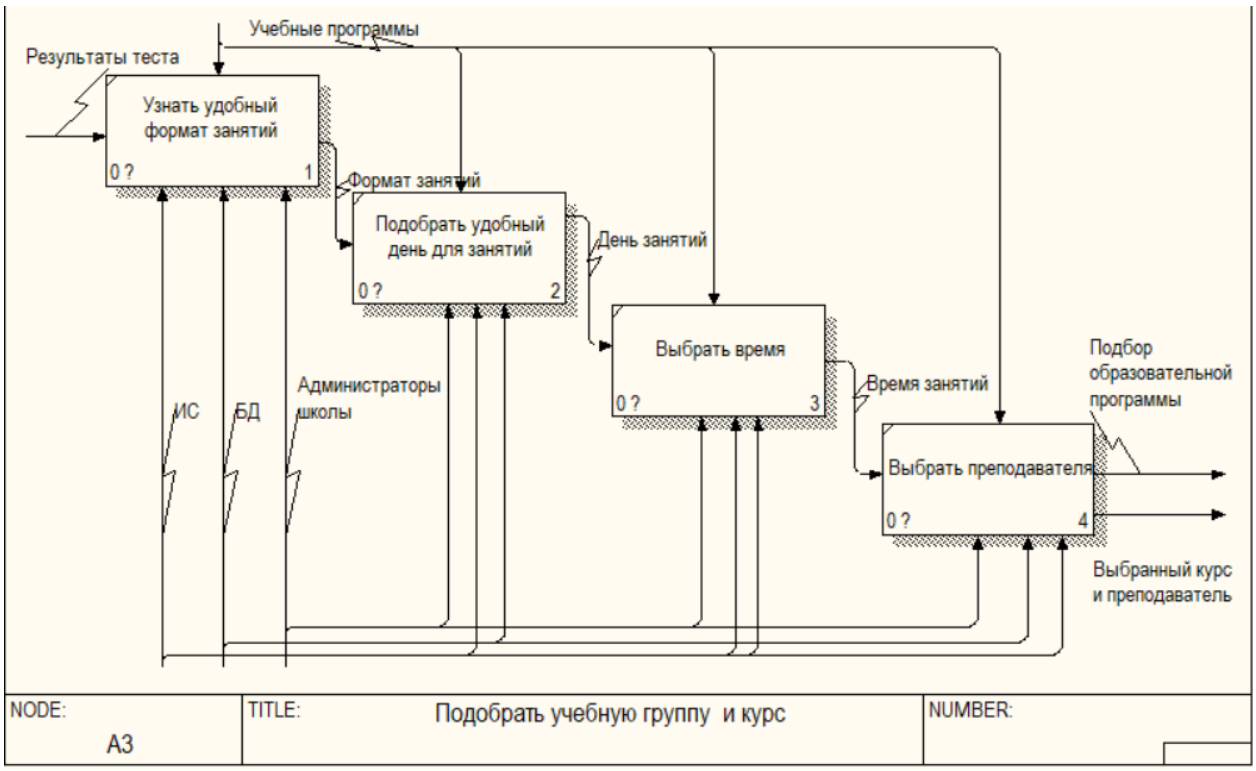


Рисунок 2.6– Декомпозиция блока «Подбор учебной группы и курса»

В данном блоке, исходя из результатов тестирования, определяется удобный формат занятий для учащегося: очный, дистанционный или смешанный. Затем подбирается подходящий день для занятий, что позволяет учесть индивидуальные предпочтения и занятость учащегося. Следующим этапом является выбор удобного времени для проведения занятий, чтобы обеспечить максимально комфортные условия для обучения. На завершающем этапе происходит выбор преподавателя, основываясь на уровне знаний учащегося и его учебных предпочтениях.

Этот блок играет ключевую роль в нашей схеме, так как здесь совершаются все основные действия, направленные на подбор учебной группы и курса, что позволяет сформировать наиболее подходящие условия для успешного обучения. После этого можно перейти к декомпозиции процесса «Оформление записи на курс» (рисунок 2.7), чтобы рассмотреть окончательные шаги по оформлению учащегося на выбранный курс.

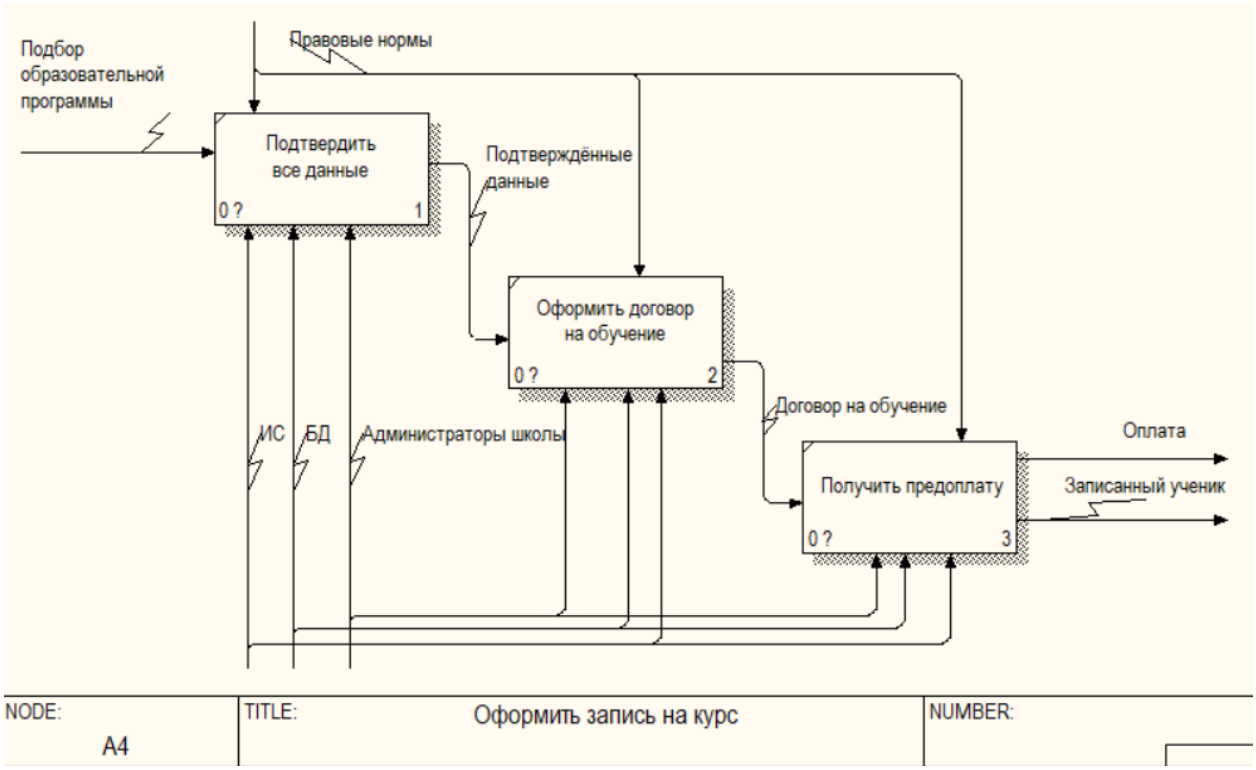


Рисунок 2.7 – Декомпозиция блока «Оформление записи на курс»

В данном блоке происходит непосредственно само оформление записи учащегося на курс иностранного языка. Основываясь на ранее выполненном подборе образовательной программы, необходимо подтвердить все данные, чтобы убедиться в их корректности и полноте. Затем осуществляется оформление договора на обучение, в котором фиксируются условия обучения, включая стоимость, продолжительность курса и выбранные форматы занятий. После этого происходит получение предоплаты за курс, что завершает процесс оформления. По окончании всех этих этапов учащийся считается успешно записанным на выбранный курс иностранного языка.

На рисунке 2.8 представлена диаграмма, демонстрирующая декомпозицию процесса «Записать ученика на курс иностранного языка», которая наглядно показывает все ключевые шаги и взаимодействия в процессе оформления записи.

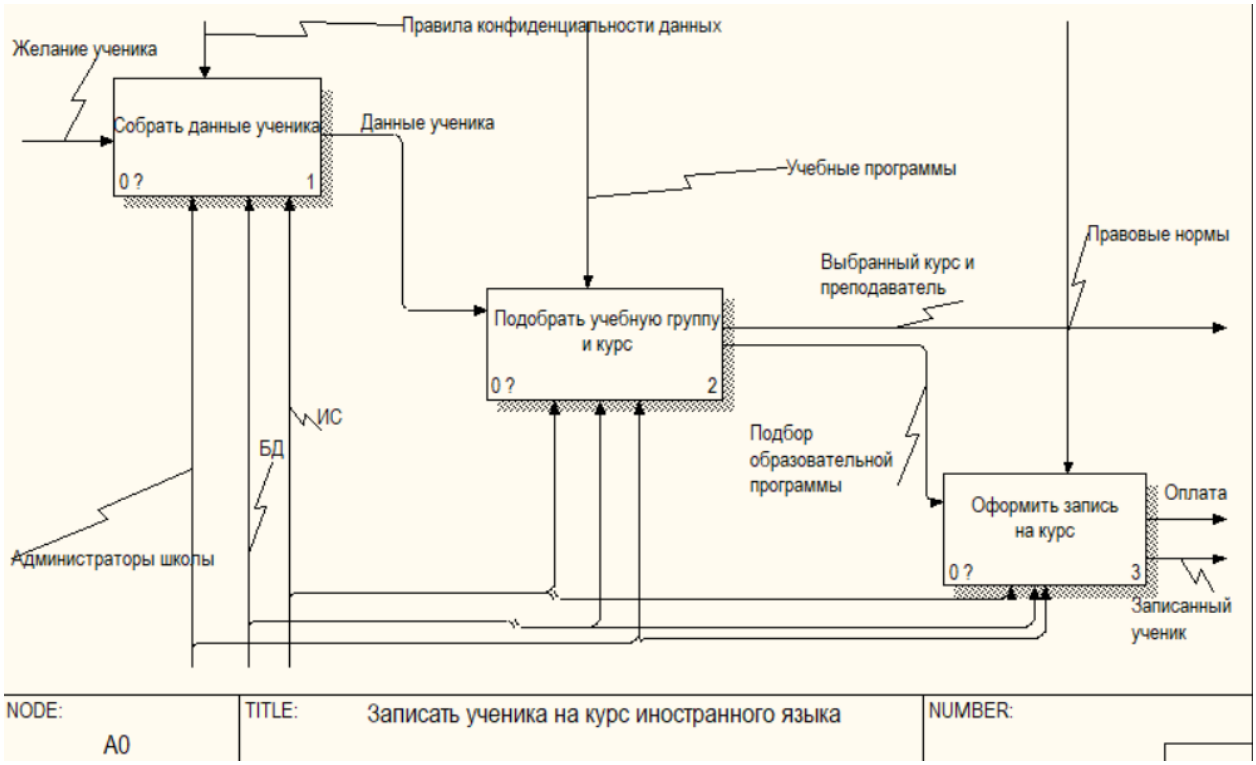


Рисунок 2.8 – Декомпозиция блока «Записать ученика на курс иностранного языка»

Остальные декомпозиции нашего основного процесса остаются неизменными и представлены выше.

Данный подход моделирования позволяет наглядно видеть, на каком этапе, какие управляющие элементы и какие механизмы задействованы. Таким образом, в данной главе был рассмотрен процесс записи ученика на курс иностранного языка путем моделирования бизнес-процессов с использованием нотации IDEF0.

В результате всех процессов данная организация сможет осуществить автоматизированный процесс записи учеников на курсы иностранного языка.

2.2 Анализ требований к программному средству и разработка их спецификации

В основе любой программной системы, а также процесса ее разработки, лежат требования к программному обеспечению. Эти требования формируются на основе анализа бизнес-процессов, задач и потребностей пользователей системы. Разработка системы автоматизации работы школы иностранных языков также начинается с анализа требований, где ключевую роль играют взаимодействия пользователей с системой.

Разрабатываемое приложение для школы иностранных языков должно соответствовать ряду требований. Необходимо предусмотреть разделение прав доступа для учеников, преподавателей и администраторов, с ограничением их полномочий. Доступ к функционалу будет предоставляться

только после успешной аутентификации в системе. Важным аспектом является наличие обработки исключительных ситуаций (например, некорректный ввод данных), обратная связь с пользователем и навигация с возможностью возврата на предыдущие шаги. Также необходимо обеспечить, чтобы имена переменных и функций были логичными и отражали их назначение.

Для реализации и проектирования системы автоматизации работы школы иностранных языков будут использованы следующие технологии:

– для серверной части: язык программирования Java, с поддержкой многопоточного обслуживания запросов;

– для клиентской части: язык Java и библиотека для создания пользовательского интерфейса JavaFX;

– для клиент-серверного взаимодействия: использование сокетов для передачи данных через потоки ввода-вывода;

– для хранения данных: реляционная база данных MySQL;

– для подключения к базе данных и управления ею: стандарт взаимодействия Java-приложений с реляционными базами данных JDBC.

Эти технологии позволят создать гибкую и масштабируемую систему, способную автоматизировать основные бизнес-процессы школы иностранных языков, включая запись студентов, управление расписанием, подбор преподавателей и контроль результатов тестирований.

Приложение должно предоставлять пользователю удобные и, главное, понятные формы для ввода информации. Интерфейс должен быть интуитивно понятным и содержать привычные элементы, такие как таблицы, списки и текстовые поля, для упрощения взаимодействия пользователей с системой. Все операции на серверной стороне, включая обработку данных о студентах, преподавателях, расписании и успеваемости, должны выполняться автоматически, без необходимости вмешательства сотрудников школы.

На основе диаграммы вариантов использования (рисунок 2.9) можно выделить ключевые требования к системе [8].

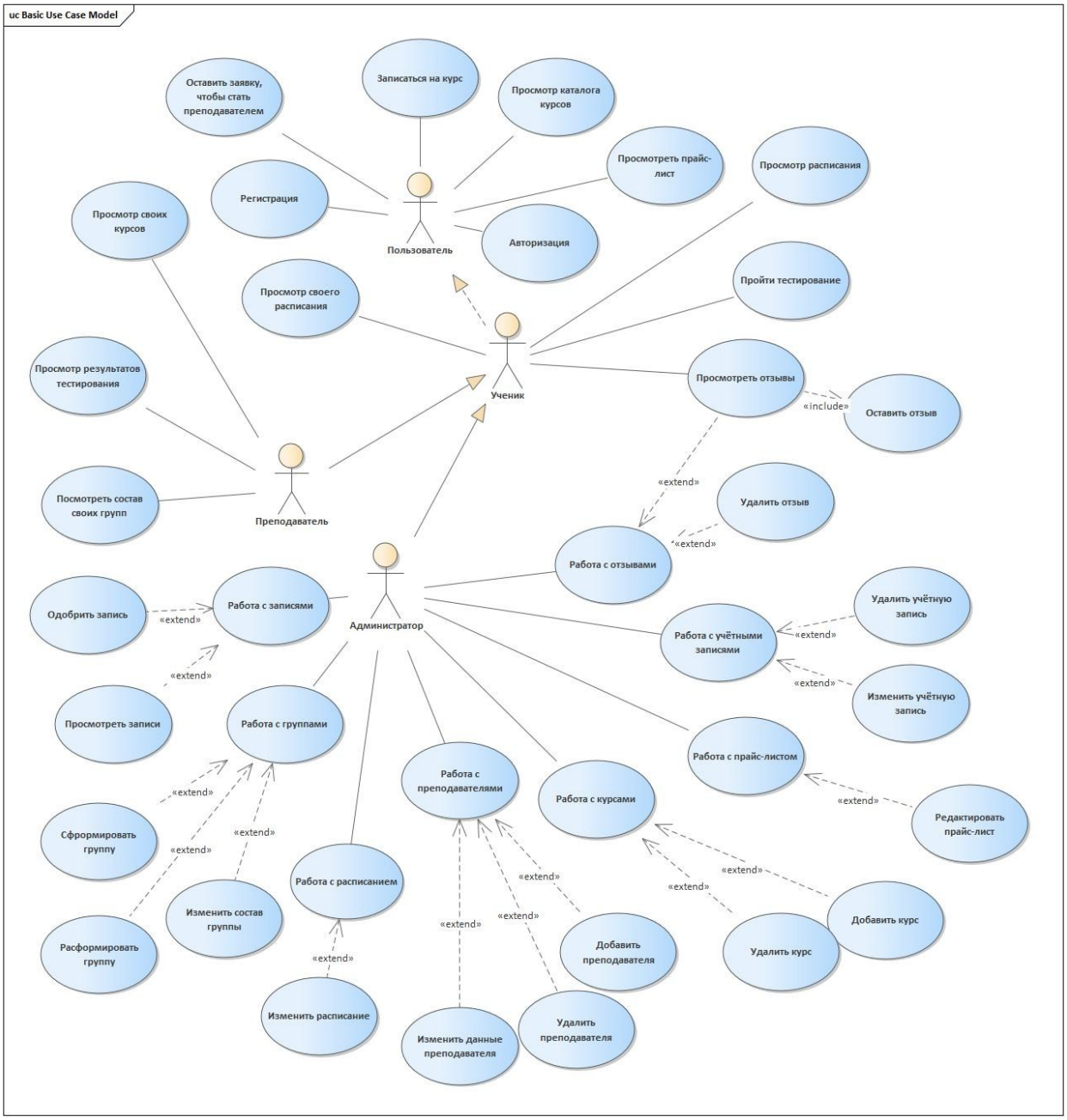


Рисунок 2.9 – Диаграмма вариантов использования

С данной программой должны работать следующие группы пользователей:

– пользователи;

– студенты;

– администраторы;

– преподаватели.

Необходимо учесть, что к системе может обратиться человек, который ранее не взаимодействовал с системой. В таком случае ему необходимо пройти регистрацию, вследствие которой он автоматически получит роль «пользователь».

Все группы пользователей могут осуществлять вход в приложение. На основании их данных будет определяться роль по записи в БД и, соответственно, отображаемое в приложении содержимое.

Модуль администратора включает следующие возможности:

– просмотр, редактирование и удаление компонентов заявок;

– просмотр и удаление отзывов пользователей;

– работу с различными аккаунтами;

– работу с прайс-листом;

– работу с расписанием;

– работу с курсами;

– работу с группами;

– просмотр личного кабинета.

Модуль студента включает в себя следующие возможности:

– просмотр прайс-листа;

– пройти тестирование;

– просмотреть, оставить и удалить отзыв;

– просмотреть расписание курсов в том числе и своё;

– просмотреть каталог курсов;

– оставить заявку на курс

– просмотр личного кабинета с возможностью редактирования личной информации и удалением аккаунта.

Модуль преподавателя включает в себя:

– просмотр прайс-листа;

– просмотреть, оставить и удалить отзыв;

– просмотреть расписание курсов в том числе и своё;

– просмотреть каталог курсов;

– просмотр личного кабинета с возможностью редактирования личной информации и удалением аккаунта;

– просмотр результатов тестирования;

– просмотр состав своих групп;

– просмотр своих курсов.

Модуль пользователя включает в себя возможность:

– оставить заявку на зачисление или на возможность преподавать;

– просмотреть каталог курсов;

– просмотр прайс-листа;

– просмотр отзывов.

Таким образом была разработана диаграмма вариантов использования, которая описывает функциональное назначение системы или, иными словами, то, что система будет делать в процессе своего функционирования.

2.3 Образ предлагаемого решения.

Бизнес-проблемы. В ходе анализа текущих процессов школы иностранных языков выявлены следующие ключевые проблемы:

– отсутствие централизованной системы для управления студентами, преподавателями, курсами, расписанием и успеваемостью, что приводит к ручному выполнению задач, ошибкам и низкой оперативности;

– неэффективное разделение прав доступа между участниками (администраторы, преподаватели, студенты, пользователи), вызывающее риски несанкционированного доступа или ограничение функциональности;

– ручная обработка заявок на курсы, тестирований, формирования групп и расписания, что увеличивает трудозатраты и замедляет процессы;

– отсутствие интуитивного интерфейса для пользователей, затрудняющее взаимодействие с системой, особенно для новых участников;

– недостаток автоматизации в отслеживании успеваемости, управлении прайс-листами и анализе отзывов.

Назначение программного средства. Разрабатываемая система направлена на автоматизацию ключевых бизнес-процессов школы иностранных языков, включая:

– управление студентами, преподавателями, курсами и группами;

– формирование и контроль расписания занятий;

– обработку заявок на обучение и преподавание;

– проведение тестирований и анализ результатов;

– взаимодействие с пользователями через ролевую модель с разделением прав доступа.

Цели создания:

– повышение эффективности управления процессами школы за счет автоматизации рутинных операций;

– обеспечение безопасности данных через аутентификацию, шифрование паролей и ролевое разделение доступа;

– улучшение пользовательского опыта за счет интуитивного интерфейса и обработки исключительных ситуаций;

– оптимизация взаимодействия между студентами, преподавателями и администраторами;

– создание единой базы данных для хранения информации о курсах, расписании, заявках и успеваемости.

Ключевые задачи системы:

– автоматизация записи студентов на курсы и обработки заявок;

– управление расписанием занятий и распределение преподавателей по группам;

– проведение тестирований студентов с автоматическим контролем результатов;

– формирование прайс-листов и каталога курсов;

– обеспечение обратной связи через отзывы и их модерацию;

– генерация отчетов для анализа успеваемости и посещаемости.

Архитектурное решение представляет собой согласованный набор принципов, концепций, компонентов и их взаимосвязей, которые закладываются в основу создания информационной системы, приложения или продукта. Оно разрабатывается на начальных этапах проекта и служит базой для дальнейшего технического проектирования [9].

Разрабатываемое приложение для школы иностранных языков разделено на клиентскую и серверную части, взаимодействующие через сокеты. Взаимодействие по сокетам обеспечивает двустороннюю связь, что позволяет клиенту и серверу эффективно обмениваться данными в реальном времени.

Для разделения логики приложения используется архитектура MVC (Model-View-Controller), адаптированная для сокетного взаимодействия. Этот шаблон проектирования помогает организовать код, отделяя данные и логику от пользовательского интерфейса. MVC включает три основные компонента:

1 Модель отвечает за данные и бизнес-логику приложения. Она управляет хранением, обработкой и обновлением данных. На стороне клиента модель представляет данные, полученные от сервера, а на сервере работает с данными из базы. Модель также включает логику валидации и анализа данных, обеспечивая корректность информации перед её отображением или сохранением.

2 Представление отвечает за отображение данных пользователю через интерфейс. На стороне клиента представление включает элементы интерфейса, такие как таблицы и текстовые поля, которые взаимодействуют с пользователем. На стороне сервера представление можно рассматривать как сформированные на основе данных модели ответы, отправляемые клиенту через сокетное соединение.

3 Контроллер является связующим звеном между моделью и представлением. Он обрабатывает события, координирует передачу данных между моделью и представлением. На клиентской стороне контроллеры управляют действиями пользователя, создавая запросы к серверу через сокеты и обрабатывая ответы. На стороне сервера контроллеры обрабатывают входящие запросы, обновляют модель или формируют ответные сообщения для клиента.

Серверное приложение включает DAO слой, отвечающий за взаимодействие с базой данных, реализованный на основе JPA (Java Persistence API). Это обеспечивает абстракцию над операциями с базой данных и позволяет управлять данными независимо от СУБД.

Использование MVC в сочетании с сокетным взаимодействием позволяет эффективно разделить ответственность между компонентами, что упрощает разработку, поддержку и масштабирование приложения. Выбранное архитектурное решение было обосновано и подтверждено как подходящее для создания данной системы.

Особенности функционирования после внедрения:

1 Ролевая модель доступа:

– администраторы: управление аккаунтами, курсами, группами, расписанием, прайс-листами и заявками.

– преподаватели: просмотр расписания, результатов тестов, состава групп и личной информации.

– студенты: запись на курсы, прохождение тестов, оставление отзывов, редактирование личных данных.

– пользователи: просмотр каталога курсов, прайс-листов и подача заявок на обучение/преподавание.

2 Автоматизация процессов: обработка данных о студентах, расписании и успеваемости без ручного вмешательства.

3 Безопасность: шифрование паролей, аутентификация пользователей, резервное копирование данных.

4 Интерфейс: использование стандартных элементов (таблицы, формы, списки) для упрощения навигации.

Результаты внедрения:

– сокращение времени на администрирование за счет автоматизации процессов;

– улучшение точности данных и снижение ошибок при ручном вводе;

– повышение прозрачности процессов для всех участников (студенты, преподаватели, администраторы);

– удобный доступ к актуальной информации о курсах, расписании и результатах тестирований.

Решение обеспечит школе иностранных языков гибкость, масштабируемость и соответствие современным стандартам автоматизации образовательных процессов.

Предлагаемое решение основано на клиент-серверной архитектуре с использованием шаблона MVC, что обеспечивает эффективное управление процессами школы иностранных языков. Система включает модули для работы с курсами, расписанием, заявками, тестированием, прайс-листами и ролевой моделью доступа, что позволяет комплексно автоматизировать ключевые образовательные и административные задачи. Такая структура гарантирует безопасность данных, масштабируемость и интуитивность интерфейса. Внедрение решения позволит оптимизировать рабочие процессы, снизить риски ошибок, улучшить взаимодействие между участниками и повысить качество образовательных услуг.

3 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Архитектурные решения и технологии реализации программного средства.

По условию задачи необходимо разработать функционирующее клиент-серверное настольное приложение с многопоточным сервером с организацией взаимодействия с БД для организации работы школы иностранных языков.

Приложение должно предоставлять пользователю удобные и, главное, понятные формы для ввода информации. Интерфейс должен быть интуитивно понятным и содержать привычные элементы, такие как таблицы, списки и текстовые поля, для упрощения взаимодействия пользователей с системой. Все операции на серверной стороне, включая обработку данных о студентах, преподавателях, расписании и успеваемости, должны выполняться автоматически, без необходимости вмешательства сотрудников школы. Так же предусмотрены ограничения для приложения, чтобы обеспечить безопасное хранение данных и контроль доступа к системе.

Предусмотрим ограничения для приложения:

– в поля, такие как price (таблица Prise_list), могут вводиться только числовые значения, и они ограничены адекватными диапазонами для каждого конкретного поля.

– пользователь не может, например, оставить заявку, если хотя бы одно из контактных полей (телефон, фамилия или т.п.) не заполнено;

– у каждого пользователя должно быть уникальное имя пользователя;

– уникальный ключ любой записи генерируется базой данных самостоятельно;

– для входа в приложение пользователь любой роли использует своё имя пользователя и пароль, выбранные при регистрации;

– администраторы могут быть добавлены только администратором;

– изменять роль может только администратор;

– суперадминистратор добавляется с помощью прямых запросов в базу данных;

– пользователь не может отправлять формы без заполнения обязательных полей в ней на сервер.

Архитектурное решение представляет собой согласованный набор принципов, концепций, компонентов и их взаимосвязей, которые закладываются в основу создания информационной системы, приложения или продукта. Оно разрабатывается на начальных этапах проекта и служит базой для дальнейшего технического проектирования.

Разрабатываемое приложение для школы иностранных языков разделено на клиентскую и серверную части, взаимодействующие через сокеты. Взаимодействие по сокетам обеспечивает двустороннюю связь, что позволяет клиенту и серверу эффективно обмениваться данными в реальном времени.

Для разделения логики приложения используется архитектура MVC (Model-View-Controller), адаптированная для сокетного взаимодействия. Этот шаблон проектирования помогает организовать код, отделяя данные и логику от пользовательского интерфейса. MVC включает три основные компонента:

4 Модель отвечает за данные и бизнес-логику приложения. Она управляет хранением, обработкой и обновлением данных. На стороне клиента модель представляет данные, полученные от сервера, а на сервере работает с данными из базы. Модель также включает логику валидации и анализа данных, обеспечивая корректность информации перед её отображением или сохранением.

5 Представление отвечает за отображение данных пользователю через интерфейс. На стороне клиента представление включает элементы интерфейса, такие как таблицы и текстовые поля, которые взаимодействуют с пользователем. На стороне сервера представление можно рассматривать как сформированные на основе данных модели ответы, отправляемые клиенту через сокетное соединение.

6 Контроллер является связующим звеном между моделью и представлением. Он обрабатывает события, координирует передачу данных между моделью и представлением. На клиентской стороне контроллеры управляют действиями пользователя, создавая запросы к серверу через сокеты и обрабатывая ответы. На стороне сервера контроллеры обрабатывают входящие запросы, обновляют модель или формируют ответные сообщения для клиента.

Серверное приложение включает DAO слой, отвечающий за взаимодействие с базой данных, реализованный на основе JPA (Java Persistence API). Это обеспечивает абстракцию над операциями с базой данных и позволяет управлять данными независимо от СУБД [10].

Использование MVC в сочетании с сокетным взаимодействием позволяет эффективно разделить ответственность между компонентами, что упрощает разработку, поддержку и масштабирование приложения. Выбранное архитектурное решение было обосновано и подтверждено как подходящее для создания данной системы.

Для разработки клиент-серверного приложения учета учеников и курсов в школе иностранных языков важно выбрать подходящие компоненты и технологии, которые обеспечат надёжную и продуктивную работу системы. Компоненты и технологии – это независимые, многократно используемые элементы программного обеспечения, которые выполняют определённые функции. Каждый компонент имеет чёткий интерфейс и может быть комбинирован с другими для создания более комплексных приложений. На основе проведённого анализа были выбраны следующие компоненты:

1 Язык программирования. Выбран Java как объектно-ориентированный язык, обеспечивающий богатый набор инструментов для клиент-серверного взаимодействия, работы с базами данных и создания пользовательских

интерфейсов. Java отличается переносимостью, поддержкой многопоточности и эффективной работой с памятью.

2 СУБД. Для хранения данных используется MySQL, реляционная СУБД, предоставляющая строгую схему таблиц для управления данными. Её популярность, надежность и производительность делают MySQL отличным выбором для таких задач.

3 Фреймворк для работы с БД. Для упрощения взаимодействия с базой данных применяется Hibernate, реализующий спецификацию JPA. Этот ORM-фреймворк позволяет работать с объектами Java, исключая необходимость написания сложных SQL-запросов. Hibernate улучшает производительность за счёт кеширования данных и автоматизации транзакций.

4 Формат данных. Для обмена данными между клиентом и сервером выбран JSON с использованием библиотеки Gson. JSON удобен для сериализации и десериализации объектов, а также для передачи структурированных данных.

5 Сокеты для передачи данных. Связь между клиентом и сервером обеспечивается через TCP-сокеты, которые гарантируют стабильное соединение и контроль целостности данных. В Java работа с сокетами поддерживается стандартными библиотеками.

6 Пользовательский интерфейс. Для создания графического интерфейса используется JavaFX, предоставляющий удобные инструменты для разработки интерфейсов с использованием кнопок, списков и текстовых полей. JavaFX поддерживает FXML, что упрощает разработку и модификацию UI.

7 Безопасность паролей. Для безопасного хранения паролей применяется библиотека jBCrypt, обеспечивающая хеширование с добавлением соли, что повышает устойчивость к атакам методом подбора [11].

Выбор указанных компонентов и технологий обеспечивает стабильную, производительную и безопасную работу системы, а также её актуальность на конец 2025 года.

3.2 Проектирование и разработка пользовательского интерфейса

Пользовательский интерфейс (UI) представляет собой средство взаимодействия между человеком и компьютерной программой, сайтом или устройством. В него входят такие элементы, как дизайн, цветовая схема, шрифты, макет экрана, а также способы ввода данных и получения обратной связи. Главная цель интерфейса – обеспечить удобное и эффективное использование приложения.

Для создания пользовательского интерфейса в разрабатываемом приложении была выбрана библиотека JavaFX как наиболее гибкое и современное решение, что также соответствует специфике проекта. JavaFX предназначена для разработки настольных приложений с графическими элементами, такими как кнопки, таблицы, текстовые поля, списки и многие другие. Она поддерживает мультимедиа, анимацию, а также 2D и 3D графику.

В JavaFX каждое окно приложения (Stage) может содержать различные наполнения (Scene). Окно – это стандартный элемент интерфейса с заголовком и кнопками управления (свернуть, уменьшить, закрыть). По умолчанию приложение JavaFX получает одно основное окно (primary), однако при необходимости можно создавать и отображать дополнительные окна. Внутри окна размещается наполнение, которое представляет собой граф, состоящий из групп связанных элементов (контейнеров) и конечных элементов (листьев). В зависимости от выбранного макета, элементы могут быть расположены в заданных координатах, последовательно, по матричной структуре и другим схемам. Граф интерфейса можно создать прямо в коде, используя конструкторы и задавая необходимые свойства, либо с помощью FXML-документа, который имеет XML-структуру и позволяет отделить логику от представления.

Одно из ключевых преимуществ JavaFX – поддержка событийной модели, что позволяет оперативно реагировать на действия пользователя посредством добавления слушателей событий. Библиотека предоставляет набор методов для работы практически со всеми элементами интерфейса, позволяя задать действия для различных событий. Также JavaFX поддерживает каскадные таблицы стилей (CSS), что делает интерфейс более гибким и настраиваемым [12].

JavaFX также включает класс контроллера, который управляет взаимодействием между графическим интерфейсом и моделью приложения, а также бизнес-логикой. Этот класс позволяет задавать начальные свойства элементов интерфейса, передавать данные из модели для отображения на экране, либо считывать данные из интерфейса для работы с ними. Контроллер выступает как посредник, упрощая разработку и тестирование.

Для удобного взаимодействия пользователя с приложением были реализованы разнообразные формы ввода данных, содержащие подсказки, кнопки подтверждения и подписи, создающие простой и понятный интерфейс.

Окно авторизации предназначено для входа пользователей в систему школы иностранных языков «SpeakEasy» (рисунок 3.1). Оно содержит следующие элементы:

– поле для ввода имени пользователя;
– поле для ввода пароля;
– кнопку «Войти» для авторизации в системе;
– кнопку «Регистрация», если «Нет аккаунта?», для перехода к созданию нового аккаунта.

Интерфейс выполнен в светло-голубых и фиолетовых тонах, что придаёт ему дружелюбный и современный вид. Название школы выделено жирным шрифтом в верхней части экрана. Удобное расположение элементов позволяет пользователю быстро и просто выполнить вход в систему.

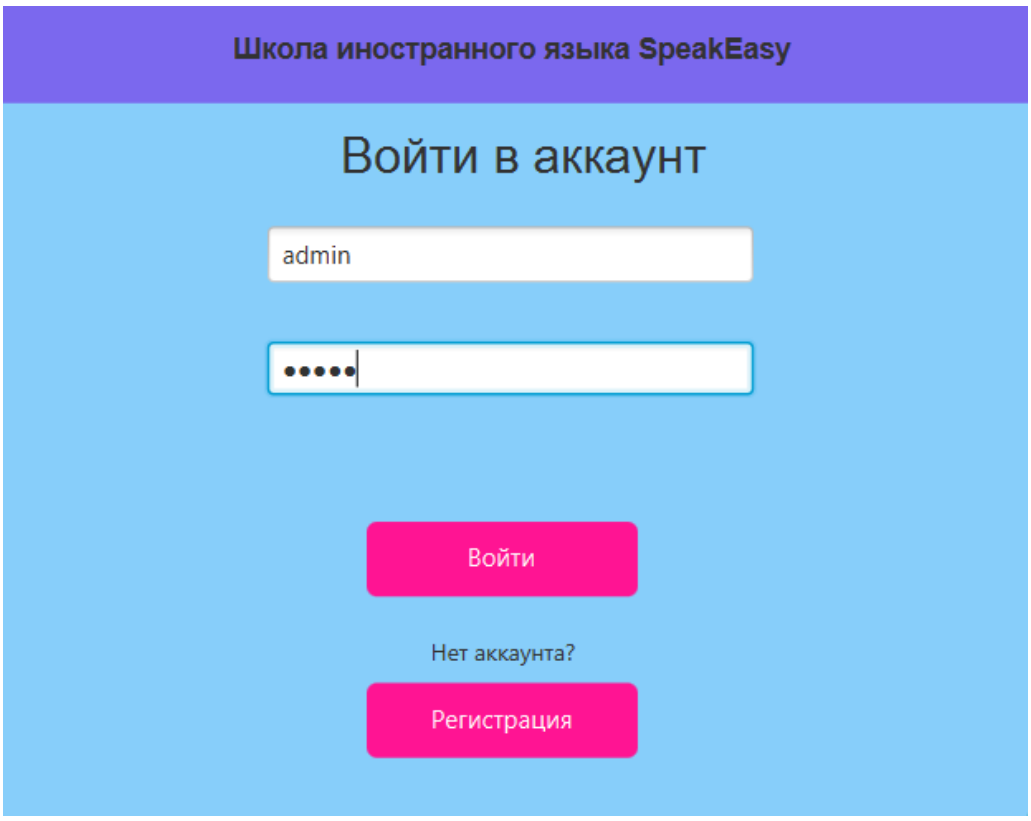


Рисунок 3.1 –Окно авторизации

Главное окно системы управления школы иностранных языков «SpeakEasy» (рисунок 3.2) предназначено для администрирования ключевых разделов платформы. Оно включает следующие элементы:

В левой части экрана расположено навигационное меню с основными модулями системы:

– заявки;
– курсы;
– прайс-лист;
– отзывы;
– группы;
– расписание;
– преподаватели;
– студенты.

Такое построение интерфейса обеспечивает быстрый доступ к различным функциям системы и позволяет эффективно управлять учебным процессом.

В нижней части панели размещена кнопка «Выход» для безопасного завершения работы с системой.

Цветовое оформление выполнено в фирменных фиолетово-голубых тонах, а логотип «SpeakEasy» в верхней части панели подчеркивает принадлежность к образовательной платформе.

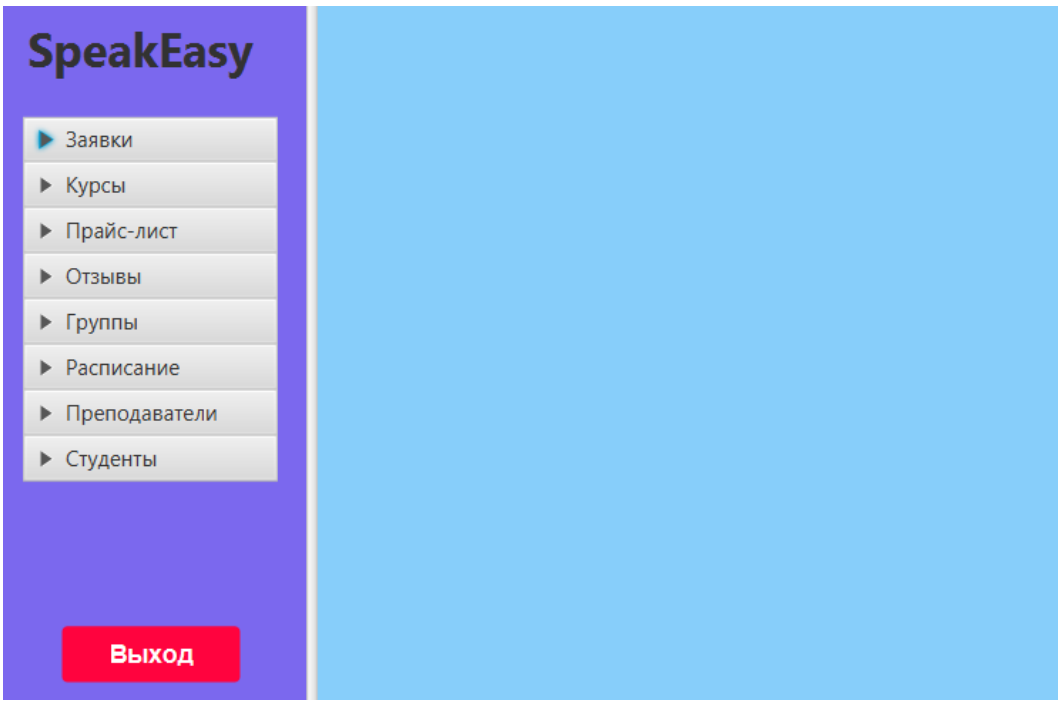


Рисунок 3.2 –Главное меню администратора

Загрузочное окно студента (рисунок 3.3), предназначенное для взаимодействия с образовательными материалами и прохождения тестирования. Оно включает:

В верхней части меню навигации расположены следующие разделы:

– курсы;
– прайс-лист;
– отзывы;
– расписание.

Такое меню позволяет студенту быстро переключаться между ключевыми информационными разделами платформы.

Ниже размещена кнопка:

– «Пройти тестирование» –переход к разделу с проверкой знаний и уровневой аттестацией.

В нижней части экрана присутствует кнопка:

– «Выход» –для безопасного завершения работы в системе.

Интерфейс оформлен в фирменных цветах школы «SpeakEasy» – фиолетовом и голубом, а логотип размещён в верхней части панели.

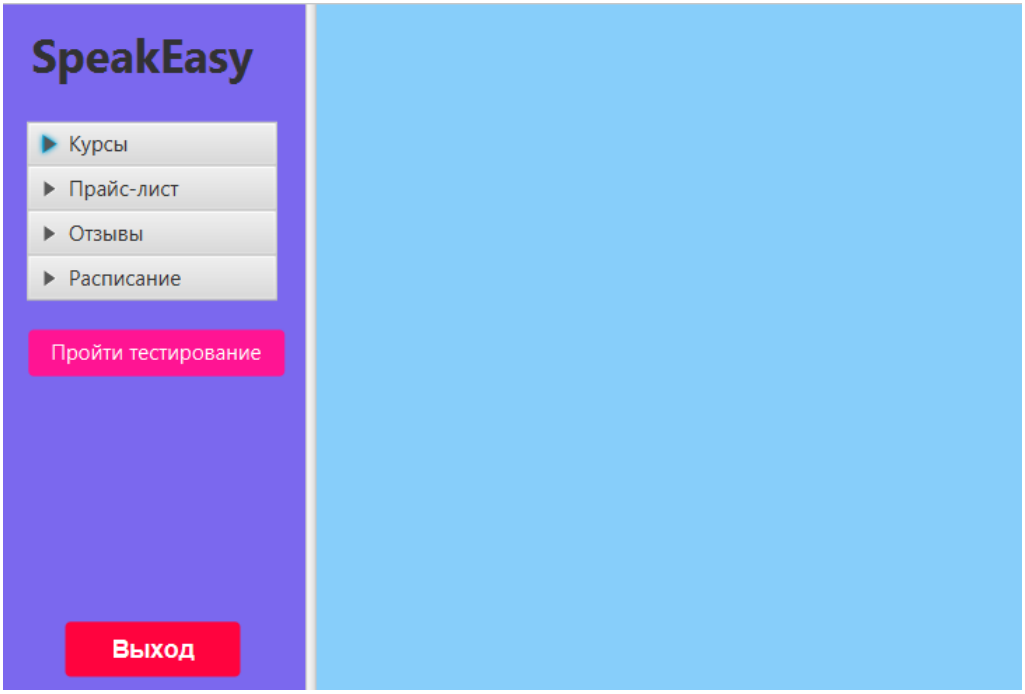


Рисунок 3.3 –Главное меню студента

Загрузочное окно преподавателя (рисунок 3.4), предназначенное для управления учебным процессом и взаимодействия со студентами. Оно включает следующие элементы:

В верхней части меню навигации расположены следующие разделы:

– курсы;
– прайс-лист;
– отзывы;
– расписание;
– результаты тестов;
– мои группы (активный раздел).

Такое меню позволяет преподавателю быстро переключаться между основными функциями, необходимыми для организации занятий, анализа успеваемости и работы с обратной связью.

В нижней части экрана размещена кнопка:

– «Выход» –для завершения сеанса работы с системой.

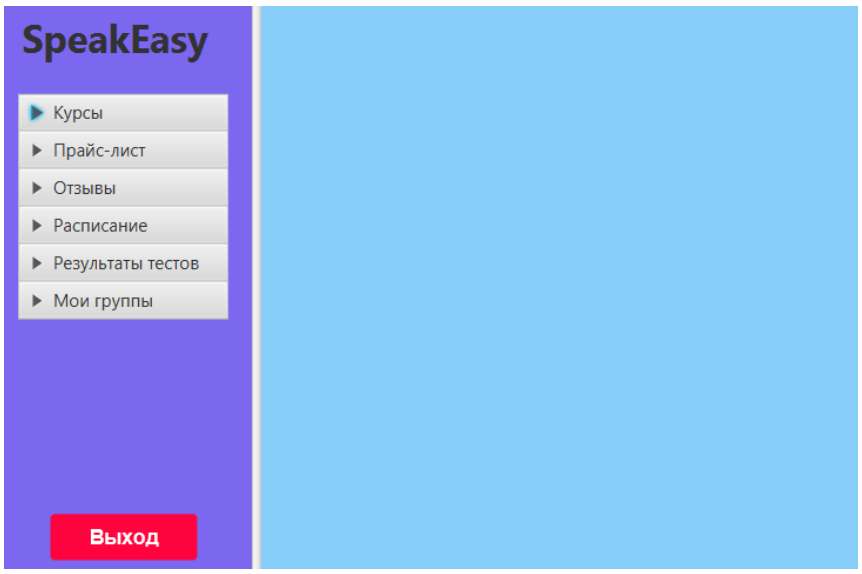


Рисунок 3.4 –Главное меню преподавателя

Экран пользователя, который еще не стал преподавателем или студентом включает следующие элементы:

В верхней части меню навигации расположены следующие разделы:

– оставить заявку;
– курсы;
– прайс-лист;
– отзывы;

Такое меню позволяет преподавателю быстро переключаться между основными функциями.

В нижней части экрана размещена кнопка:

– «Выход» –для завершения сеанса работы с системой.

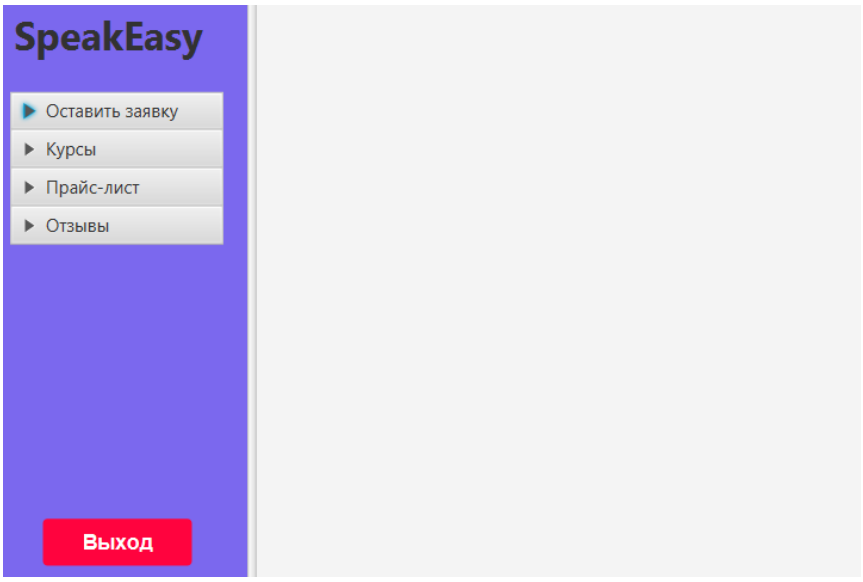


Рисунок 3.5 –Окно пользователя

3.3 Разработка модели данных

Информационная модель предметной области – это структура данных, представленная в виде таблиц, где каждая сущность описывается отдельной таблицей, а её характеристики представлены в виде столбцов. Для создания связей между сущностями используются внешние ключи, которые гарантируют целостность данных и служат указателями на другие таблицы. Эта модель лежит в основе проектирования и разработки информационных систем, адаптированных для работы в определённой предметной области.

Информационная модель системы в нотации IDEF1X представлена на рисунке 3.6.

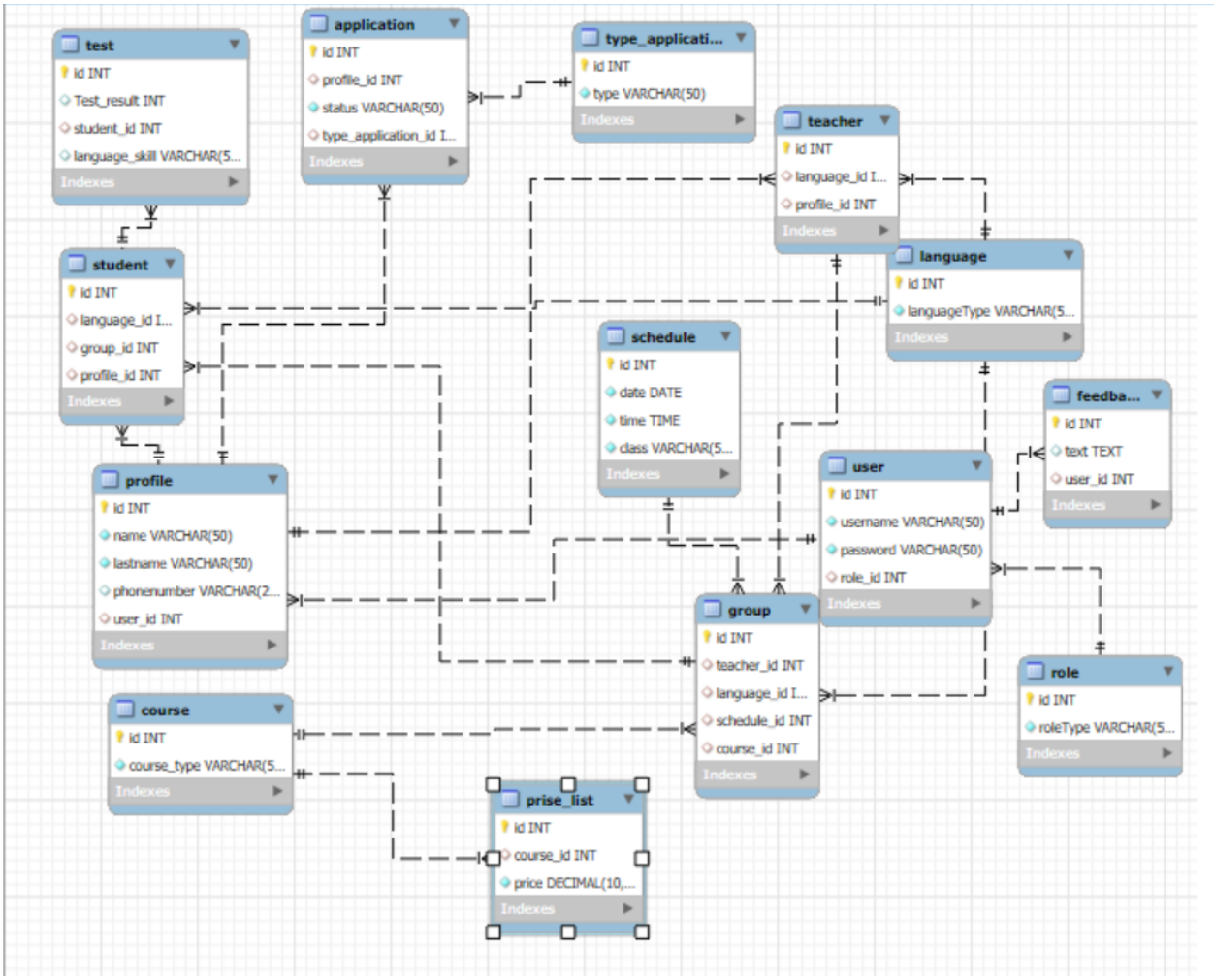


Рисунок 3.6 – Информационная модель системы в нотации IDEF1X

Сущность role – хранит типы ролей (например, "студент", "учитель", "администратор") для пользователей. Поле roleType указывает конкретную роль, а id используется как внешний ключ в других таблицах.

Сущность language – содержит информацию о языках, на которых обучаются студенты или преподают учителя. Поле languageType определяет тип языка, а id используется в других таблицах как ссылка на язык.

User – представляет учетные записи пользователей системы. Поля username и password хранят имя пользователя и его пароль. Внешний ключ role_id связывает пользователя с его ролью в таблице Role. Уникальный идентификатор id используется для ссылок на пользователя в других таблицах.

Profile – хранит персональные данные профиля, включая имя (name), фамилию (lastname), телефонный номер (phonenumber) и связь с учетной записью пользователя (user_id). В случае удаления пользователя профиль также будет удален благодаря ON DELETE CASCADE.

Teacher – хранит информацию о преподавателях, включая язык, на котором они преподают (language_id), и ссылку на профиль (profile_id). Если язык удаляется, поле language_id становится пустым.

Course – описывает курсы, которые предлагаются в школе. Поле course_type указывает тип курса (например, "базовый", "продвинутый"), а id используется для связи с другими таблицами.

Prise_list – хранит информацию о стоимости курсов. Поле price указывает стоимость курса, а внешний ключ course_id связывает таблицу с таблицей Course. Если курс удаляется, удаляется и соответствующая запись в таблице Prise_list.

Schedule – хранит информацию о расписании занятий, включая дату (date), время (time) и место (class).

Group – представляет учебные группы, объединяя преподавателя (teacher_id), язык обучения (language_id), расписание (schedule_id) и курс (course_id). Удаление расписания или курса приводит к удалению группы.

Student – хранит информацию о студентах, включая язык обучения (language_id), принадлежность к группе (group_id) и ссылку на профиль (profile_id). Удаление профиля студента делает запись в таблице пустой.

Test – хранит результаты тестов студентов. Поля включают результат теста (Test_result), языковую компетенцию (language_skill) и внешний ключ student_id, который связывает запись с конкретным студентом.

Type_application – описывает типы заявок, подаваемых пользователями, например "на обучение", "на перевод". Поле type определяет тип заявки, а id используется в других таблицах.

Application – представляет собой заявки, подаваемые профилями (например, на обучение, участие). Поле status указывает состояние заявки, а type_application_id связывает заявку с типом.

Feedback – хранит отзывы пользователей. Поле text содержит текст отзыва, а user_id связывает отзыв с конкретным пользователем.

Таким образом, была рассмотрена и описана информационная модель системы в нотации IDEF1X, определены необходимые сущности и входящие в них атрибуты.

Перед началом разработки любого приложения важно уделить внимание этапу проектирования, так как именно он закладывает основу для дальнейшей реализации. Одним из ключевых инструментов проектирования является диаграмма классов, которая помогает разработчикам лучше понять структуру

программы, определить взаимосвязи между ее компонентами и спланировать архитектуру системы.

Диаграмма классов – это структурная диаграмма, разработанная на основе языка моделирования UML (Unified Modeling Language). Она предоставляет визуальное представление классов системы, их атрибутов (полей), методов, интерфейсов, а также связей между ними. Этот инструмент полезен как для документирования существующей системы, так и для проектирования новой системы с использованием методов прямого или обратного моделирования.

Диаграмма классов позволяет увидеть общую иерархию системы, определить ключевые компоненты и их роли. Она отображает такие элементы, как:

– Классы, представляющие основные сущности системы. Каждый класс включает атрибуты (данные, которые он хранит) и методы (действия, которые он выполняет).

– Интерфейсы, описывающие контракт, который должны реализовывать определенные классы.

– Связи между классами, такие как ассоциация, агрегация, композиция и наследование.

Агрегация представляет собой тип ассоциации, который характеризует отношение "целое-часть". Это означает, что один объект состоит из нескольких других объектов, которые могут существовать независимо друг от друга

Композиция – это более жесткая форма агрегации, при которой объекты-части не могут существовать независимо от агрегирующего объекта. Это отношение также характеризуется как "целое-часть", но с условием, что объекты-части уничтожаются вместе с агрегирующим объектом.

При агрегации объекты могут существовать независимо, а при композиции они зависят от своего агрегирующего объекта.

Наследование – это еще один важный тип связи, который моделирует отношение "общее-частное". Оно используется, чтобы связать классы в иерархию, где один класс наследует атрибуты и методы другого класса. Базовый класс предоставляет общую функциональность, а производный класс может дополнять или переопределять эту функциональность.

Диаграмма классов необходима для документирования системы, так как она помогает разработчикам, аналитикам и другим участникам проекта получить общее представление о структуре приложения. Она также используется для анализа и проектирования, поскольку позволяет выявить потенциальные проблемы, такие как избыточные зависимости или нарушения принципов проектирования.

Кроме того, диаграмма классов способствует улучшению коммуникации в команде, так как предоставляет визуальное средство для обсуждения архитектуры системы. Она помогает задокументировать систему на этапе проектирования, а также может быть использована для генерации кода или проверки существующего.

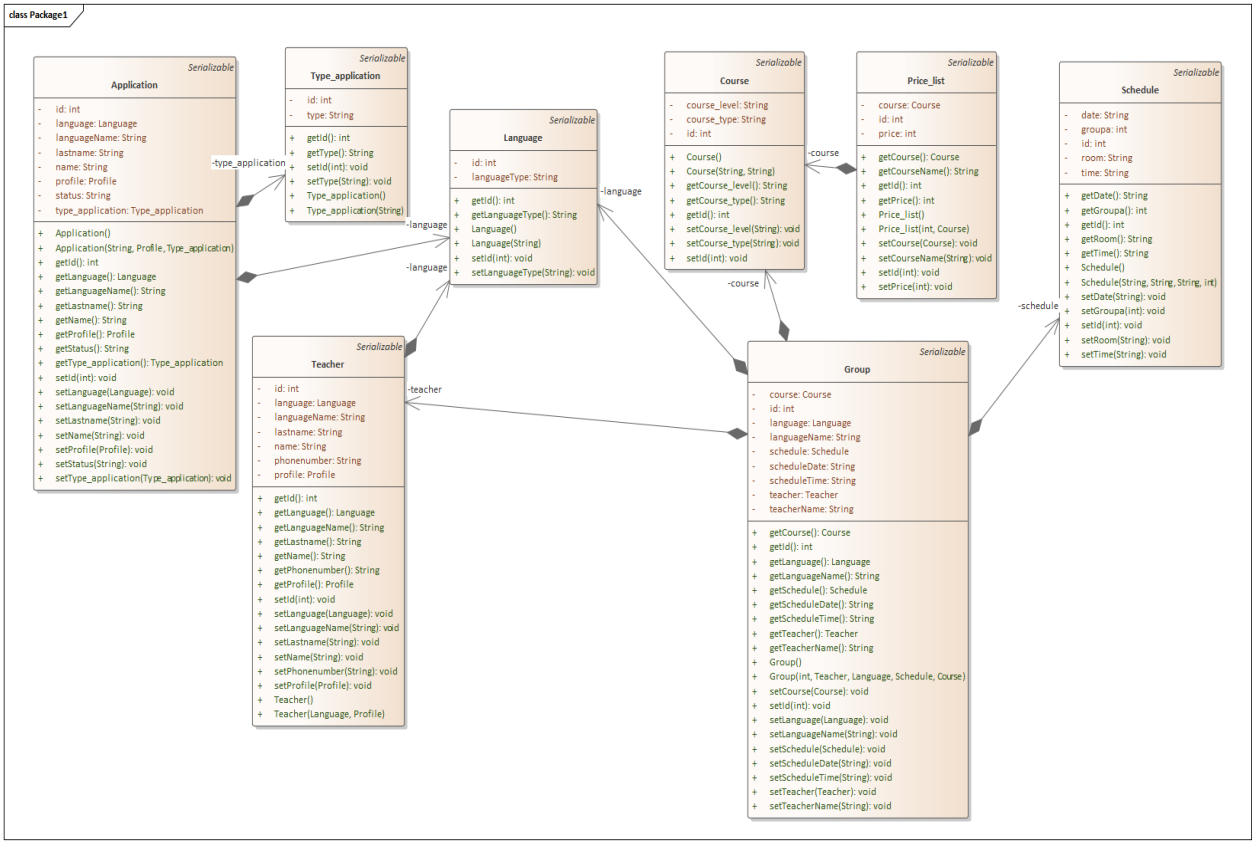


Рисунок 3.7 – Первая часть диаграммы классов серверной стороны

Диаграмма на рисунке 3.7 и рисунке 3.8 отображает объектную модель приложения для языковых курсов. Центральной сущностью является Student, представляющий студента с привязками к группе (Group), языку (Language), профилю (Profile) и другим базовым данным. Важной особенностью является наличие конструкторов с параметрами, а также геттеров и сеттеров, обеспечивающих доступ к приватным полям.

Teacher – класс преподавателя, связанный с языком преподавания и профилем пользователя. Group объединяет студентов, преподавателей, курсы и расписание, выступая точкой координации между этими сущностями. В Group реализованы методы установки учителя, курса и расписания, что делает её центром учебного процесса.

Класс Course хранит информацию о курсе, включая уровень сложности и язык. Price_list связан с курсом и определяет цену. Расписание (Schedule) содержит даты и время занятий.

Поддержка аутентификации реализована через классы User и Role. Каждый пользователь имеет роль, а также связан с профилем, содержащим имя, фамилию, номер телефона и email. Система отзывов реализована через класс Feedback, который содержит ссылку на пользователя, текст отзыва и дату.

Типы заявок (Application) представлены через связь с Type_application, Profile и языком. Это позволяет отслеживать заявки по типам и языковым предпочтениям.

Тестирование организовано через класс Test, содержащий информацию о навыке, дате и результатах, привязанный к конкретному студенту. Диаграмма построена на принципах объектно-ориентированного моделирования, с четким разделением ответственности и удобной навигацией по связям.

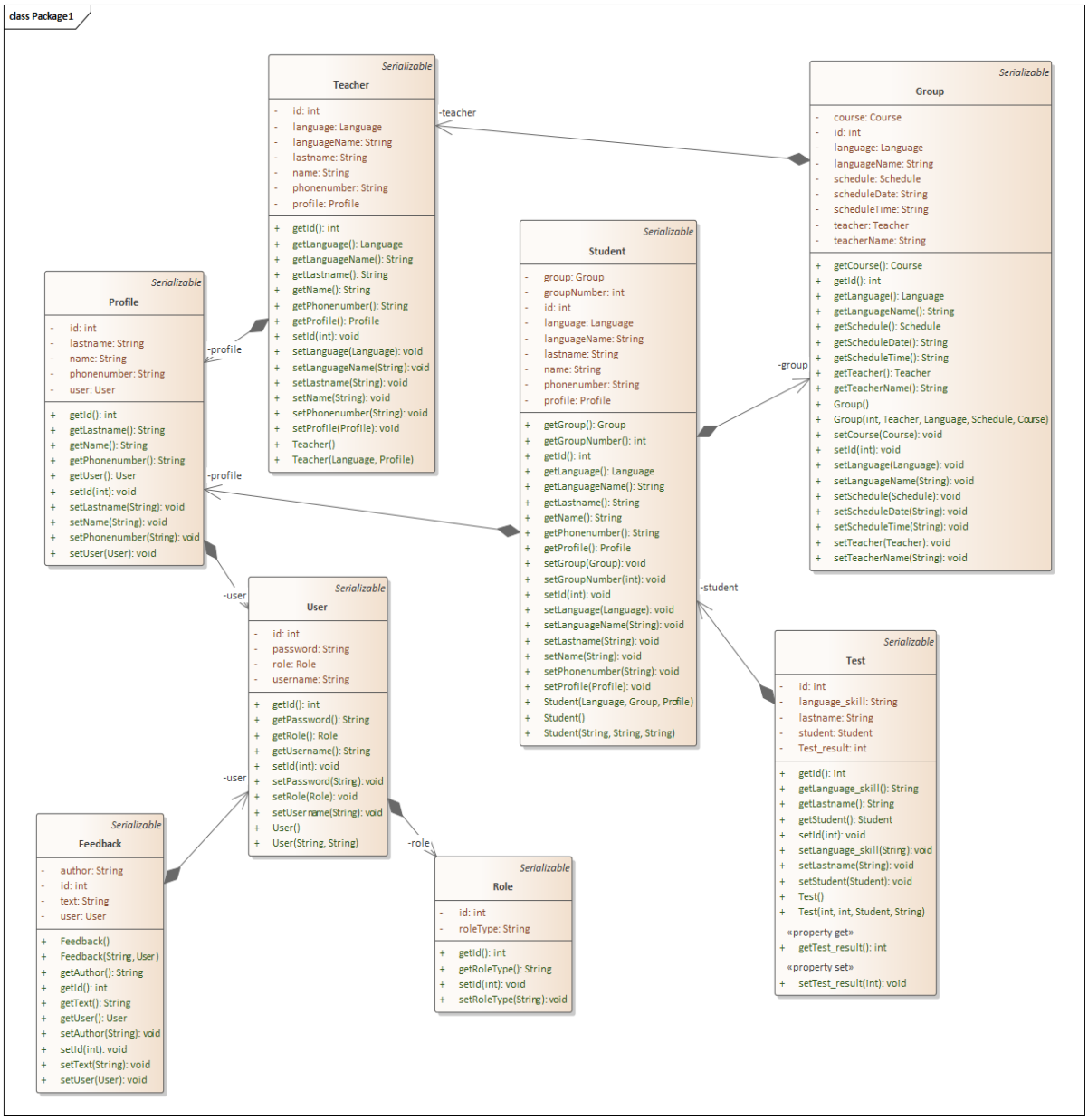


Рисунок 3.8 – Вторая часть диаграммы классов серверной стороны

На диаграмме (рисунок 3.9) представлена архитектура сервисного слоя, который реализует бизнес-логику и обрабатывает действия, связанные с различными сущностями системы. Каждый сервис взаимодействует с соответствующим DAO-объектом и предоставляет методы для выполнения операций с данными. ApplicationService управляет заявками, предоставляя возможность добавления, удаления, поиска по профилю, типу и идентификатору. CourseService реализует функциональность по управлению

курсами, включая их выборку, добавление и обновление. FeedbackService отвечает за обработку отзывов пользователей, позволяя сохранять, изменять и удалять отзывы. GroupService управляет группами, в том числе их созданием, обновлением и поиском по различным параметрам. LanguageService позволяет получать все языки, искать конкретные по ID или типу и управлять их списком. Price_listService связан с курсами и реализует логику ценообразования, включая обновление цены курса по его идентификатору. Профильная информация пользователей обрабатывается в ProfileService, который предоставляет методы получения профиля по userID или profileID и их обновления. ScheduleService управляет расписаниями, предоставляя методы для добавления, поиска и обновления расписания занятий. StudentService предоставляет расширенные функции для получения студентов, включая выборку без группы, по ID, профилю, группам, а также методы подсчета студентов по группам. TeacherService реализует бизнес-логику, связанную с преподавателями, включая выборку по языку и профилю. TestService работает с тестами, предоставляя методы вставки и обновления результатов. UserService отвечает за операции, связанные с пользователями системы, включая регистрацию, проверку логина, обновление данных и аутентификацию. Вся архитектура построена по принципу модульности, обеспечивая четкое разграничение ответственности между слоями и удобство расширения.

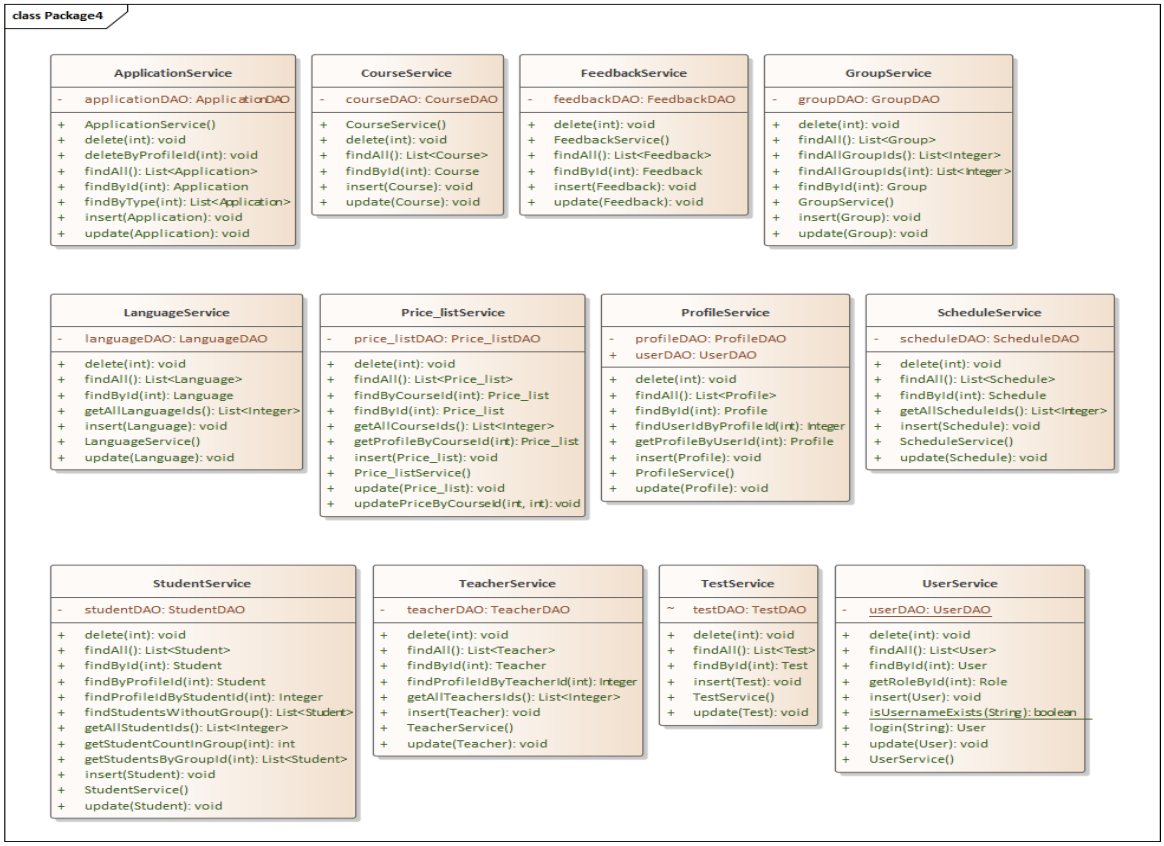


Рисунок 3.9 – Третья часть диаграммы классов серверной стороны

Диаграмма (рисунок 3.10 и рисунок 3.11) отображает слой доступа к данным, построенный на использовании шаблона Data Access Object (DAO) с применением SessionFactory, что указывает на интеграцию с ORM, вероятно Hibernate. Каждый DAO-класс реализует стандартный набор операций, таких как вставка, обновление, удаление и поиск сущностей по идентификатору или другим параметрам. ApplicationDAO управляет объектами заявок, предоставляя выборку по профилю, типу и ID. CourseDAO реализует доступ к курсам, включая массовую выборку и фильтрацию. FeedbackDAO позволяет выполнять CRUD-операции с отзывами. GroupDAO управляет данными о группах, включая выборку по спискам ID и возможность вставки или обновления группы. LanguageDAO предоставляет методы для получения списка всех поддерживаемых языков и их идентификацию. Price_listDAO позволяет оперировать с ценами на курсы, включая обновление стоимости по курсу. ProfileDAO реализует доступ к профилям пользователей, включая поиск по userID или profileID. RoleDAO управляет ролями пользователей, предоставляя простые методы для получения списка всех ролей и вставки новых. ScheduleDAO реализует управление расписаниями занятий. StudentDAO предоставляет расширенные методы доступа к студентам, включая выборку по профилю, поиск без группы, по ID, а также массовую вставку. TeacherDAO реализует доступ к данным преподавателей, включая поиск по языку, профилю и ID. TestDAO управляет сущностями тестов, предоставляя стандартные операции. Type_applicationDAO работает с типами заявок. UserDAO реализует доступ к учетным записям пользователей, включая проверку логина и выборку по имени. Вся архитектура DAO-слоя строго разделена по сущностям и подчиняется принципу единственной ответственности, что обеспечивает гибкость при работе с базой данных и упрощает сопровождение проекта.

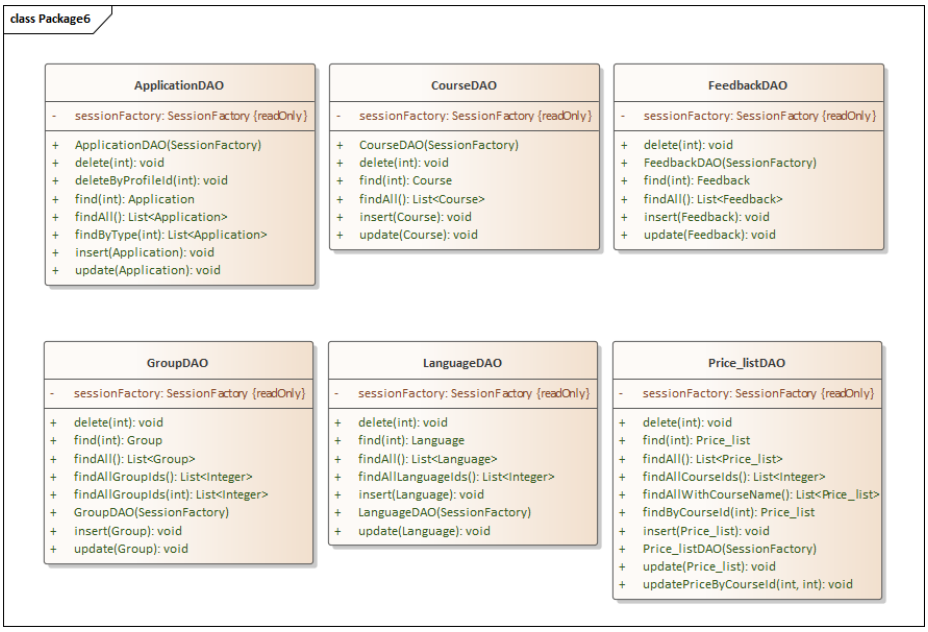


Рисунок 3.10 – Четвертая часть диаграммы классов серверной стороны



Рисунок 3.11 – Пятая часть диаграммы классов серверной стороны

Диаграмма (рисунок 3.12) представляет архитектуру обмена сообщениями между клиентом и сервером через объекты запроса и ответа. Класс Request инкапсулирует информацию, отправляемую клиентом, включая сообщение, тип запроса и статус выполнения. Тип запроса задается через перечисление RequestType, которое содержит полный набор возможных действий и операций в системе, таких как регистрация, вход в систему, подача заявок, управление курсами, преподавателями, студентами, расписанием и другими сущностями. Наличие таких типов, как MenuAdmin, MenuUser, TeachingApplication, AddGroup, DeleteTeacher, StudentCountInGroup и GetStudentData, позволяет гибко и безопасно маршрутизировать запросы, обеспечивая поддержку множества сценариев взаимодействия.

Класс Response содержит ответ сервера на клиентский запрос. Он дублирует структуру запроса, включая сообщение, тип ответа и флаг успешности выполнения. Таким образом, обеспечивается унифицированный и сериализуемый механизм передачи данных между клиентской и серверной частью, упрощающий отладку и расширение протокола взаимодействия.

Архитектура построена по принципу Command Pattern, где каждый тип запроса соответствует определенному действию на серверной стороне.

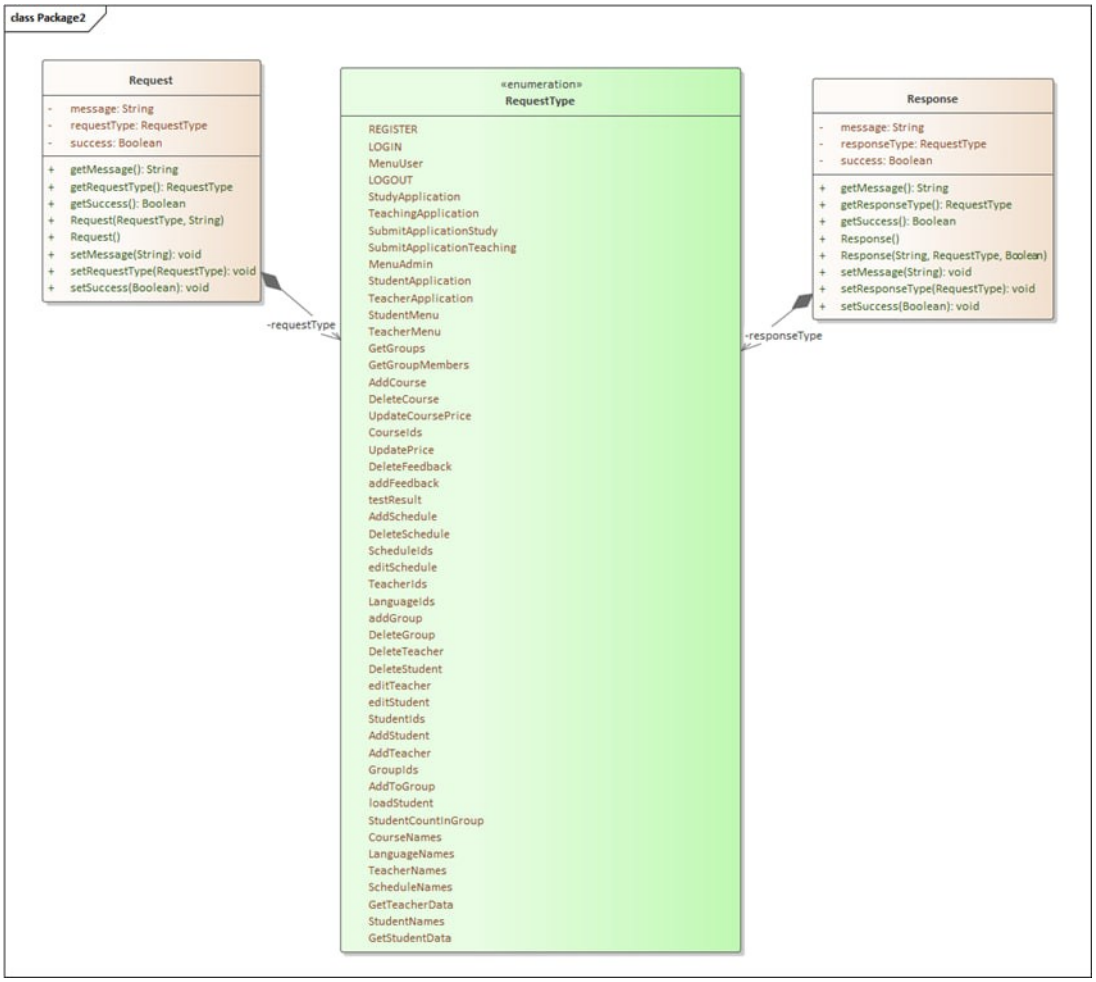


Рисунок 3.12 – Диаграмма классов взаимодействия клиент-сервер

Диаграмма на рисунке 3.13 и рисуноке 3.14 описывает техническую реализацию серверной части, в частности – поток обработки клиентских запросов. Центральным элементом является класс ClientThread, реализующий интерфейс Runnable, что позволяет запускать его в отдельном потоке для каждого подключенного клиента. Он содержит ссылки на все сервисы бизнес-логики: ApplicationService, CourseService, FeedbackService, GroupService, LanguageService, Price_listService, ProfileService, ScheduleService, StudentService, TeacherService, TestService и UserService, которые используются для выполнения конкретных действий в ответ на полученные запросы.

Также в ClientThread представлены поля для хранения сокета клиента, текущих идентификаторов профиля и студента, а также потоки ввода и вывода (BufferedReader, PrintWriter) и объекты сериализации (Gson). Сами запросы и ответы представлены в виде объектов Request и Response.

Набор методов handle* реализует обработку каждого возможного запроса, соответствующего значениям перечисления RequestType. Среди них

обработка регистрации, авторизации, добавления, удаления и редактирования студентов, преподавателей, курсов, групп, расписаний и т. д. Методы типа handleLogin, handleRegistration, handleAddCourse, handleDeleteSchedule реализуют соответствующие бизнес-действия, при этом для каждого метода создаётся цепочка вызовов через соответствующий сервис.

Поддержка безопасности реализована с помощью утилитного класса PasswordUtils, предоставляющего методы хеширования и проверки паролей. Работа с базой данных осуществляется через HibernateUtil, который управляет сессиями и фабриками сессий Hibernate, обеспечивая доступ к данным в многопоточном окружении.

Таким образом, класс ClientThread является координирующим компонентом серверной части, который принимает запросы, направляет их в соответствующий сервис и формирует ответ, обеспечивая многопоточную обработку клиентов и масштабируемость всей системы.

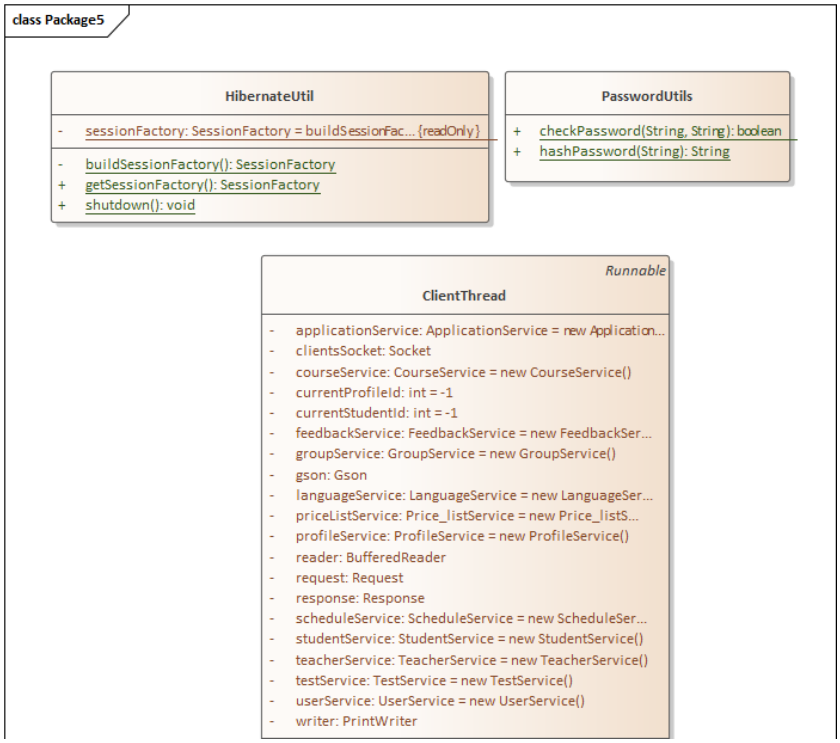


Рисунок 3.13 – Диаграмма классов серверной инфраструктуры и потока обработки ClientThread

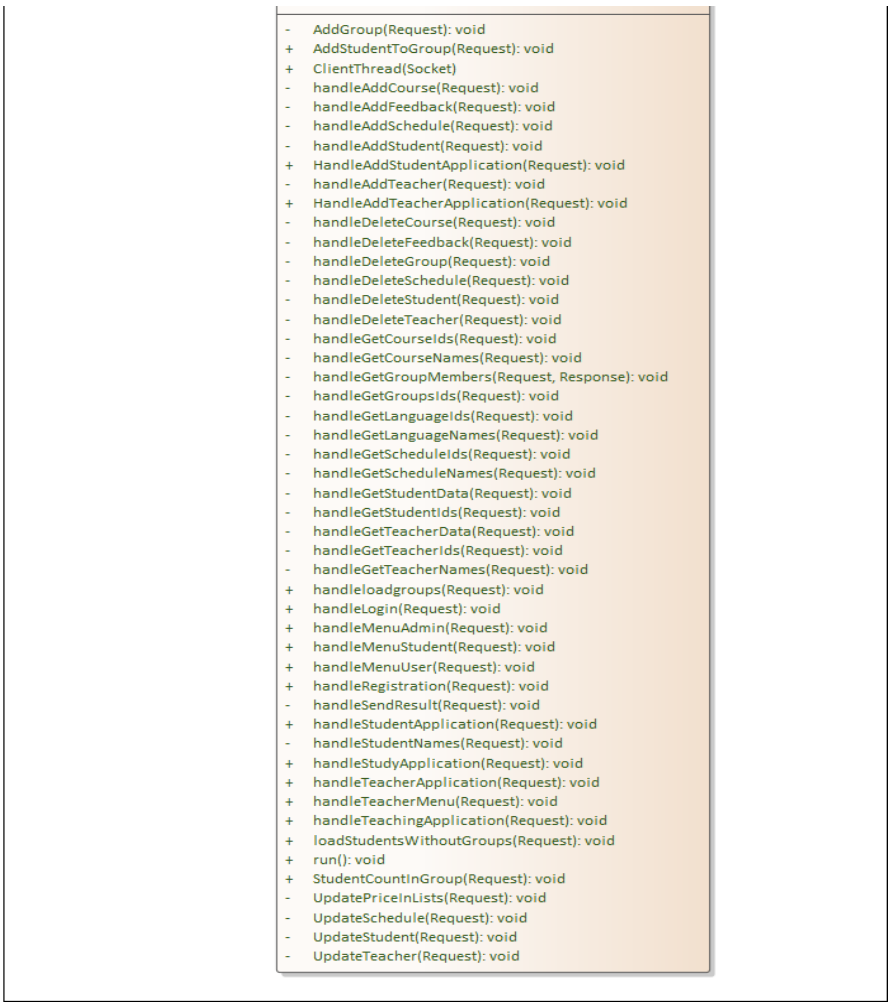


Рисунок 3.14 – Диаграмма классов серверной инфраструктуры и потока обработки ClientThread

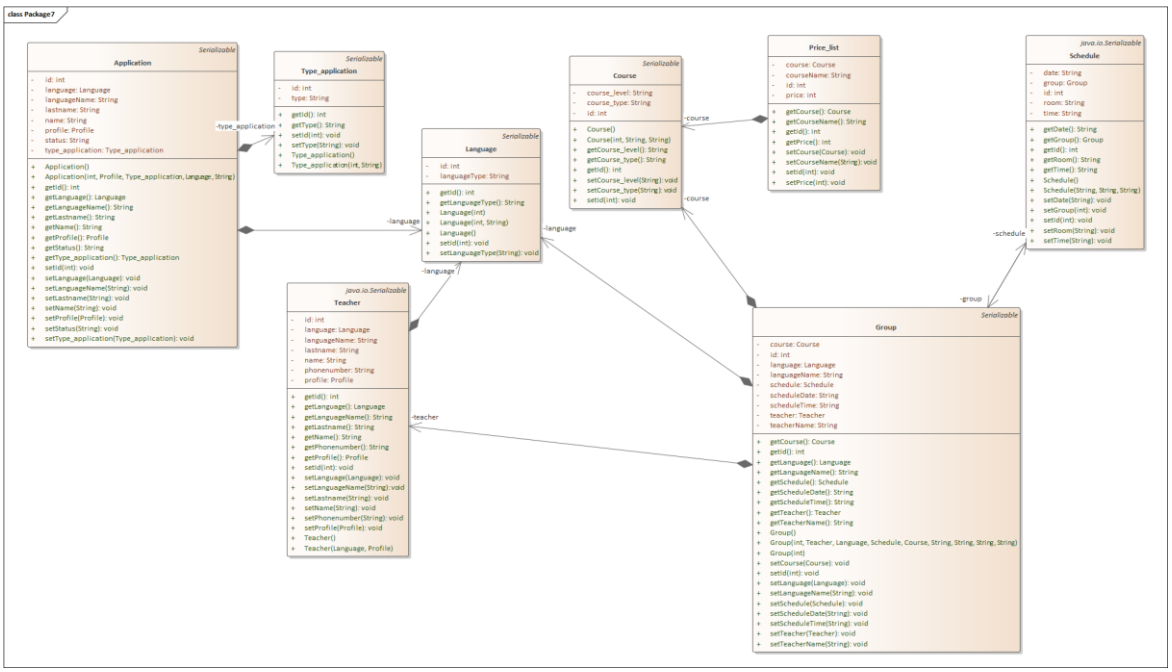


Рисунок 3.15 – Первая часть диаграммы классов клиентской стороны

Диаграмма (рисунок 3.15 и 3.16) отражает клиентскую репрезентацию модели предметной области, фактически дублируя структуру серверной модели, но с ориентацией на работу в контексте клиентского интерфейса. Центральное место занимает класс Student, который инкапсулирует всю информацию о студенте: имя, фамилию, язык, группу, профиль и логические связи с другими сущностями. Он напрямую ассоциирован с классом Group, содержащим сведения о курсе, преподавателе, языке, расписании, а также временем начала и названием группы. Такая структура позволяет клиентской стороне визуализировать, редактировать и отправлять информацию, связанную с учебным процессом.

Класс Teacher представляет преподавателя и связан с языком и профилем, аналогично серверной модели. Course определяет курс обучения и его уровень, а Price_list хранит информацию о стоимости курса, обеспечивая прозрачность в ценообразовании. Language представляет язык и применяется как в курсах, так и в заявках, профилях, студентах и преподавателях, обеспечивая сквозную интеграцию языковой составляющей в пользовательском интерфейсе.

Классы User и Role участвуют в аутентификации и разграничении прав пользователей на клиентской стороне. Профиль пользователя представлен классом Profile, который содержит личные данные и связывается с учетной записью. Feedback служит для отображения и отправки отзывов пользователей, а Test – для отображения результатов тестирования студентов.

Класс Application используется для подачи заявок и связан с языком, профилем и типом заявки (Type_application), позволяя отправлять запросы на обучение через клиентскую форму. Вся структура диаграммы отражает богатую клиентскую модель, поддерживающую двухстороннюю сериализацию данных и обеспечивающую полное отображение учебного процесса, профилей, заявок, тестов и пользовательских ролей.

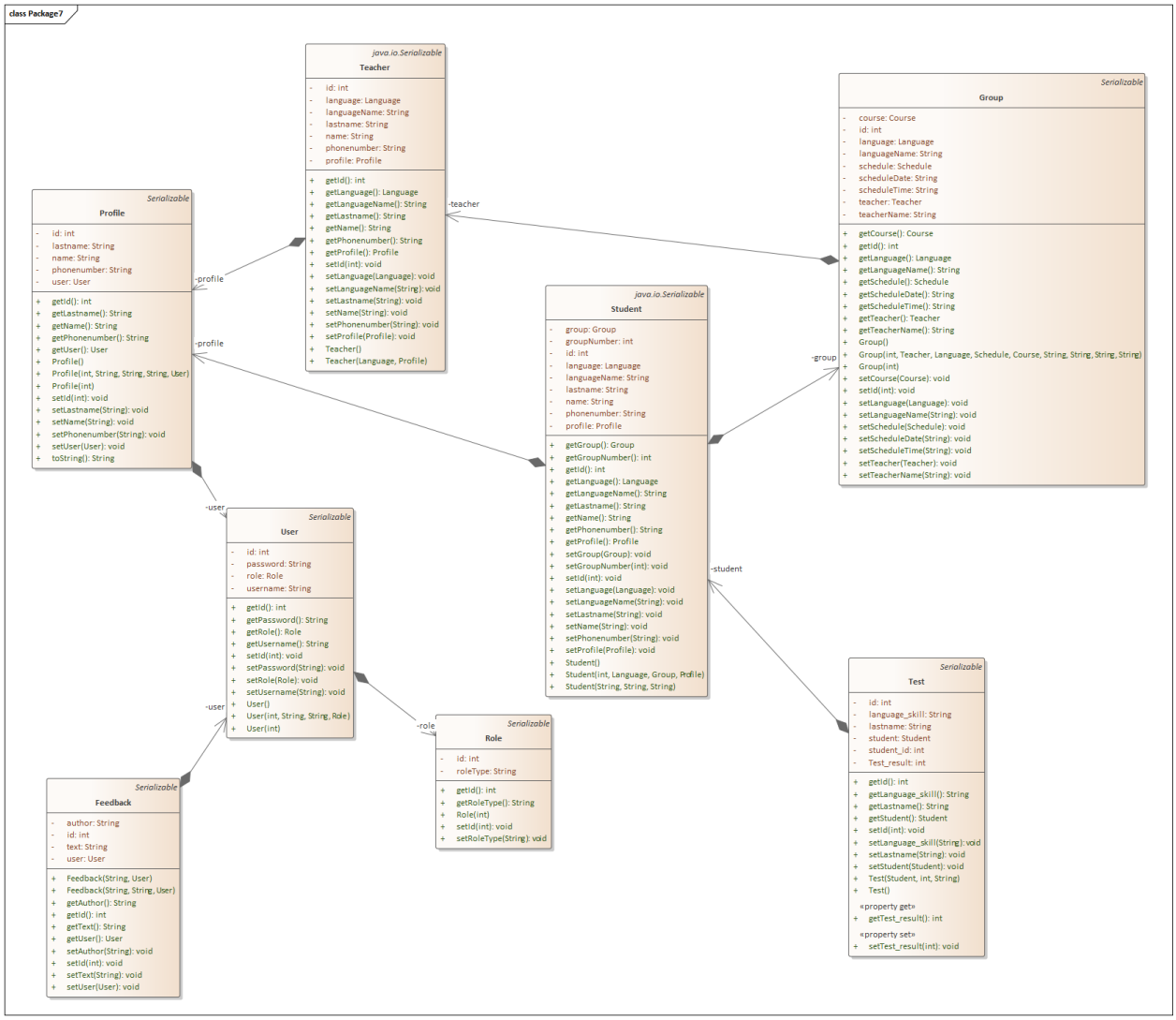


Рисунок 3.16 – Вторая часть диаграммы классов клиентской стороны

На диаграмме (рисунок 3.17) реализована логика формирования и обработки клиентских запросов и ответов. Класс Request используется для генерации запросов к серверу и содержит сообщение, тип запроса и флаг успешности. Типы запроса определяются перечислением RequestType, в котором представлены все возможные действия, поддерживаемые системой: регистрация, авторизация, работа с заявками, добавление и удаление студентов, преподавателей, курсов, групп и расписаний, редактирование сущностей, получение списков, названий, ID и результатов тестов. Эта структура обеспечивает универсальный механизм маршрутизации и классификации клиентских команд.

Класс Response инкапсулирует ответ сервера, содержащий тип ответа, сообщение и статус выполнения. Благодаря унифицированной структуре классов запроса и ответа, обеспечивается легко масштабируемый протокол обмена данными между клиентом и сервером.

Вспомогательный класс CurrentSession отслеживает текущее состояние сеанса пользователя на клиентской стороне. Он хранит текущий ID профиля, ID студента, а также строковое представление роли, что позволяет гибко управлять правами доступа и логикой отображения интерфейса. Благодаря использованию паттерна Singleton, класс гарантирует наличие только одного активного сеанса, доступного из любого места в клиентском коде.



Рисунок 3.17 – Диаграмма классов взаимодействия клиента с сервером

На диаграмме (рисунок 3.18) представлена реализация сетевого взаимодействия клиента с сервером. Основным компонентом выступает класс

ClientSocket, реализующий клиентскую часть TCP-соединения. Он содержит сокет, потоки ввода и вывода, а также объект пользователя. Параметры подключения, включая адрес сервера и порт, заданы как константы, что обеспечивает гибкость при необходимости изменения конфигурации. Используется паттерн Singleton, гарантирующий существование единственного экземпляра ClientSocket, управляющего сетевым взаимодействием.

Класс содержит методы для получения сокета, потоков ввода-вывода и текущего пользователя, что упрощает отправку и получение данных. Такая инкапсуляция логики обмена данными в одном классе способствует модульности клиентской части и облегчает поддержку кода.

Дополнительным вспомогательным компонентом является класс GsonConfig, предоставляющий метод createGson для получения объекта Gson. Это обеспечивает централизованную конфигурацию сериализации и десериализации JSON-объектов, используемых в процессе обмена данными между клиентом и сервером. Такая реализация позволяет поддерживать единый формат данных и уменьшает вероятность ошибок при передаче информации.

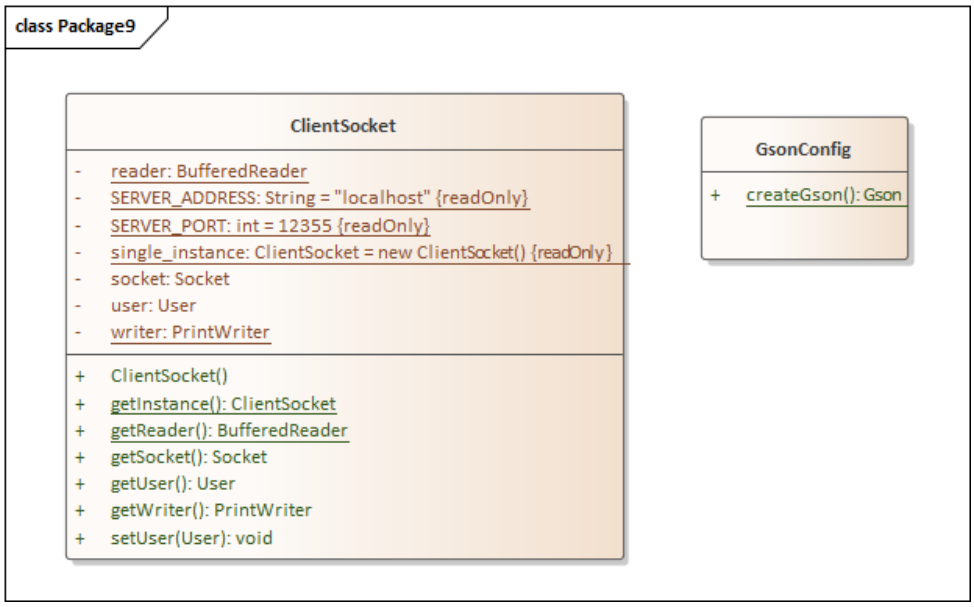


Рисунок 3.18 – Диаграмма классов клиентского соединения и конфигурации

На диаграмме классов (рисунок 3.20 и 3.21) представлена структура образовательного приложения на JavaFX с разделением по ролям пользователей: администратор, преподаватель и студент. Основные классы интерфейсов – AdminMenu, TeacherMenu, StudentMenu – наследуются от Application и отвечают за отображение соответствующего функционала: управление курсами, преподавателями, расписанием, группами, студентами и отзывами.

Класс AdminMenu содержит наибольшее количество элементов управления – таблицы, панели, кнопки, и предоставляет администратору доступ ко всем разделам системы. Он реализует методы инициализации, отображения данных, открытия окон редактирования и добавления сущностей. Преподавательский и студенческий интерфейсы, реализованные в классах TeacherMenu и StudentMenu, ориентированы на просмотр информации и взаимодействие с курсами, расписанием и отзывами. Общие компоненты для этих ролей выделены в класс UserMenu.

Авторизация пользователей реализована в классе Login, а регистрация – в Registration. Они содержат поля ввода, кнопки и методы обработки действий. Для добавления и редактирования данных используются отдельные классы: AddCourse, EditTeacherData, AddGroup, EditSchedule и другие, каждый из которых представляет отдельное окно с необходимыми элементами интерфейса.

Класс EnglishTest отвечает за проведение тестирования, содержит массивы вопросов, вариантов ответов и логику проверки. Запуск системы осуществляется из классов StudyApplication и TeachingApplication, представляющих точки входа для студента и преподавателя.

Для поддержки локализации используются классы Languages, StudyApplication.Language и TeachApplication.Language, обеспечивающие работу с языковыми строками и смену языка интерфейса.

Таким образом, система реализует модульный подход с чётким разграничением ролей, поддержкой добавления и редактирования данных, авторизации и локализации, что делает её удобной и масштабируемой.

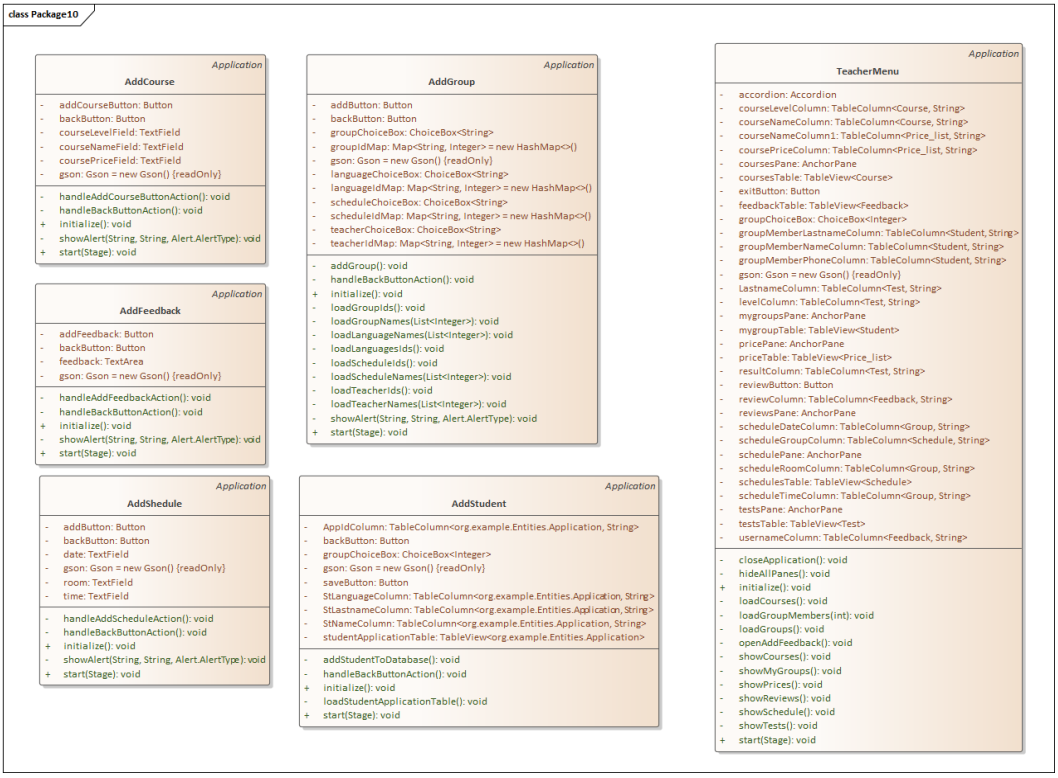


Рисунок 3.20– Диаграмма классов образовательного приложения на JavaFX

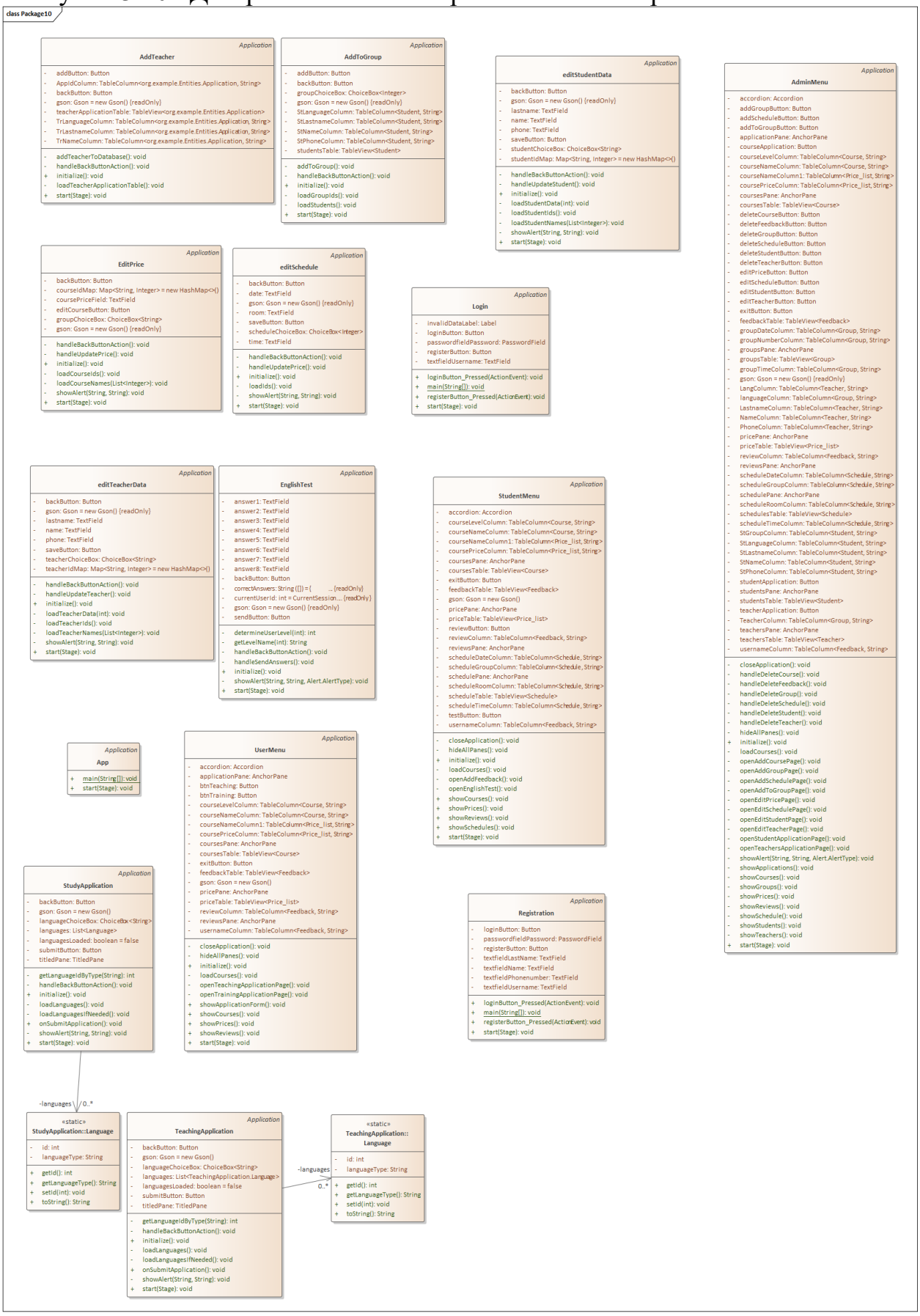


Рисунок 3.21– Диаграмма классов образовательного приложения на JavaFX

3.4 Описание статических и динамических аспектов поведения программных объектов

UML-диаграммы расшифровываются как Unified Modeling Language. Это стандарт, который в первую очередь используется для создания объектно-ориентированных моделей и документации для любых программных систем, представленных в реальном мире. UML позволяет разрабатывать детализированные модели, которые описывают работу различных программно-аппаратных систем.

UML является отличным инструментом для создания профессиональной документации, которая является необходимым элементом любой разработки проекта. Кроме того, он играет ключевую роль в создании объектно-ориентированных проектных систем.

Диаграмма последовательностей является ключевым инструментом визуализации взаимодействия между компонентами системы в рамках выполнения конкретного бизнес-процесса. Она относится к языку UML и позволяет наглядно отобразить хронологический порядок обмена сообщениями между объектами, что особенно важно для понимания динамики работы системы. Основное преимущество такой диаграммы заключается в её способности демонстрировать временные отношения между операциями, помогая разработчикам и аналитикам выявлять потенциальные проблемы и оптимизировать процессы[13].

Основные элементы диаграммы последовательности включают в себя:

− стрелки, указывающие на обмен сигналами или сообщениями между объектами;

− прямоугольники, отображающие действия объекта или выполнение им определённой функции (прямоугольники расположены на пунктирной «линии жизни»);

− вертикальные «линии жизни», показывающие течение времени;

− обозначения объектов (прямоугольники с названиями объектов).

На представленной диаграмме (рисунок 3.22) изображён процесс подачи и обработки заявки в информационной системе. Процесс начинается с действия пользователя, который нажимает кнопку "Оставить заявку". Это инициирует цепочку взаимодействий между компонентами системы:

Инициация процесса: пользователь отправляет запрос через интерфейс, что приводит к созданию заявки и передаче данных в систему.

1 Обработка заявки: компонент StudyApplication.java обрабатывает запрос, используя метод HandleAddStudent(request), где выполняется валидация и подготовка данных.

2 Хеширование данных: для обеспечения безопасности и целостности данных система дважды применяет функцию HashHash(application), что может указывать на дополнительные проверки или шифрование информации.

3 Сохранение заявки: обработанные данные передаются в ApplicationService.java, который взаимодействует с ApplicationDAO.java для сохранения заявки в базе данных MySQL.

4 Подтверждение успешного выполнения: после сохранения данных пользователь получает уведомление "Заявка добавлена успешно", что завершает процесс.

Особенностью данной диаграммы является чёткая последовательность операций, включая этапы обработки, проверки и сохранения данных. Взаимодействие между ClientThread.java, ApplicationService.java и ApplicationDAO.java подчёркивает распределённую архитектуру системы, где каждый компонент выполняет свою роль. Визуализация этих шагов позволяет легко идентифицировать этапы процесса, что делает диаграмму ценным инструментом для анализа и документирования бизнес-процесса подачи заявки.

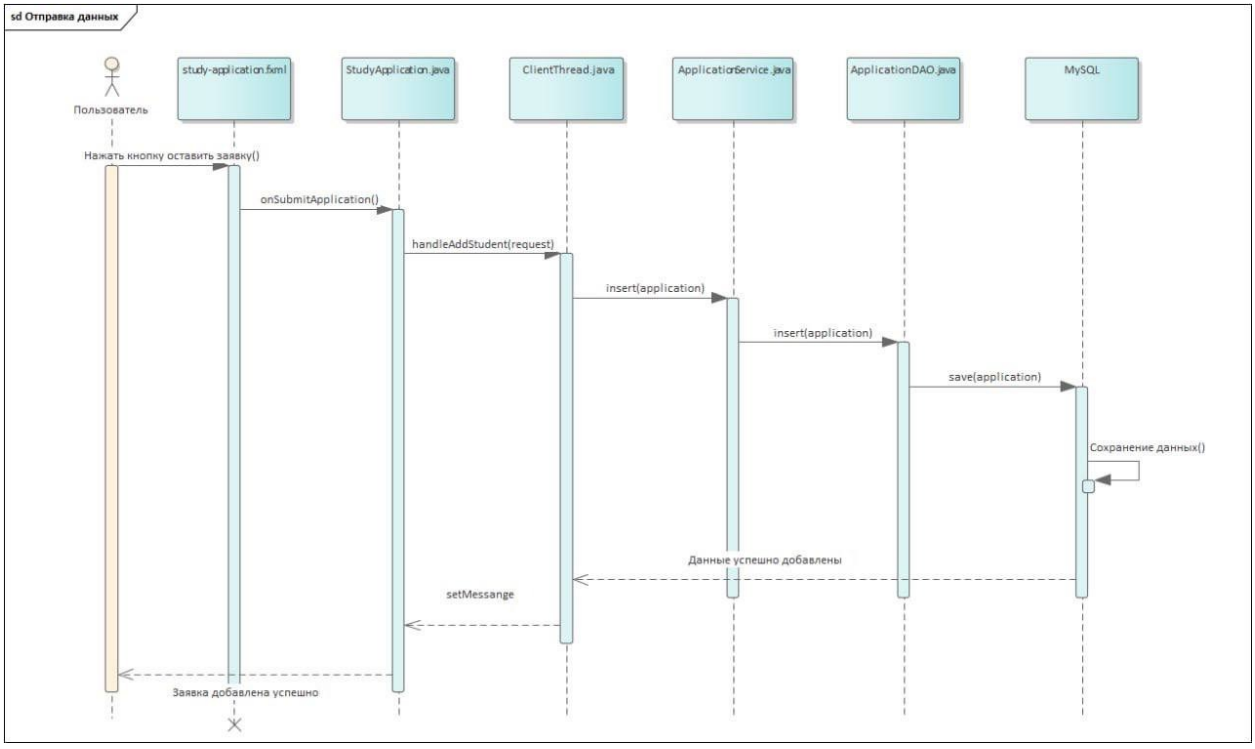


Рисунок 3.22 – Диаграмма последовательности процесса «Оставить заявку на обучение» программного средства

Диаграмма состояний отображает переходы объекта из одного состояния в другое. Эти диаграммы предназначены для моделирования динамических аспектов системы и полезны при анализе жизненного цикла объекта. В отличие от других типов диаграмм, диаграмма состояний фокусируется на изменениях состояний только одного экземпляра конкретного класса – одного объекта, который является реактивным, то есть его поведение определяется его реакцией на внешние события.

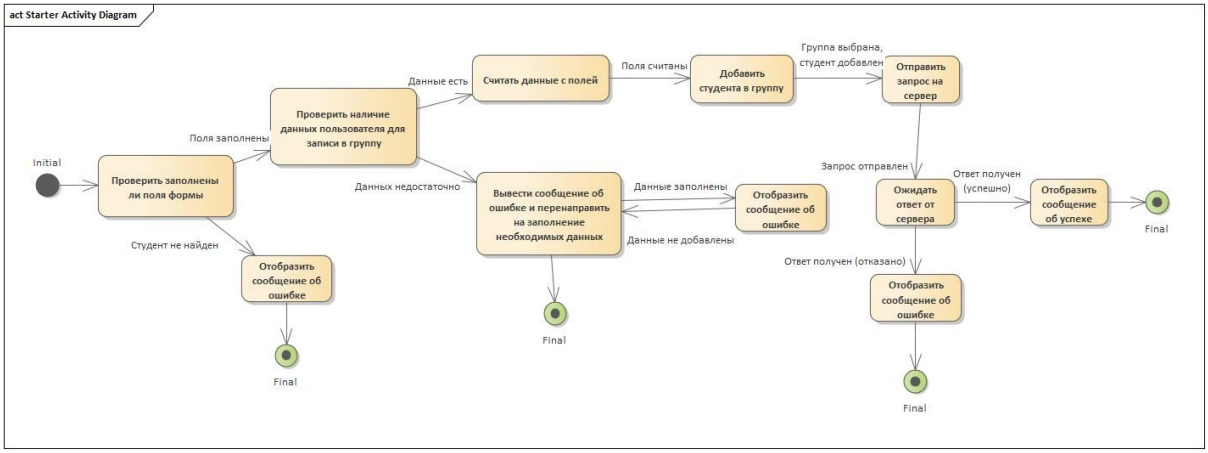


Рисунок 3.23 –Диаграмма состояний системы

На представленной диаграмме активности (рисунок 3.23) отображен процесс обработки заявки на добавление студента в учебную группу с детализацией всех возможных сценариев выполнения. Диаграмма начинается с проверки полноты заполнения полей формы, где система анализирует наличие обязательных данных. Если поля остаются незаполненными, пользователь получает соответствующее уведомление об ошибке с возвратом к начальному этапу для повторного ввода информации.

При успешной проверке заполнения полей система переходит к верификации данных конкретного студента в базе данных. На этом этапе возможны два варианта развития событий: если студент не найден или предоставленных данных недостаточно, процесс прерывается с выдачей пользователю информационного сообщения, после чего система вновь предлагает заполнить недостающие сведения. В случае положительного результата проверки система выполняет операцию добавления студента в указанную учебную группу.

Следующим критически важным этапом является отправка сформированного запроса на сервер для фиксации изменений. Здесь также предусмотрена обработка возможных ошибок передачи данных с информированием пользователя о возникших проблемах. Финальная стадия процесса предполагает два альтернативных исхода: при успешной обработке запроса сервером пользователь получает подтверждение о корректном выполнении операции, тогда как в случае отказа система формирует и отображает сообщение об ошибке с указанием причины.

Ключевой особенностью данной диаграммы является ее разветвленная структура, наглядно демонстрирующая не только основной сценарий выполнения операции, но и все возможные пути обработки исключительных ситуаций. Четко прослеживается циклическая природа процесса, позволяющая системе возвращаться на предыдущие этапы при необходимости исправления ошибок или уточнения данных. Визуализация включает явные критерии перехода между состояниями, что обеспечивает однозначность трактовки условий выполнения каждого действия. Подобное представление

процесса значительно упрощает анализ взаимодействия пользователя с системой и помогает выявить потенциальные точки отказа, делая диаграмму ценным инструментом как для разработчиков, так и для бизнес-аналитиков при проектировании и оптимизации функционала.

Диаграмма компонентов (Component Diagram) отображает структурные элементы информационной системы (ИС) и их взаимосвязи. К компонентам относятся информационные объекты, такие как файлы, модули, библиотеки, пакеты и т.д.

Компонент представляет собой физически существующую часть системы, которая отвечает за реализацию классов, их взаимосвязей и обеспечивает функциональность моделируемой информационной системы.

Диаграмма компонентов используется для описания архитектуры системы, её компонентов, их связей и интерфейсов, что позволяет получить общее представление о структуре системы. Диаграмма компонентов представлена на рисунке 3.24.

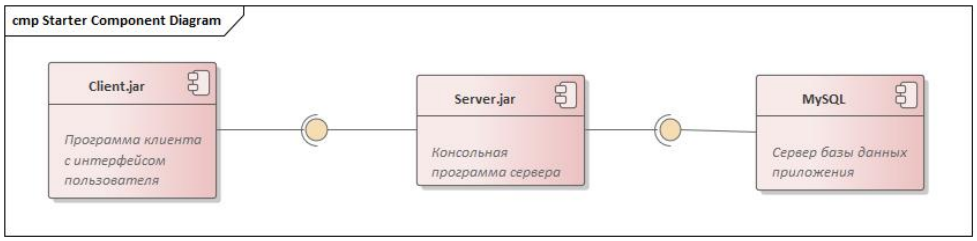


Рисунок 3.24 – Диаграмма компонентов

3.5 Разработка и описание алгоритмов, реализующих бизнес-логику программного средства.

Алгоритм представляет собой чёткую последовательность шагов или действий, выполнение которых приводит к заранее определённому результату. Иными словами, это набор инструкций для эффективного решения задачи.

Алгоритм бизнес-логики – это совокупность правил и инструкций, определяющих порядок действий для выполнения операций в соответствии с требованиями бизнеса. Он представляет собой реализацию операций, характерных для определённой предметной области [14].

В ходе разработки приложения было внедрено множество алгоритмов, предназначенных для обработки данных, поступающих от пользователей и администраторов, а также данных, извлекаемых из базы. Рассмотрим более подробно некоторые из них.

На рисунке 3.25 представлена блок-схема алгоритма обработки клиентского запроса на создание заявки на обучение. Этот процесс является частью серверной логики взаимодействия с пользователями автоматизированной информационной системы.

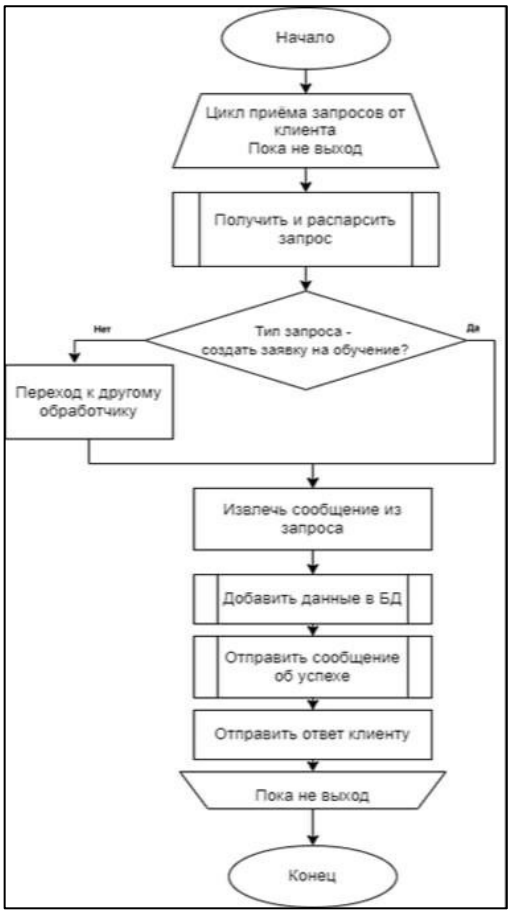


Рисунок 3.25 – Блок-схема алгоритма «Оставить заявку на обучение» со стороны сервера

Процесс начинается с запуска цикла, в котором сервер ожидает входящих запросов от клиента. Цикл выполняется до тех пор, пока не поступит команда завершения (выход из системы). После получения очередного запроса, он передаётся на этап разбора, где извлекаются его структура и параметры.

Далее осуществляется проверка типа запроса. Если он не связан с созданием заявки на обучение, управление передаётся другому обработчику, ответственному за соответствующий функционал. Если же тип запроса соответствует созданию заявки, выполняется серия действий: из тела запроса извлекается сообщение, данные сохраняются в базе данных, клиенту отправляется подтверждение об успешной операции, а затем формируется и отправляется итоговый ответ.

Таким образом, данный алгоритм обеспечивает корректную последовательную обработку одного из основных сценариев работы клиента с системой — подачу заявки на обучение. За счёт цикличной структуры достигается непрерывность взаимодействия с клиентом, а этапы проверки и сохранения данных гарантируют целостность и надёжность операций.

На рисунке 3.26 представлена блок-схема «добавить студента группы».

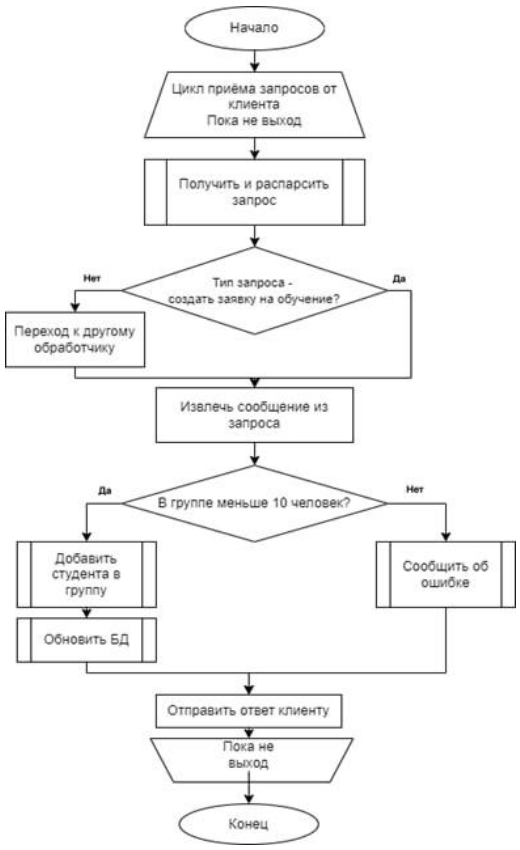


Рисунок 3.26 – Блок-схема алгоритма «Добавить студента в группу» со стороны сервера

Этот алгоритм уточняет поведение системы при добавлении нового студента в группу, что повышает гибкость и точность контроля над учебным процессом.

Процесс начинается аналогично — с запуска сервера и входа в цикл ожидания запросов. После получения и разбора запроса осуществляется проверка его типа. В случае, если тип не соответствует созданию заявки на обучение, управление передаётся другому обработчику.

Если тип запроса соответствует ожидаемому, система извлекает сообщение и приступает к проверке количества участников в целевой группе. Ключевым условием является лимит в 10 человек. Если в группе менее 10 студентов, система добавляет нового участника, обновляет соответствующую запись в базе данных и формирует ответ, который затем отправляется клиенту.

Если же лимит превышен, вместо выполнения операций добавления система отправляет клиенту сообщение об ошибке, указывая на невозможность включения нового участника в группу.

Представленный алгоритм демонстрирует важные принципы защиты от некорректного использования ресурсов системы (переполнения групп), а также взаимодействия с клиентом в формате понятной обратной связи. Благодаря такой структуре обеспечивается автоматизированный контроль за численностью учебных групп и повышается надёжность функционирования образовательной платформы.

3.6 Механизмы обеспечения информационной безопасности

Разрабатываемая система автоматизации работы школы иностранных языков предусматривает комплексную реализацию механизмов информационной безопасности, ориентированную на защиту персональных данных студентов и сотрудников, а также на сохранение целостности и конфиденциальности учебной и административной информации.

В основе архитектуры безопасности лежит принцип многоуровневой защиты, включающей идентификацию и аутентификацию пользователей, разграничение прав доступа, шифрование данных, защиту от сетевых угроз и мониторинг действий в системе. Все пользователи разделены на категории: администраторы, пользователи, преподаватели и студенты. Каждой категории присвоены строго ограниченные права, соответствующие их роли и функциональным задачам, что исключает возможность несанкционированного доступа к закрытой информации.

Для авторизации пользователей используется безопасная форма входа с обязательной валидацией логина и пароля. Важные операции, такие как изменение данных, написание отзывов и управление расписанием, доступны только авторизованным пользователям с соответствующими правами.

Передача данных между клиентской частью (веб-интерфейсом) и сервером защищена с использованием HTTPS-протокола, который обеспечивает шифрование трафика и предотвращает перехват информации в процессе передачи. Для хранения паролей применяется алгоритм хеширования с уникальной солью, что делает невозможным восстановление паролей даже при получении доступа к базе данных.

Особое внимание уделяется защите от типичных уязвимостей веб-приложений, таких как SQL-инъекции. Все запросы к базе данных реализованы с использованием параметризованных выражений, а пользовательский ввод проходит обязательную фильтрацию и валидацию. Это исключает возможность внедрения вредоносного кода и нарушения целостности данных.

В целях обеспечения прозрачности и контроля за действиями пользователей в системе реализован механизм логирования. Журналы событий фиксируют ключевые операции, включая вход в систему, изменения в расписании, добавление новых пользователей и тд. Эти данные позволяют своевременно выявлять и предотвращать попытки несанкционированного доступа или нарушения правил использования системы.

Таким образом, система автоматизации школы иностранных языков строится на современных принципах информационной безопасности, что обеспечивает надежную защиту персональных данных, стабильную работу программного обеспечения и доверие со стороны пользователей.

4 ТЕСТИРОВАНИЕ И ПРОВЕРКА РАБОТОСПОСОБНОСТИ ПРОГРАММНОГО СРЕДСТВА

Для обеспечения правильного функционирования системы учёта студентов и курсов в школе иностранных языков было решено провести всестороннее тестирование приложения. Это позволит своевременно обнаружить возможные уязвимости и устранить их, гарантируя пользователю беспроблемное взаимодействие с системой.

Метод тестирования. В данном проекте было выбрано тестирование, основанное на сценариях использования (use case testing). Этот метод позволяет создавать тест-кейсы, описывающие, как конечные пользователи взаимодействуют с системой, что в свою очередь помогает убедиться, что приложение функционирует в соответствии с требованиями. Сценарное тестирование подразделяется на два типа: высокоуровневое и низкоуровневое.

Высокоуровневые тест-кейсы. Эти тесты проверяют ключевые, основные процессы приложения, такие как регистрация пользователя, добавление нового курса, запись студента на курс и т.д. Высокоуровневые тест-кейсы направлены на проверку общей логики работы системы и чаще всего состоят из минимального количества деталей, фокусируясь на последовательности действий пользователя. Такие тесты помогают оценить работоспособность крупных функций системы и дают базовое представление о том, как приложение ведет себя в целом. В рамках высокоуровневого тестирования используются инструменты для UI- и интеграционного тестирования. Например, высокоуровневый тест может проверять сценарий регистрации нового студента, включая действия по заполнению формы, проверку правильности введенных данных и успешную запись информации о студенте в базу данных. Такие тесты часто служат основой для более детализированных низкоуровневых тест-кейсов.

Низкоуровневые тест-кейсы. Эти тесты фокусируются на проверке мелких и детализированных аспектов системы, например, отдельных полей ввода или валидации данных. Низкоуровневые тесты позволяют проверить правильность выполнения конкретных методов, что помогает обнаружить ошибки на уровне реализации. Для низкоуровневого тестирования в проекте будет использован фреймворк JUnit, который предоставляет удобные методы для проверки отдельных модулей кода. В таких тестах указываются входные данные, ожидаемый результат и метод, который должен быть выполнен; JUnit проверяет, совпадает ли фактический результат с ожидаемым, и позволяет оценить корректность работы каждого отдельного компонента. Например, один из низкоуровневых тестов может проверить корректность валидации телефона студента в форме регистрации: если данные введены неверно, система должна отобразить соответствующее сообщение.

Таблица 4 ниже включает основные тест-кейсы для проверки пользовательских требований системы учёта школы иностранных языков.

Таблица 4 – Тест-кейсы для проверки пользовательских требований

Идентификатор тест-кейса	Заглавие тест- кейса	Шаги тест-кейса	Ожидаемый результат
1	Регистрация пользователя	1 Открыть страницу регистрации. 2 Ввести имя пользователя, фамилию, имя, номер телефона и пароль. 3 Нажать кнопку «Зарегистрироваться».	1 Появляется уведомление об успешной регистрации. 2 Пользователь перенаправляется на страницу меню пользователя. 3 Данные сохраняются в БД.
2	Валидация имени пользователя	1 Открыть страницу регистрации. 2 Ввести username (которое уже есть в системе). 3 Нажать кнопку «Зарегистрироваться».	1 Появляется ошибка «Некорректное имя пользователя» под полем ввода. 2 Регистрация не выполняется, пока имя пользователя не станет валидным.
3	Добавление нового курса	1 Перейти в раздел «Управление курсами» администратора. 2 Нажать кнопку «Добавить курс». 3 Заполнить поля курса (название, описание, уровень, цена). 4 Нажать кнопку «Сохранить».	1 Появляется уведомление об успешном добавлении курса. 2 Курс отображается в списке доступных курсов и прайс-листе. 3 Данные о курсе сохраняются в БД.

Продолжение таблицы 4

Идентификатор тест-кейса	Заглавие тест- кейса	Шаги тест-кейса	Ожидаемый результат
4	Изменение статуса заявки	1 Перейти в раздел «Заявки на обучение» администратора. 2 Выбрать заявку студента со статусом «Ожидает рассмотрения». 3 Изменить статус на «Принято» или «Отклонено». 4 Выбрать курс и группу и язык 5 Нажать «Сохранить».	1 Статус заявки обновляется на «Принято» или «Отклонено». 2 Заявка сохраняется с новым статусом в БД. 3 Студент добавляется в соответствующую таблицу в БД. 4 Появляется уведомление об изменении статуса.
5	Создание заявки на преподавание	1 Открыть страницу подачи заявки на преподавание. 2 Ввести данные о преподавателе (указать язык). 3 Нажать кнопку «Отправить заявку».	1 Появляется уведомление об успешной отправке заявки. 2 Заявка сохраняется в БД со статусом «Ожидает рассмотрения»

Рассмотрим низкоуровневое тестирование системы управления школой иностранных языков с использованием фреймворка JUnit. Для изоляции тестов и предотвращения загрязнения основной базы данных будет использоваться легковесная реляционная база данных H2. Она работает в оперативной памяти и позволяет создавать и удалять базу данных целиком при каждом запуске теста, что делает её оптимальным выбором для тестирования [15].

Первым модулем для проверки является функция регистрации пользователя. При регистрации пользователь должен ввести корректные данные, такие как имя пользователя, адрес электронной почты и пароль. Валидация данных происходит следующим образом:

1 Если имя пользователя уже существует, выбрасывается исключение.

2 Если email имеет некорректный формат, также выбрасывается исключение.

3 При успешной регистрации, пользователь сохраняется в базе данных.

Результат выполнения тестов будет отражать успешность регистрации и обработку исключений.

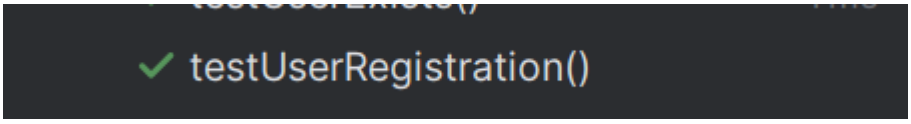


Рисунок 4.1 – Результат выполнения теста на регистрацию пользователя

Следующим модулем будет функция добавления нового курса. При добавлении курса необходимо выполнить следующие проверки:

1 Название курса не должно быть пустым.

2 Уровень курса должен соответствовать одному из заранее определенных значений (например, "Начальный", "Средний", "Продвинутый").

3 При успешном добавлении курс должен быть сохранен в базе данных.

Тесты будут проверять успешное добавление курсов и обработку ошибок при вводе некорректных данных.

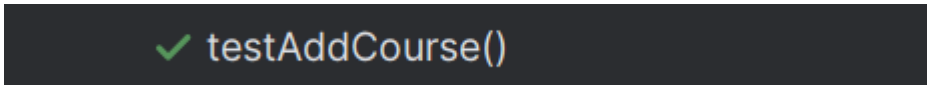


Рисунок 4.2 – Результат выполнения теста на добавления курса

` Еще одной важной функцией является проверка существования пользователя в системе. При создании заявки на обучение необходимо удостовериться, что указанный пользователь существует в базе данных. Если пользователь не найден, выбрасывается соответствующее исключение.
Для тестирования этой функции будет создана база данных с тестовыми пользователями.

Результаты тестов будут подтверждать наличие или отсутствие исключений при попытке найти пользователя.

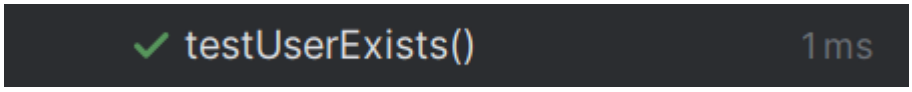


Рисунок 4.3 – Результат выполнения теста на проверку существования пользователя

Следующим шагом будет проверка роли пользователя. При обработке заявки необходимо убедиться, что пользователь имеет роль "USER". Если роль отличается, выбрасывается исключение с соответствующим сообщением. Тесты будут нацелены на корректность определения ролей пользователей и обработку исключений.

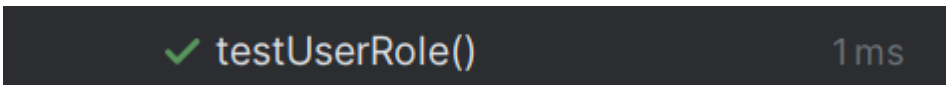


Рисунок 4.4 – Результат выполнения теста на проверку роли пользователя

Последним тестируемым модулем будет проверка наличия свободных мест на курсе. При создании заявки на обучение необходимо проверить, достаточно ли мест для нового студента. Если количество мест недостаточно, выбрасывается исключение. Тесты будут проверять корректность работы этой логики.



Рисунок 4.5 – Результат тестирования проверки наличия свободных мест на курсе

Листинг кода всех упомянутых низкоуровневых тестов представлен в приложении Г.

Программный продукт был протестирован с использованием методики use case тестирования, которая предусматривает проверку сценариев использования системы. Этот подход включает как высокоуровневое, так и низкоуровневое тестирование.

На высоком уровне тесты описываются в виде общих сценариев, охватывающих ключевые функциональные возможности системы. Такие тесты не содержат конкретных входных данных или ожидаемых результатов, их цель – убедиться в корректности выполнения базовых операций. Это позволяет сосредоточиться на оценке общего поведения системы в реальных условиях использования.

Низкоуровневое тестирование основывалось на детальной проверке соответствия фактических результатов работы системы ожидаемым значениям. Для этого разрабатывались тест-кейсы с конкретными входными данными и четко определенными критериями успешности. Это помогло выявить ошибки в реализации функциональности и подтвердить точность выполнения ключевых операций.

Таким образом, комбинированный подход use case тестирования обеспечил всестороннюю проверку программного продукта, охватывая как стратегические, так и тактические аспекты его функционирования. Это позволило выявить критически важные сценарии взаимодействия пользователей с системой и гарантировать их корректную реализацию. Благодаря такому подходу удалось минимизировать риски возникновения ошибок в ключевых бизнес-процессах, что повысило качество и надежность программного обеспечения.

5 РУКОВОДСТВО ПО УСТАНОВКЕ (РАЗВЕРТЫВАНИЮ) И ИСПОЛЬЗОВАНИЮ ПРОГРАММНОГО СРЕДСТВА

5.1 Руководство по установке (развертыванию) программного средства

Диаграмма развертывания является одним из видов UML-диаграмм, которая моделирует физическое размещение и взаимодействие различных компонентов системы в контексте ее аппаратного и программного обеспечения. Она предоставляет визуальное представление того, как артефакты (например, модули программного обеспечения, базы данных, файлы конфигурации) размещаются на физических узлах (аппаратных компонентах) и как эти узлы связаны друг с другом. Такая диаграмма особенно полезна для понимания архитектуры системы и взаимодействия ее частей на уровне инфраструктуры.

Для описания приложения диаграмма развертывания должна включать следующие элементы:

– Узлы устройств (Device Nodes). Узлы устройств представляют собой физические вычислительные ресурсы, такие как серверы, настольные компьютеры, мобильные телефоны, планшеты или другие устройства, на которых выполняется программное обеспечение. Каждый узел имеет свои характеристики, включая память, процессорные мощности и доступные сервисы. Например, для веб-приложения узел устройства может представлять сервер, на котором размещен веб-сервер или API.

– Узлы среды выполнения (Execution Environment Nodes). Узлы среды выполнения моделируют программные вычислительные ресурсы, функционирующие внутри физических узлов. Они обеспечивают выполнение других программных компонентов. Примером узла среды выполнения может быть виртуальная машина Java, веб-сервер (например, Apache Tomcat), контейнер Docker или любая среда выполнения, такая как сервер приложений или платформа мобильной разработки. Эти узлы являются связующим звеном между аппаратным оборудованием и программным обеспечением.

– Артефакты (Artifacts). Артефакты представляют собой физические файлы или программные модули, такие как исполняемые файлы (.exe), библиотеки (.dll, .jar), базы данных или конфигурационные файлы, которые размещены на узлах. На диаграмме отображается, какой артефакт разворачивается на каком узле.

– Связи между узлами (Communication Paths). Диаграмма также моделирует взаимодействие между узлами с помощью коммуникационных путей. Эти связи показывают, как узлы обмениваются данными, например, через сетевые соединения (HTTP, TCP/IP) или взаимодействуют через API.

Диаграмма развёртывания системы представлена на рисунке 5.1.

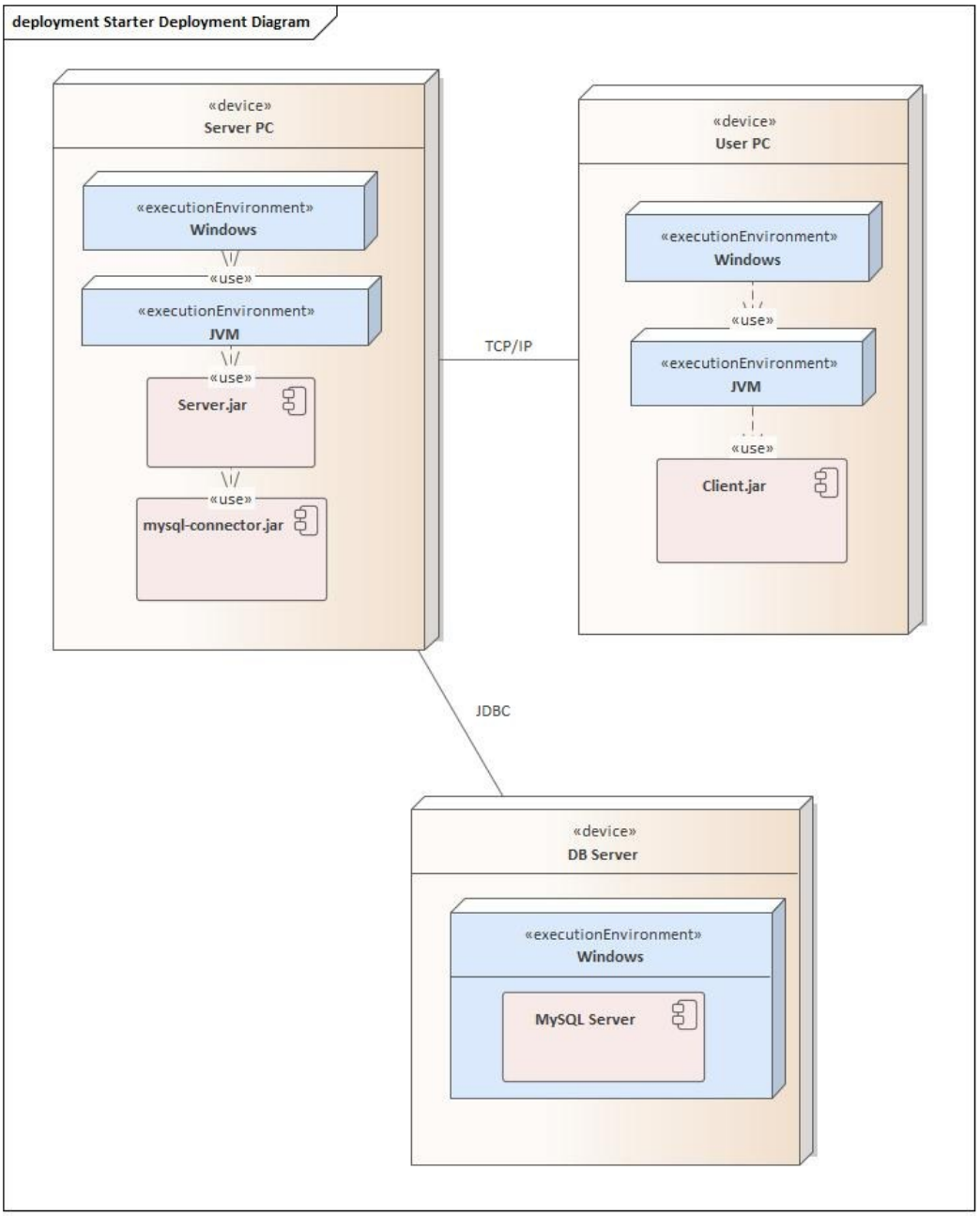


Рисунок 5.1 – Диаграмма развёртывания

Для запуска приложения на любом компьютере необходимо выполнить последовательность действий.

В первую очередь нужно создать базу данных. Для этого следует установить сервер баз данных MySQL, если он еще не установлен, и скачать утилиту MySQL Workbench с официального сайта. После этого создайте новую схему базы данных и назовите schooldb. Убедитесь, что логин и пароль совпадают с данными, указанными в файле config.properties, который находится в папке META-INF. Далее необходимо выполнить SQL-скрипт, расположенный в файле schooldb, чтобы подготовить структуру базы данных. При необходимости откройте файл pom.xml и проверьте, чтобы все зависимости подгрузились корректно.

Для настройки серверной части проекта потребуется установить JDK версии 22, если данный компонент еще не установлен. Затем скачайте приложение IntelliJ IDEA версии 2023 года или более поздней. Для более удобной работы с базой данных рекомендуется использовать версию Ultimate. После установки откройте серверную часть проекта в IntelliJ IDEA, настройте JDK в разделе File -> Project Structure -> Project -> SDK, выбрав JDK 22, и запустите сервер, нажав кнопку «Run» в правом верхнем углу среды разработки.

Для настройки клиентской части приложения также требуется установить JDK версии 22. После этого скачайте IntelliJ IDEA, подходящую для работы с клиентской частью.Проверьте зависимости в файле pom.xml, чтобы все необходимые библиотеки были корректно загружены. После этого выполните запуск клиентской части проекта, предварительно прописав пути к библиотекам JavaFX, далее используя кнопку «Run» в интерфейсе IntelliJ IDEA. Также проверьте конфигурации подключения клиента к серверу, чтобы они совпадали с указанными в файле конфигурации проекта.

Следуя этим инструкциям, вы сможете корректно настроить и запустить как серверную, так и клиентскую части приложения.

5.2 Руководство пользователя

Первое, что встречает пользователь при запуске приложения это страницу авторизации. Программа требует ввести логин и пароль, так же в случае, если у пользователя нет аккаунта, система предлагает зарегистрироваться.

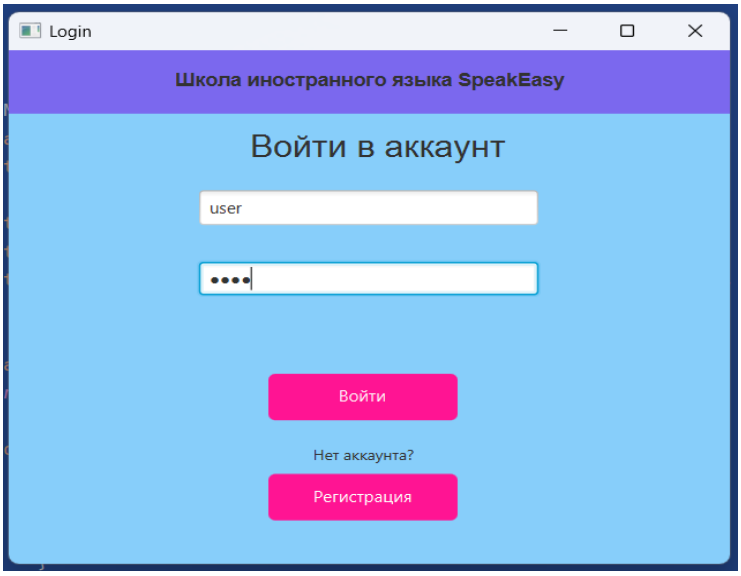


Рисунок 5.2 – Окно авторизации

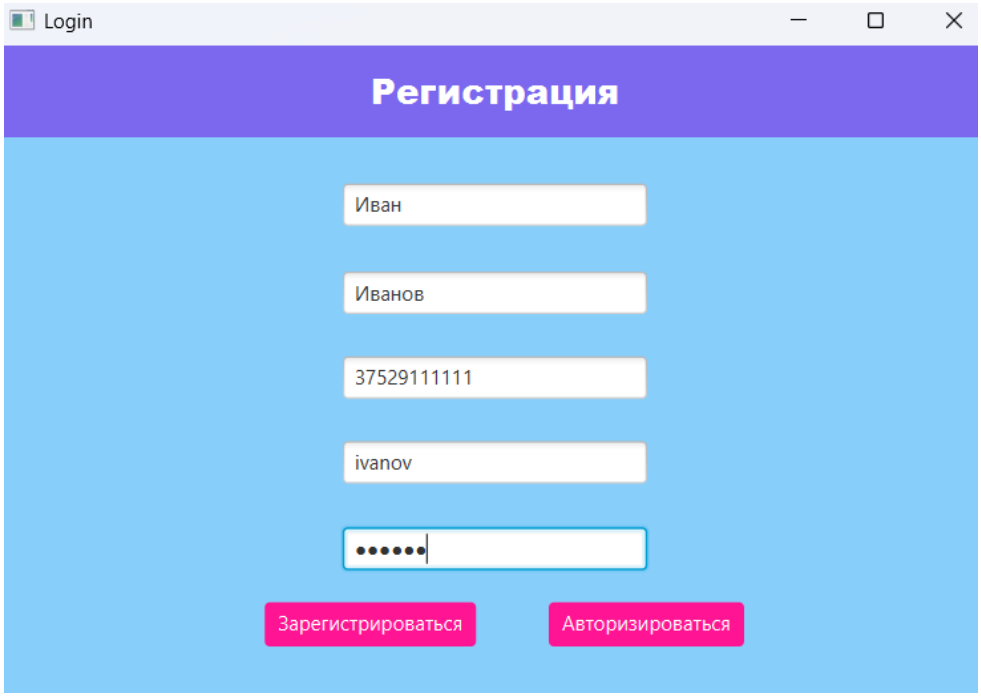


Рисунок 5.3 – Окно регистрации

В случае, если авторизация прошла успешно или же регистрация (имя пользователя оказалось уникальным), выведется сообщение о успехе

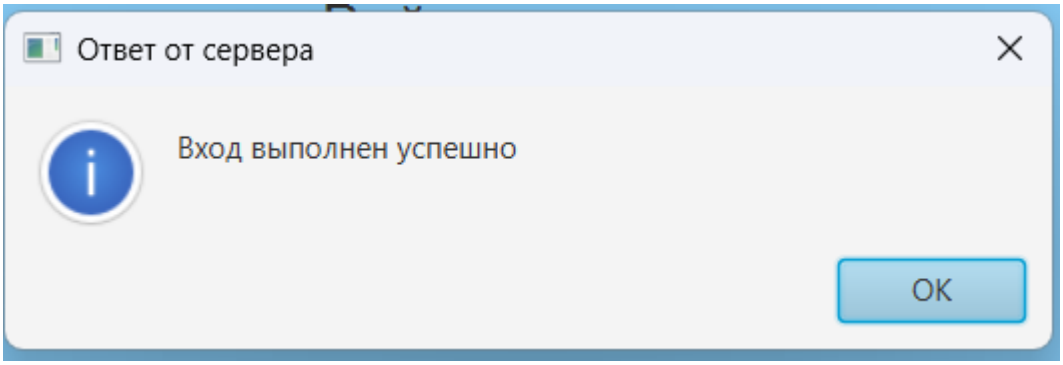


Рисунок 5.4 – Сообщение об успехе

После успешного входа пользователя или регистрации нового пользователя будет представлена панель пользователя, имеющая самый скромный функционал.

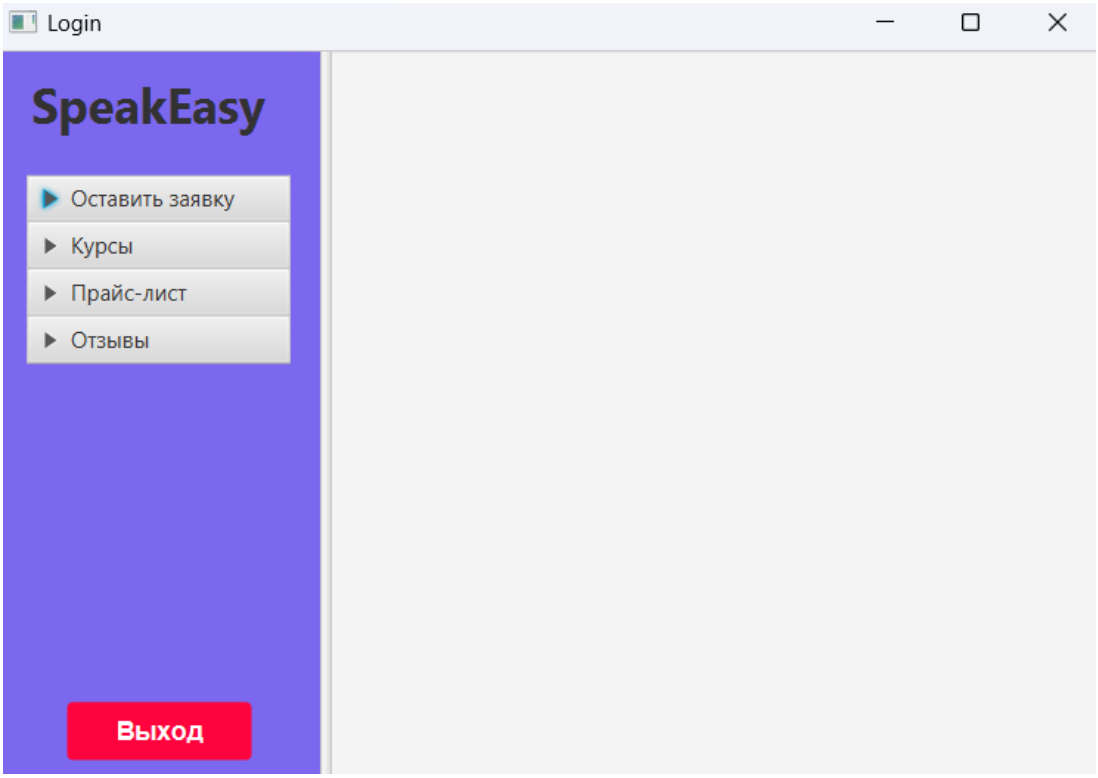


Рисунок 5.5 – Панель пользователя

Пользователь может подать 2 типа заявок: на обучение и на преподавание. В случае одобрения заявки администратором пользователь может стать студентом или преподавателем. В заявке пользователь выбирает язык изучения или язык преподавания.

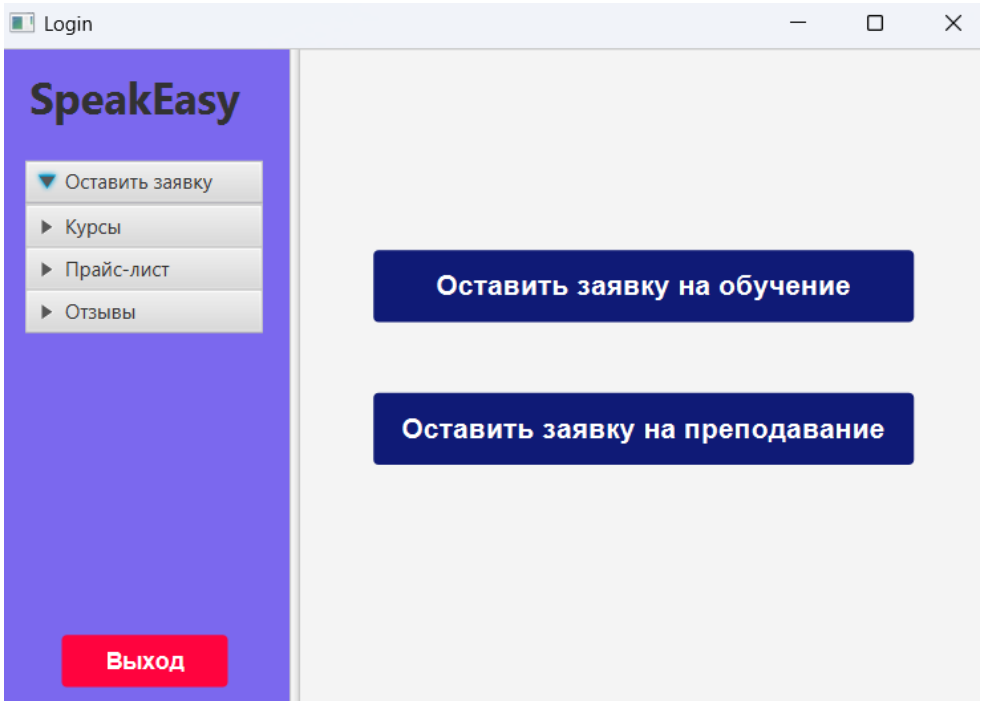


Рисунок 5.6 – Выбор типа заявки

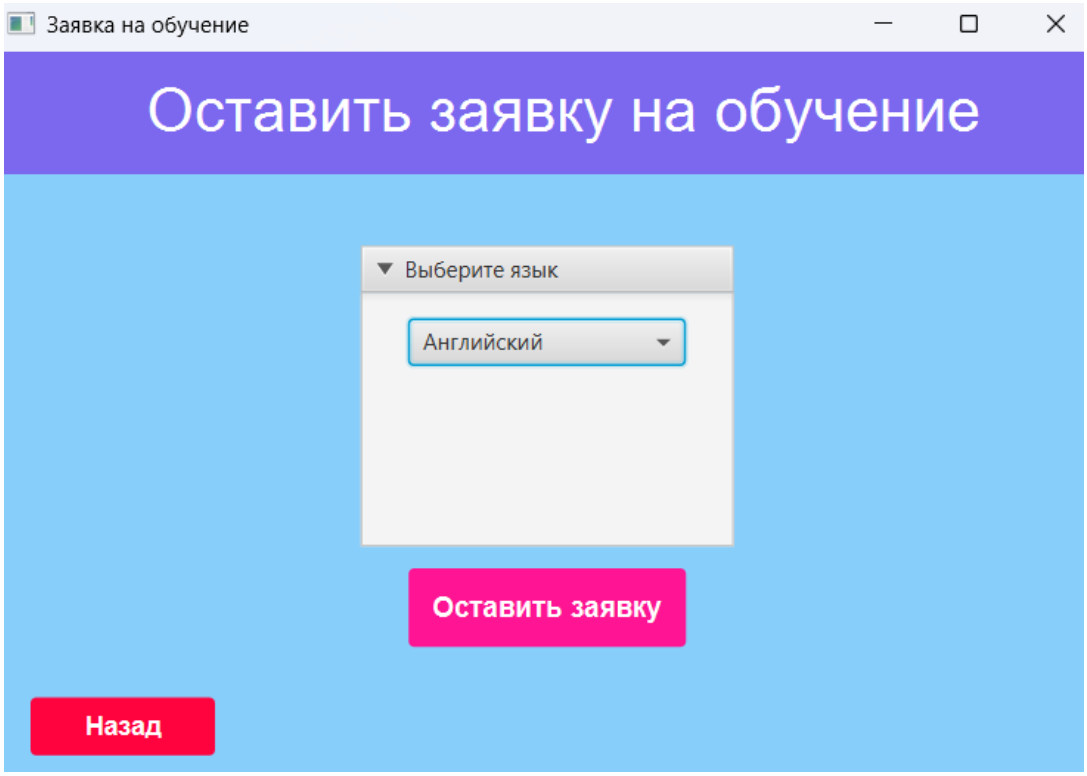


Рисунок 5.7 – Окно подачи заявки на обучение

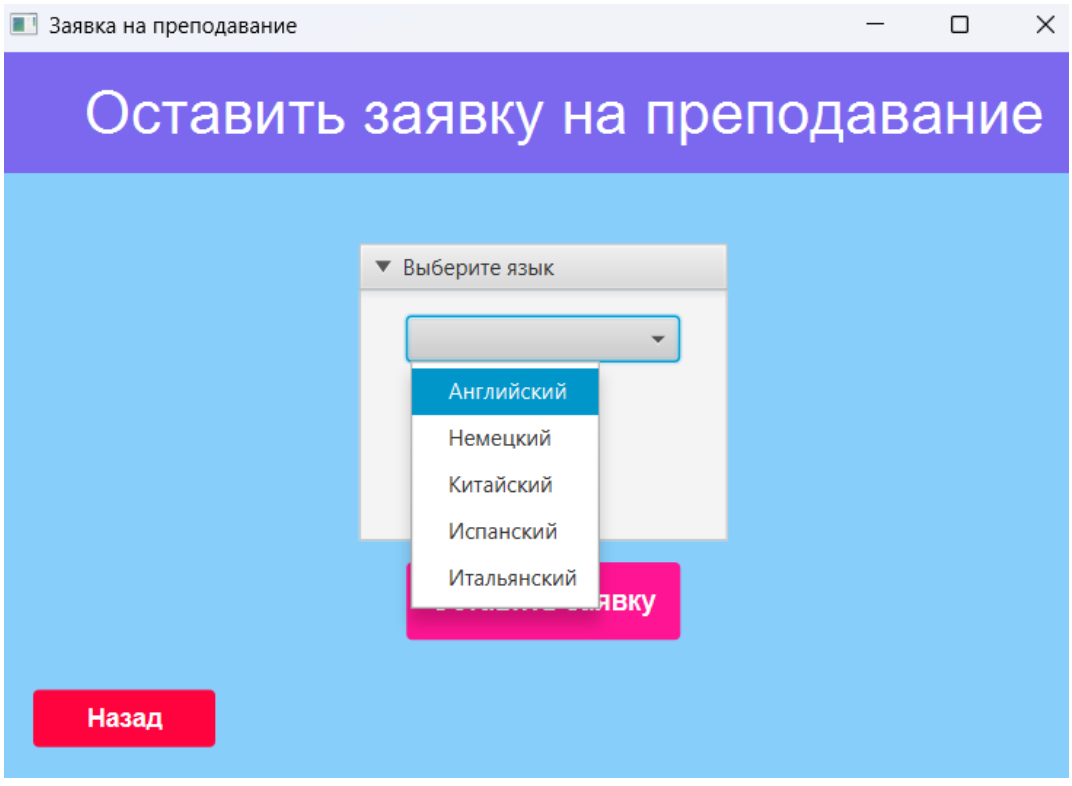


Рисунок 5.8 – Окно подачи заявки на преподавание

Пользователю так же доступен просмотр курсов, прайс-листа и отзывов.

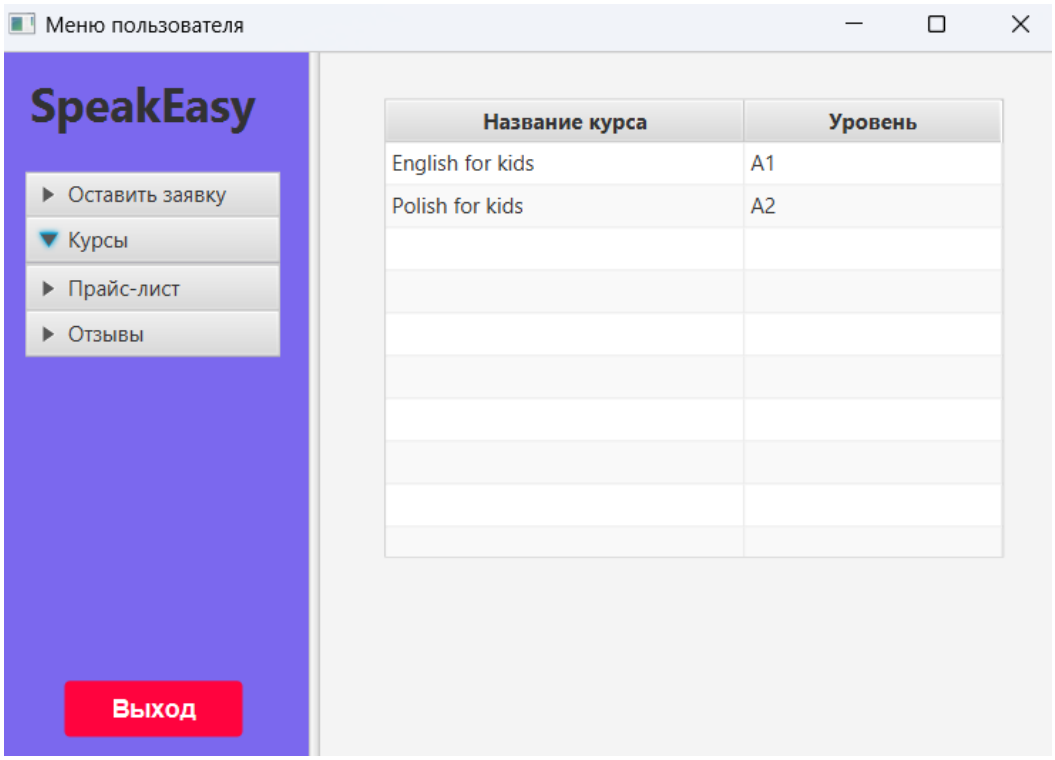


Рисунок 5.9 – Просмотр курсов

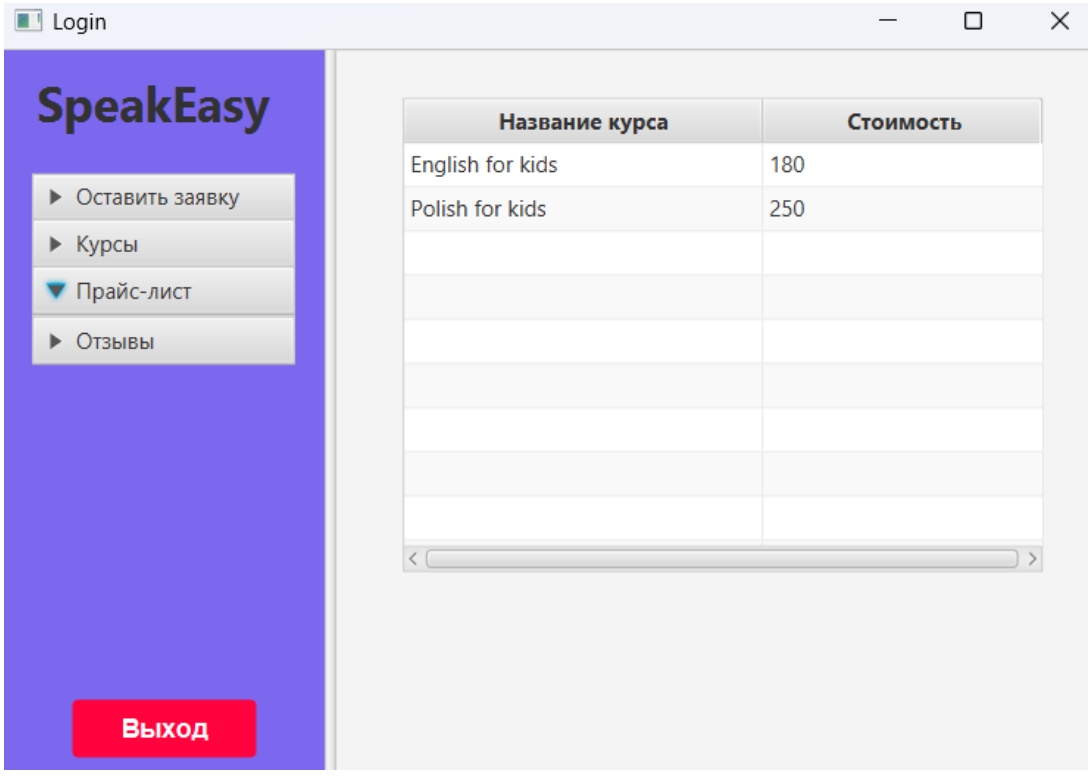


Рисунок 5.10 – Просмотр прайс-листа

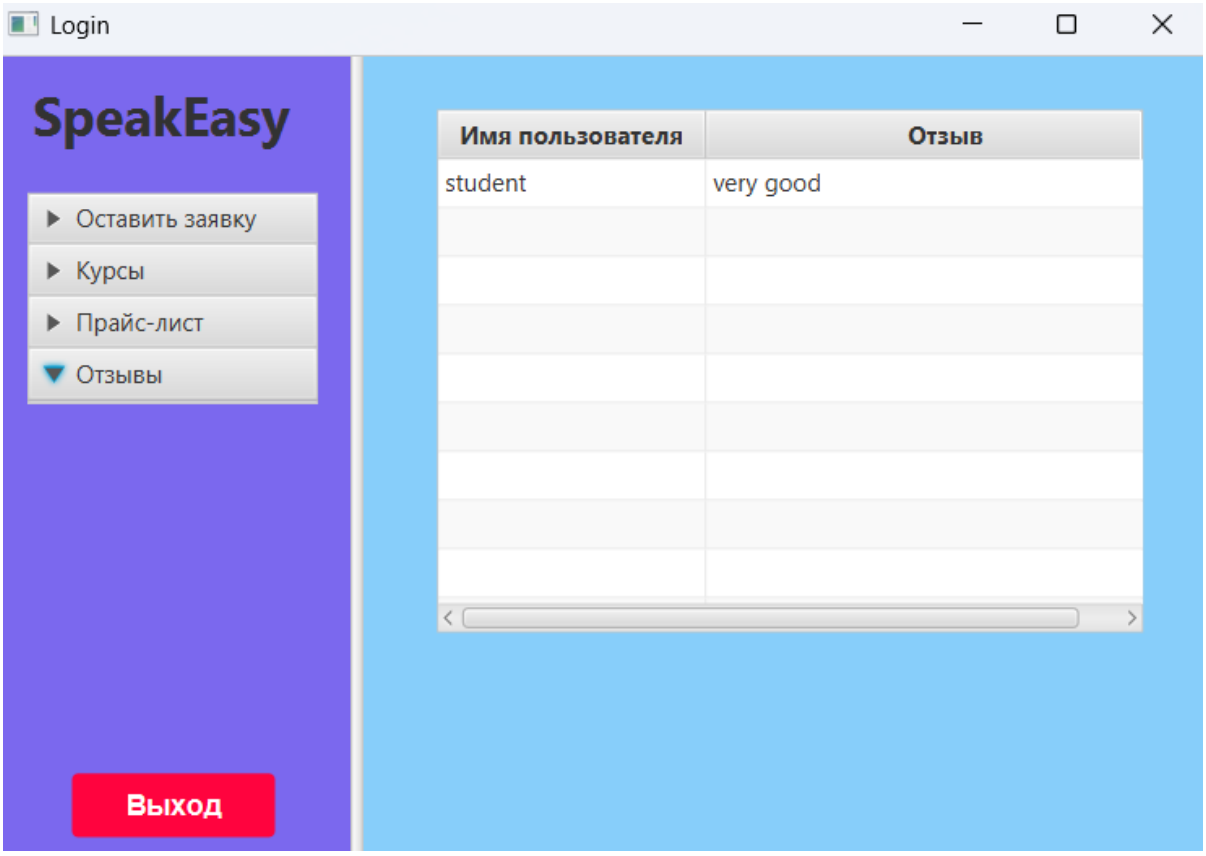


Рисунок 5.11 – Просмотр отзывов

5.3 Руководство студента

Функционал студента ненамного шире, чем функционал простого пользователя. Однако у него есть отличительные особенности.

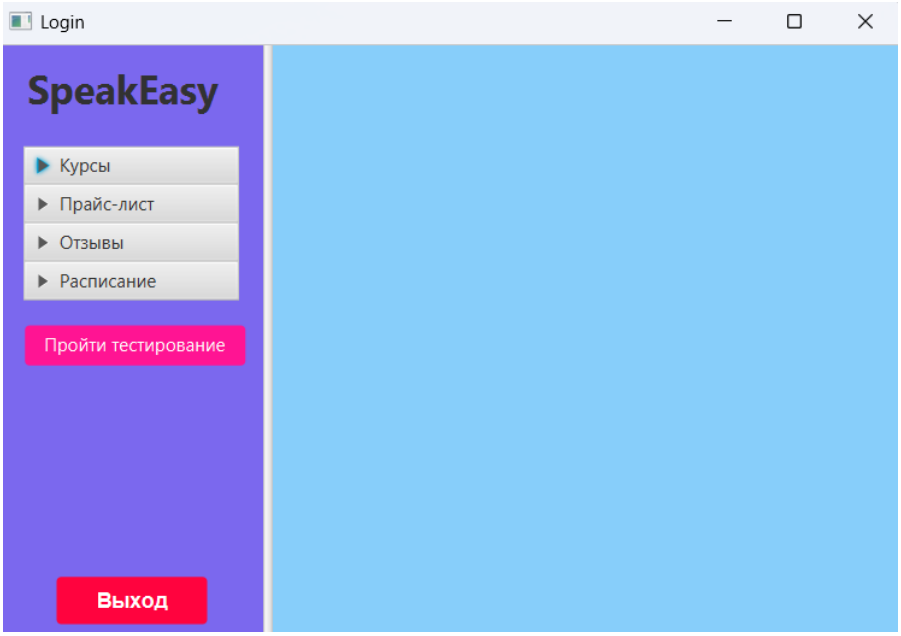


Рисунок 5.12 – Панель студента

Студент, в отличие от обычного пользователя, может оставить отзыв, просмотреть расписание занятий и пройти тестирование, узнав свой уровень английского языка.

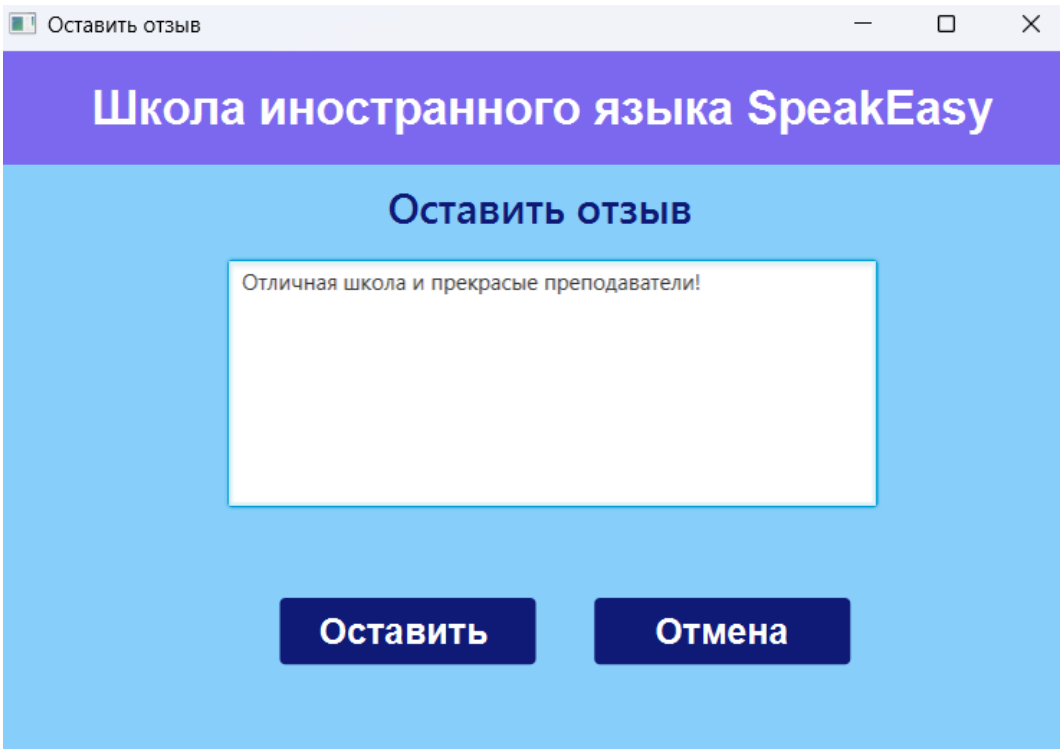


Рисунок 5.13 – Оставить отзыв

После успешной отправки высветится сообщение.

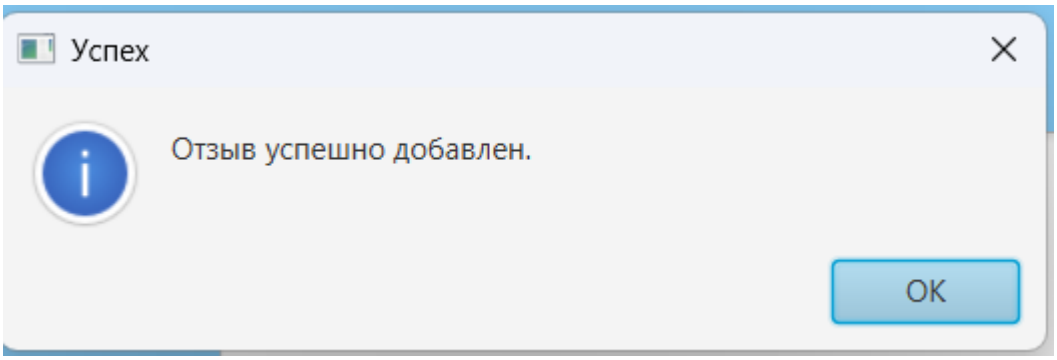


Рисунок 5.14 – Успешно отправленный отзыв

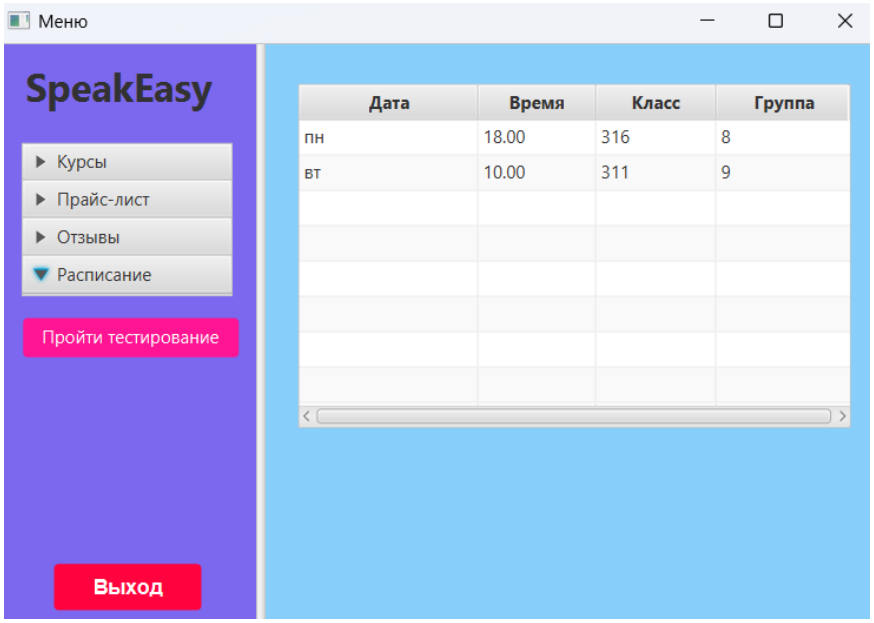


Рисунок 5.15 – Просмотр расписания

Тестирование студента происходит следующим образом. Ему предоставляется форма из 8 вопросов из раздела грамматики английского языка. Задача студента – поставить глаголы в нужную форму в зависимости от контекста предложения. Далее ответы сравниваются с правильными, и выставляется соответствующий балл. По баллам выставляется уровень знаний студента: 0-1 – Beginner, 2 – Elementary, 3 – Pre-intermediate, 4 – Intermediate, 5 – Intermediate Plus, 6 – Upper-intermediate, 7 – Advanced, 8 – Advanced Plus.

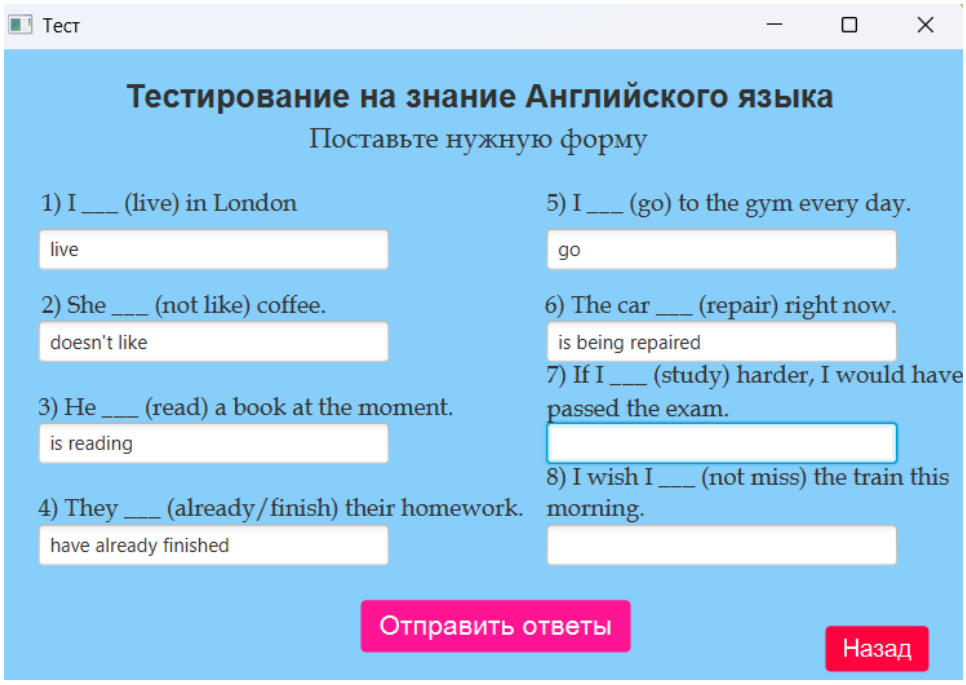


Рисунок 5.16 – Тестирование по английскому языку

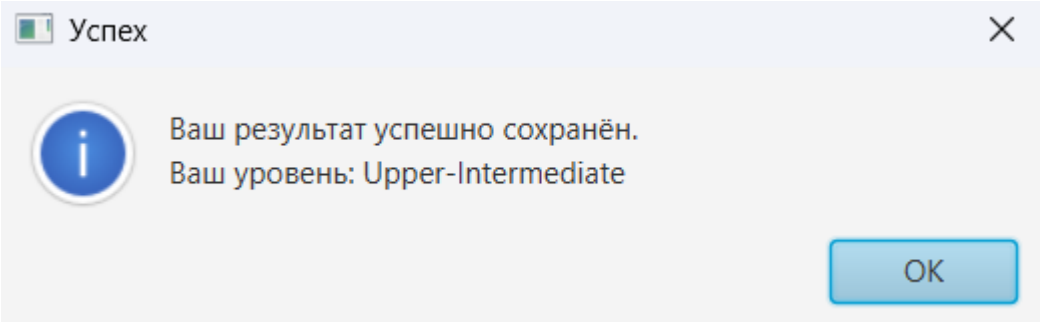


Рисунок 5.17 – Уведомление о результате

5.4 Руководство преподавателя

Функционал преподавателя расширяет функционал студента. Однако он значительно меньше функционала администратора. Отличительной чертой является то, что преподаватель имеет возможность просмотреть результаты тестов студентов, а также просмотреть состав своих групп.

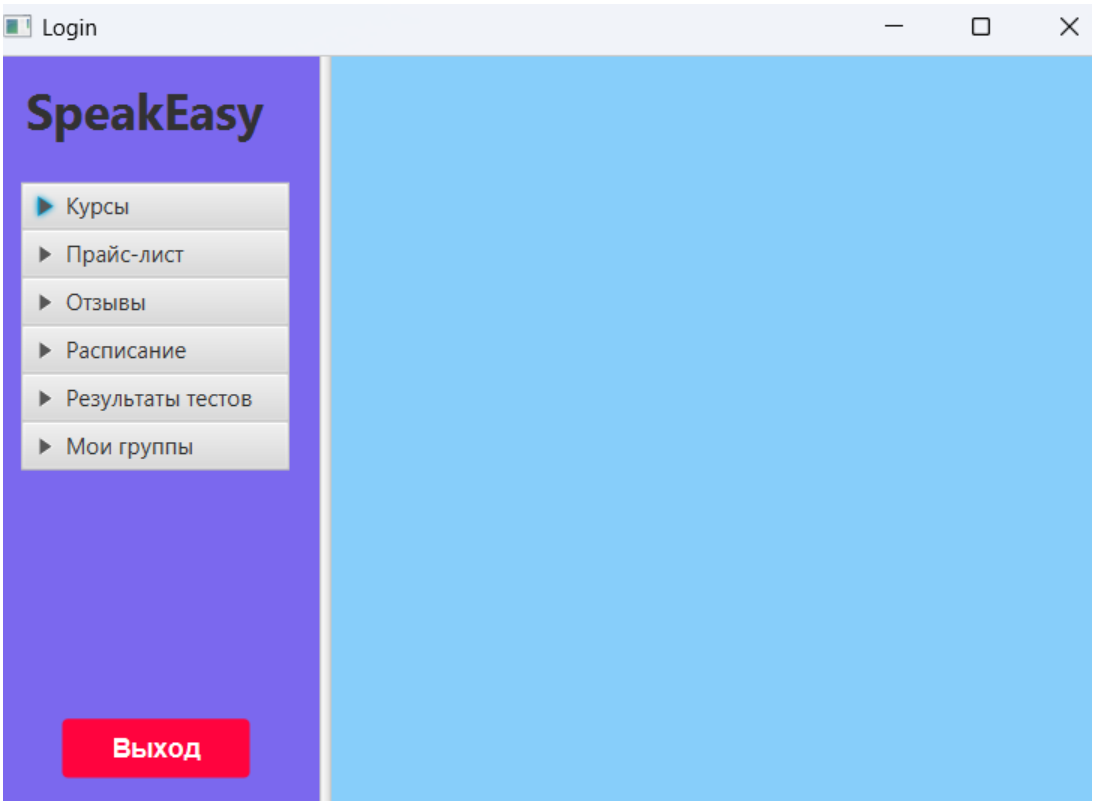


Рисунок 5.18 – Панель преподавателя

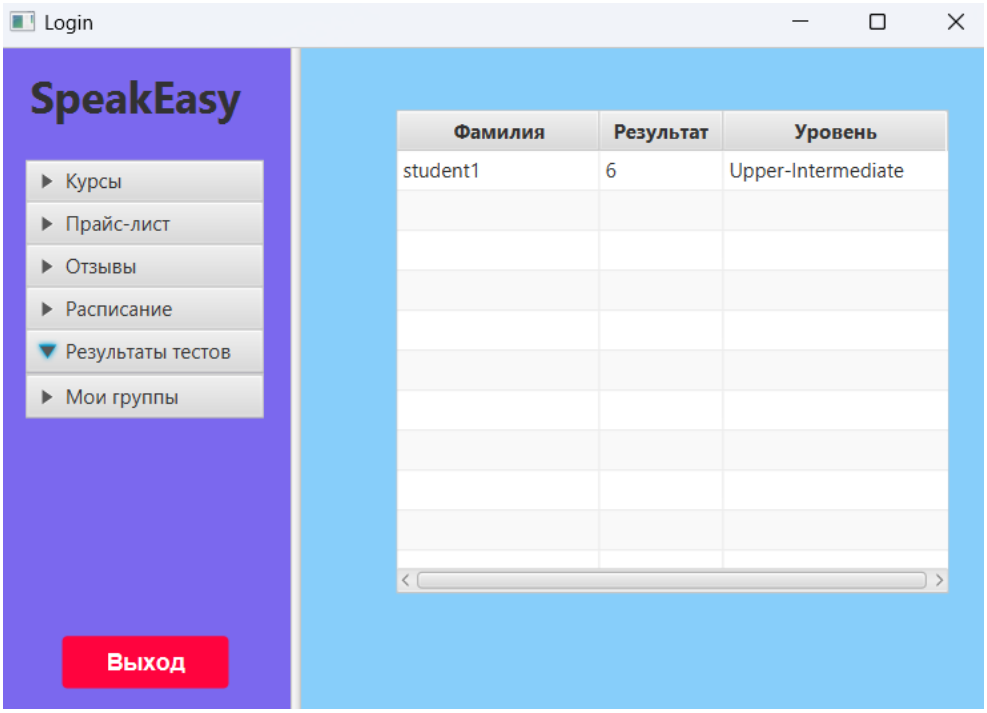


Рисунок 5.19 – Просмотр результатов тестирования

Для просмотра состава своих групп преподаватель должен выбрать номер одной из своих групп. После этого на сервер отправится запрос на получения списка студентов состоящей в этой группе. Сервер найдёт всех студентов из данной группы и передаст их на клиент. Клиентская часть в свою очередь разместит данные в таблицу. Данные в таблице можно будет отсортировать по каждому столбцу в порядке возрастания и убывания.

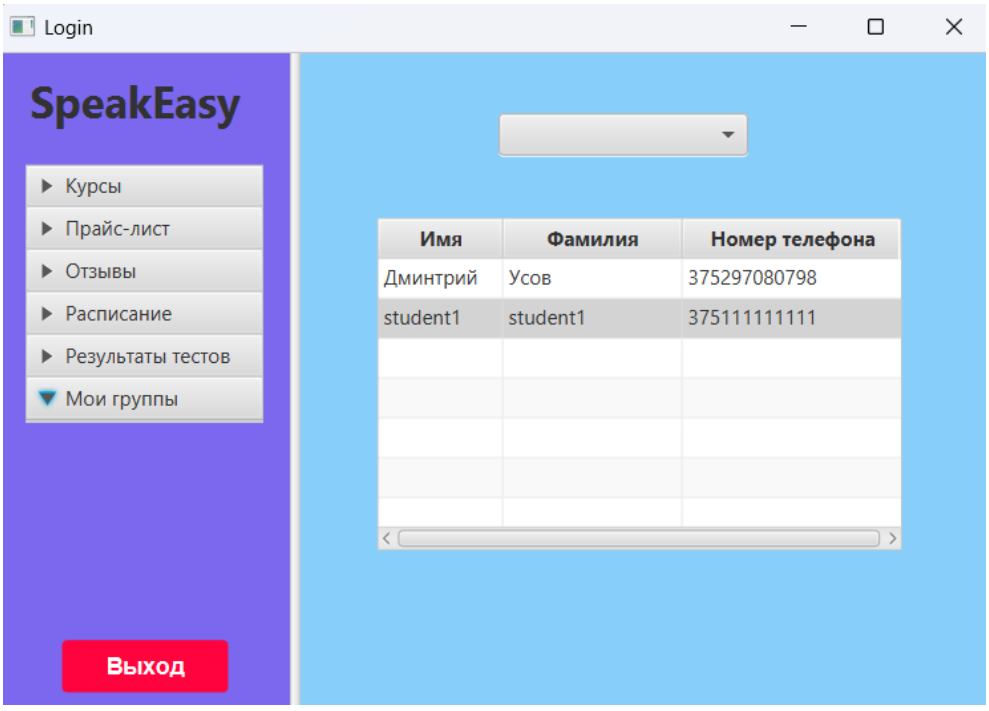


Рисунок 5.20 – Просмотр состав своих групп

5.5 Руководство администратора

Функционал администратора является самым большим в системе. Администратор может принимать заявки на обучение, распределяя студентов по группам. Так же администратор принимает заявки на преподавание.

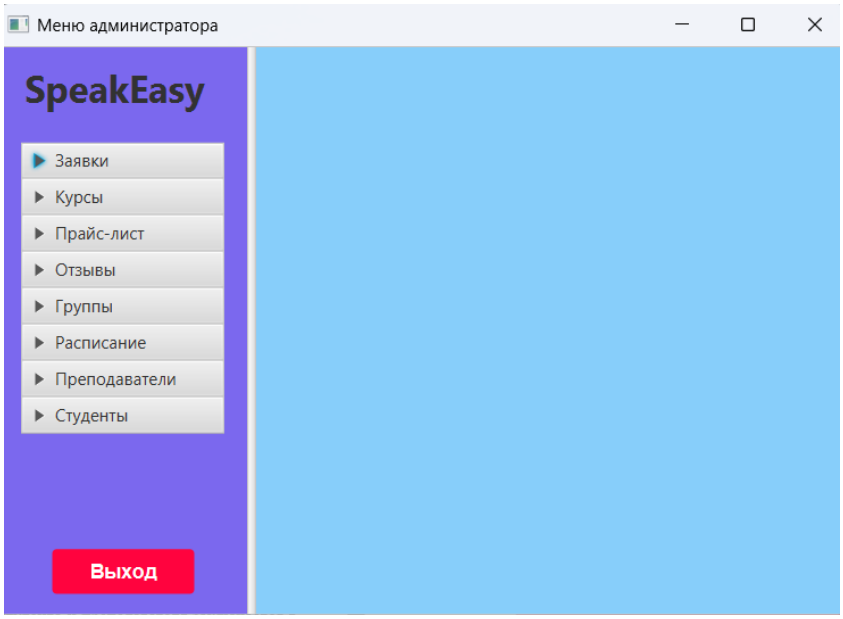


Рисунок 5.21 – Панель администратора

Отличия между добавлением студента и преподавателя заключается в том, что при добавлении студента нужно выбрать группу, в которую он добавляется, а для преподавателя таких действий не требуется.

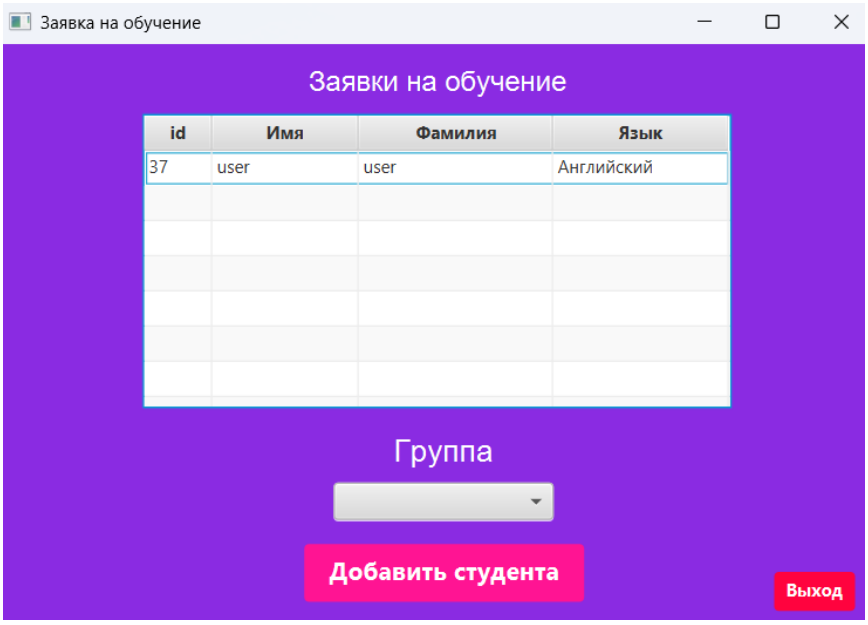


Рисунок 5.22 – Принять заявку на обучение

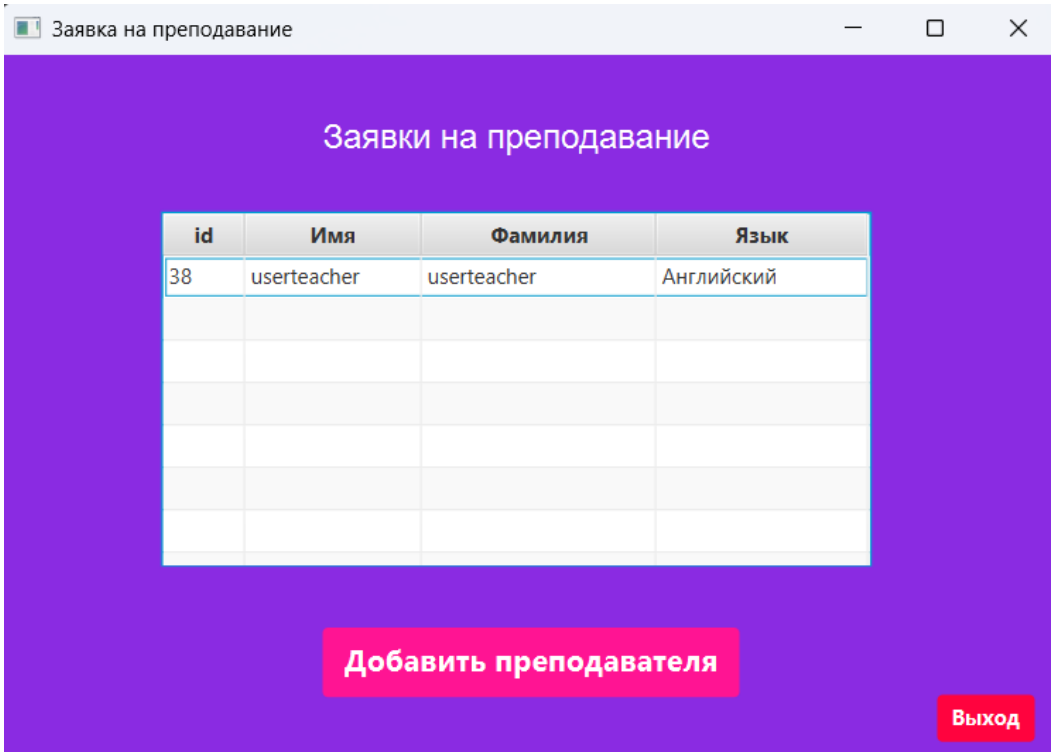


Рисунок 5.23 – Принять заявку на преподавание

Администратору открыт доступ к работе с курсами. Он может как добавить курс, так и удалить его.

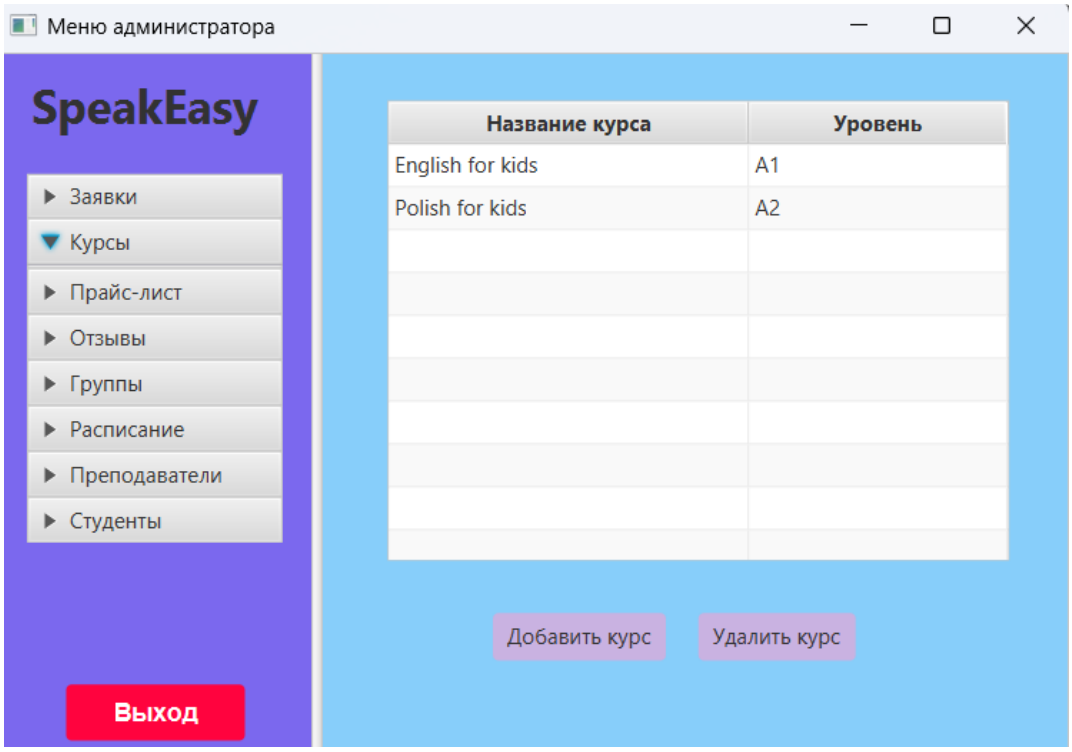


Рисунок 5.24 – Работа с курсами

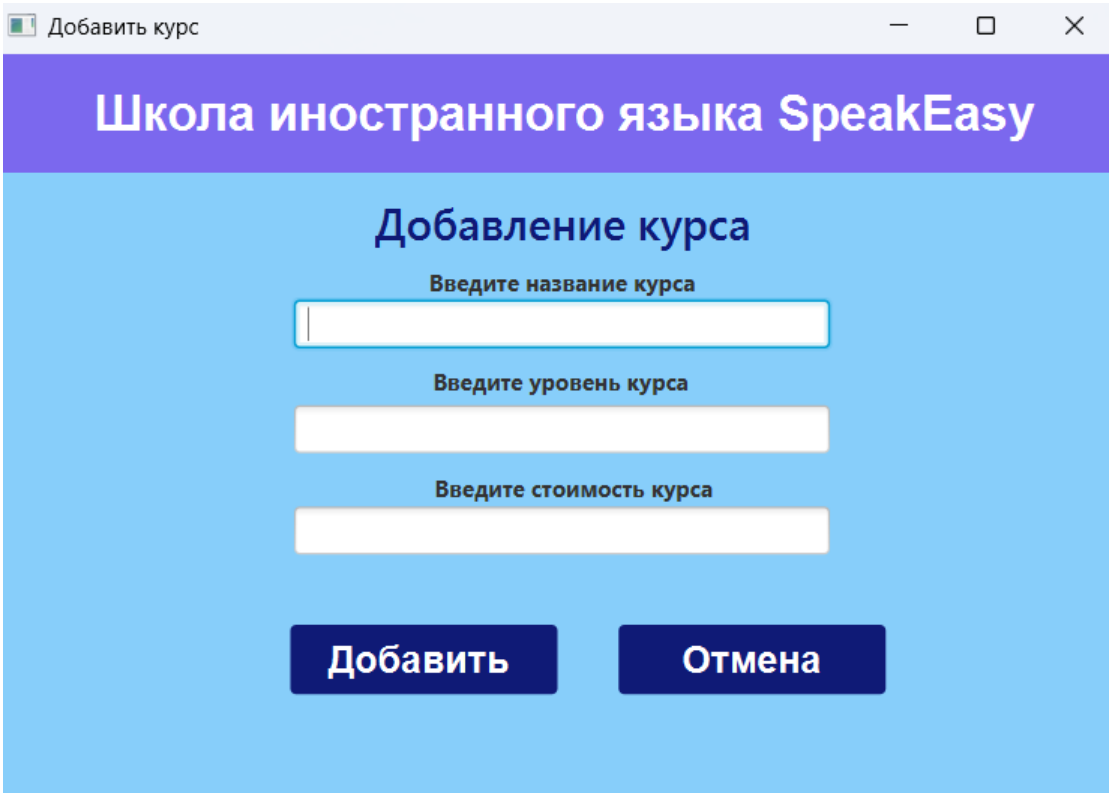


Рисунок 5.25 – Добавление курса

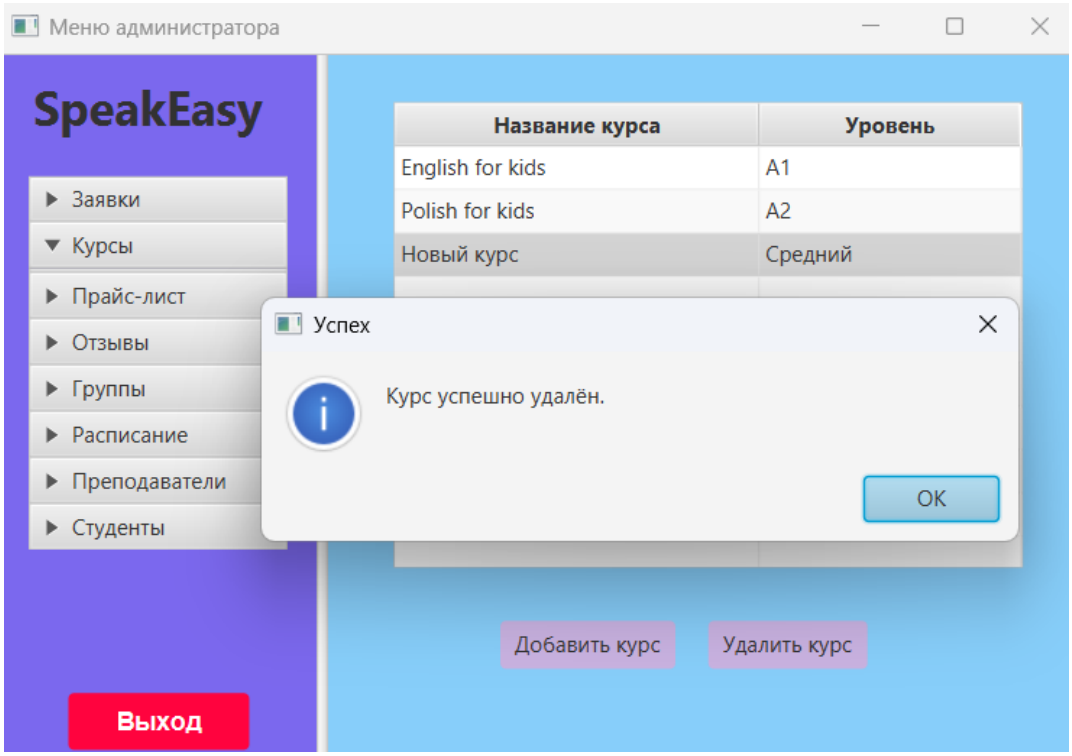


Рисунок 5.26 – Удаление курса

Администратор может изменять стоимость курсов.

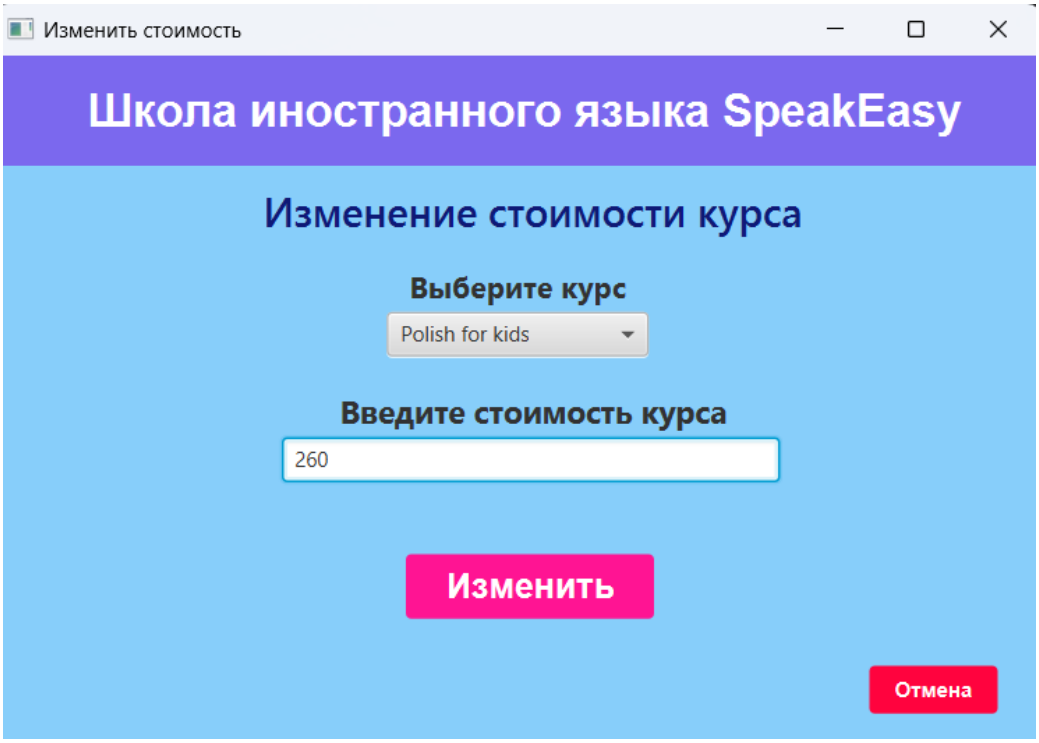


Рисунок 5.27 – Изменение стоимости курса

Администратор может заниматься модерацией отзывов, а именно удалять их.

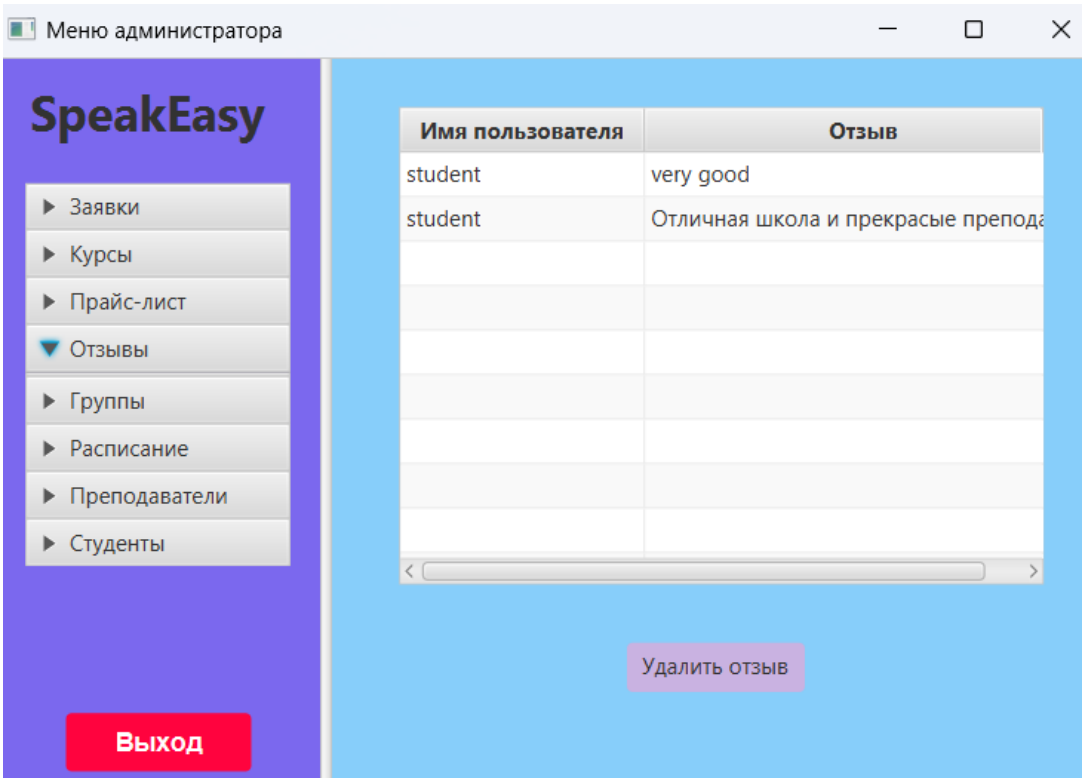


Рисунок 5.28 – Работа с отзывами

Администратор может работать с группами, создавать группу, удалять группу, изменять состав группы (переводить студентов из удалённых групп).

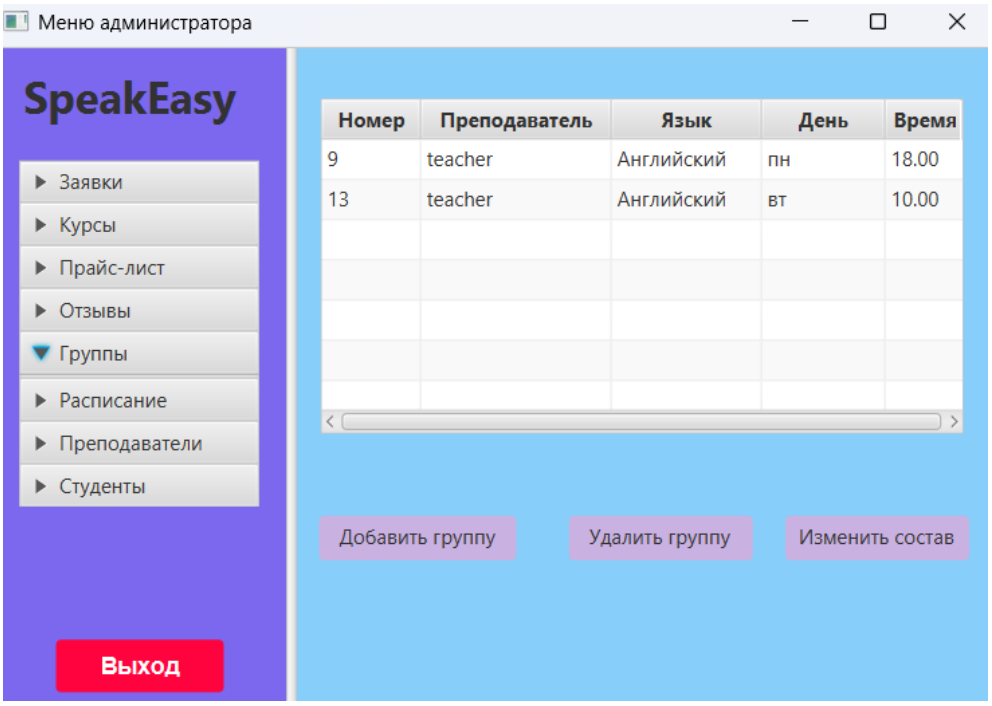


Рисунок 5.29 – Работа с группами

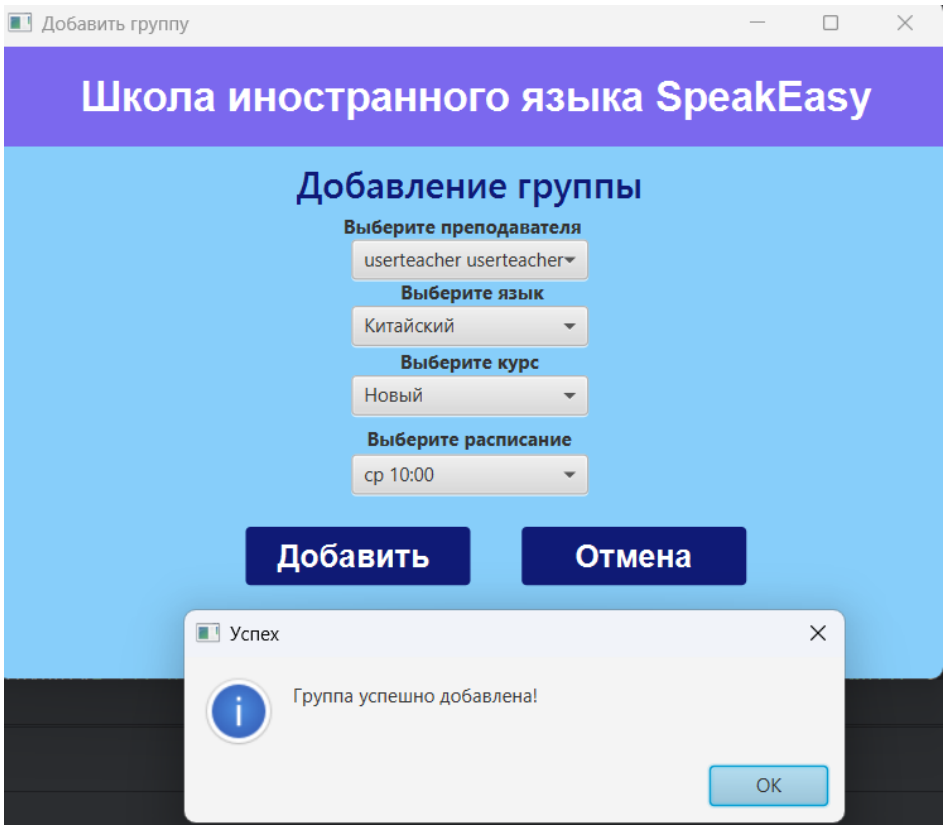


Рисунок 5.30 – Добавление группы

Удаление работает по выбору в таблице элемента и нажатия на кнопку удалить.

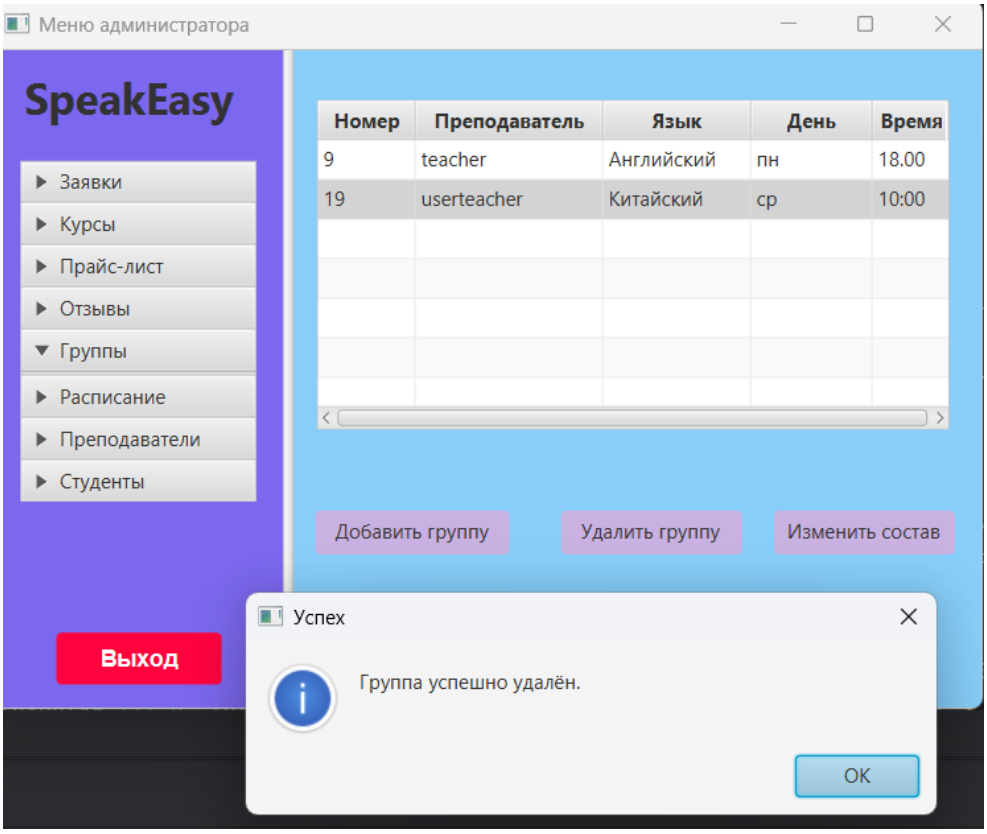


Рисунок 5.31 – Удаление группы

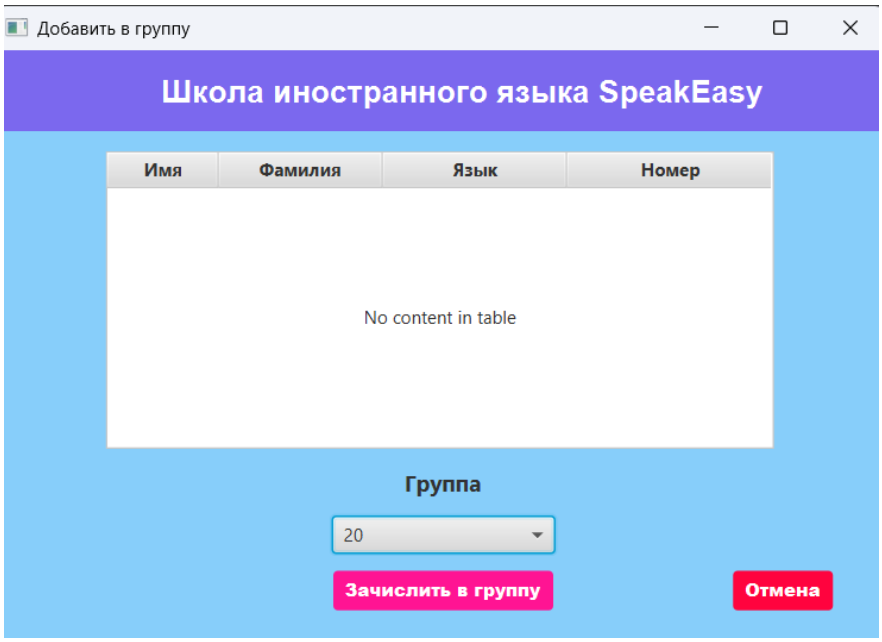


Рисунок 5.32 – Изменение состава групп

Администратор может работать с расписанием (добавить, изменить, удалить расписание).

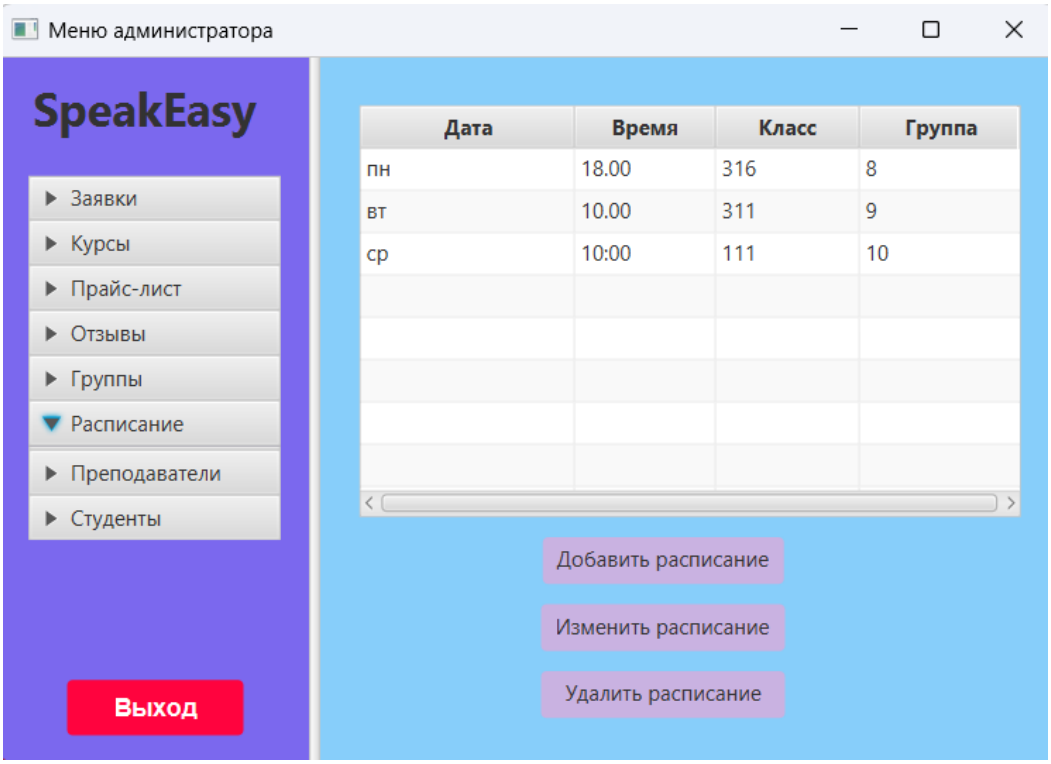


Рисунок 5.33 – Работа с расписанием

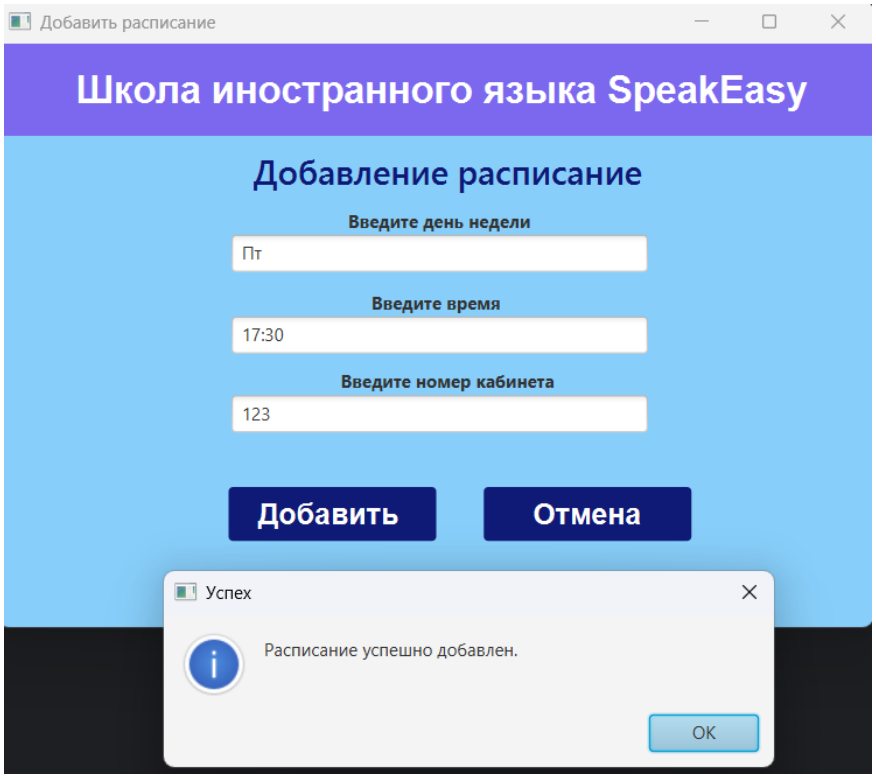


Рисунок 5.34 – Добавление расписания

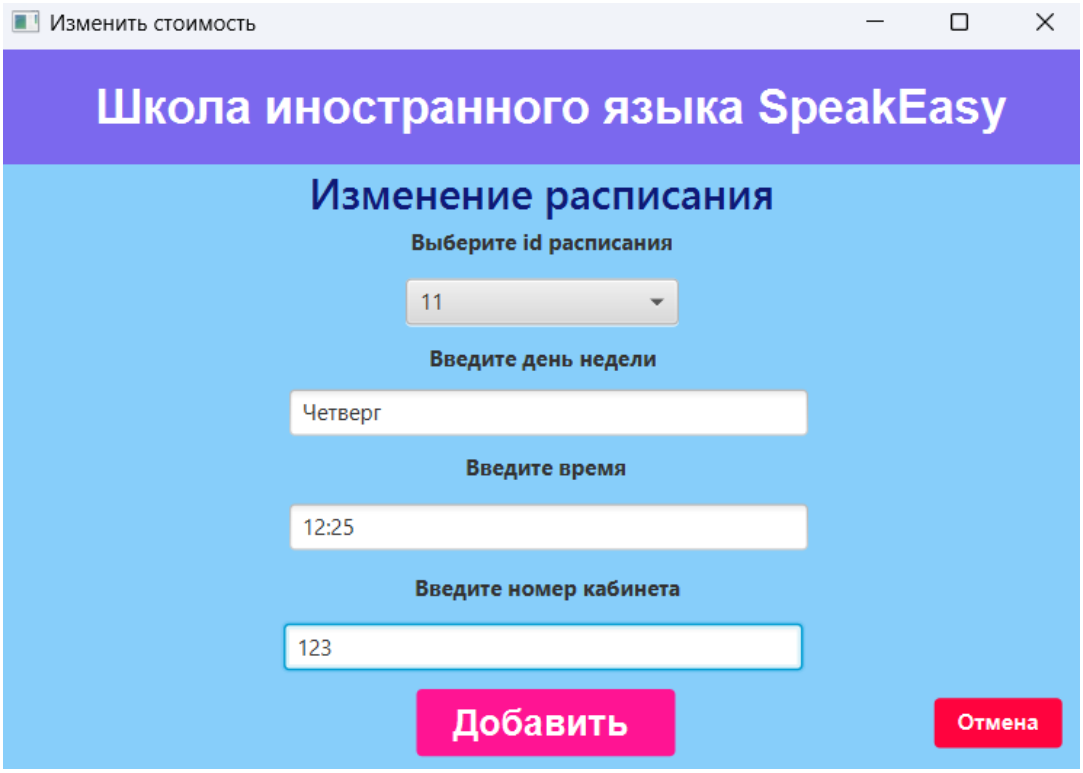


Рисунок 5.35 – Изменение расписания

Администратор может изменять данные преподавателя и удалять преподавателя.

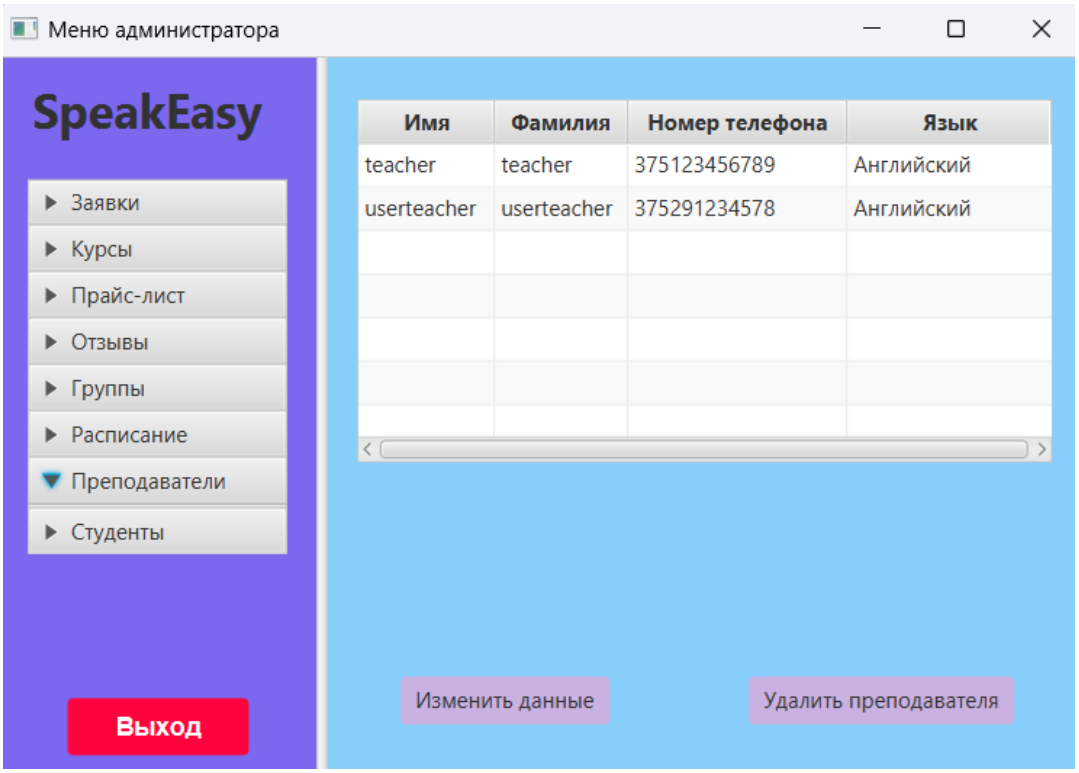


Рисунок 5.36 – Работа с преподавателями

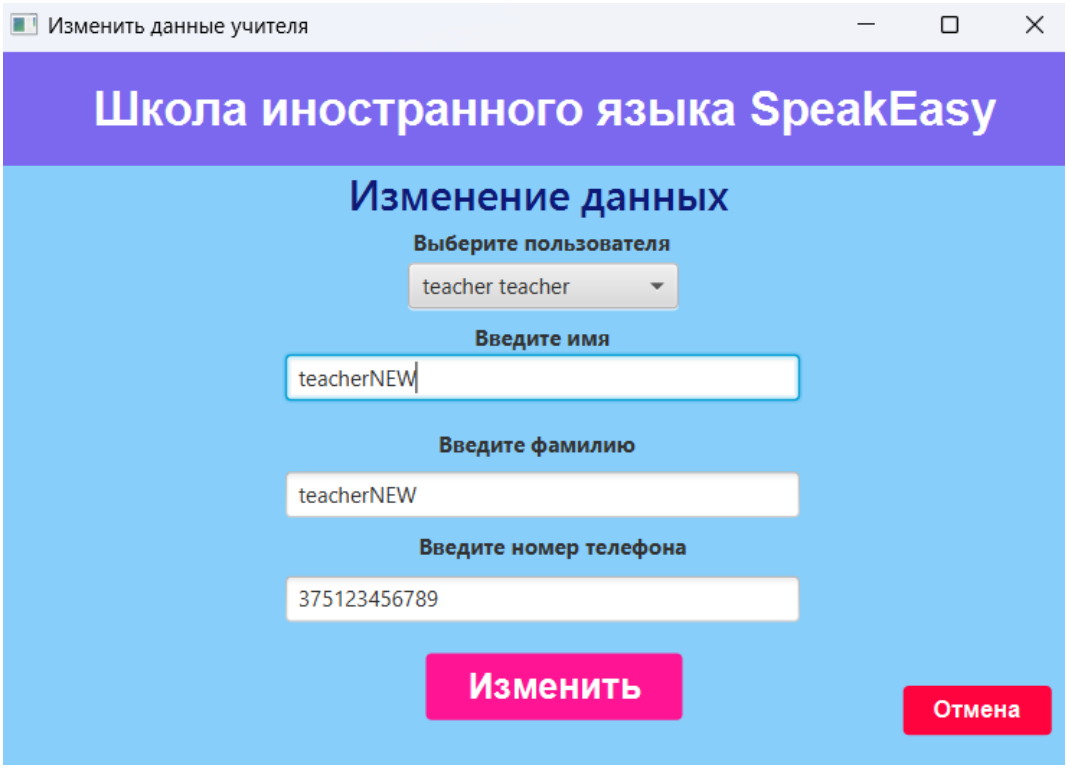


Рисунок 5.37 – Изменение данных преподавателя

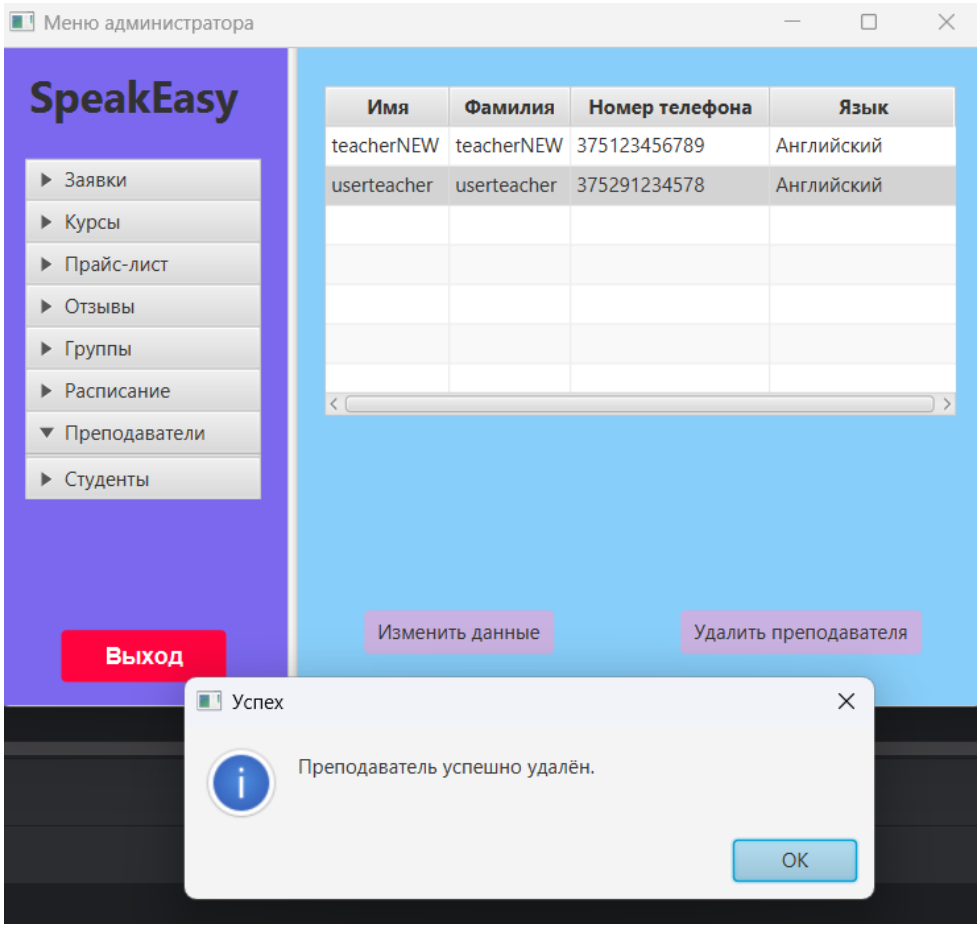


Рисунок 5.37 – Удаление преподавателя

Так же администратор может проводить такие операции и со студентами.

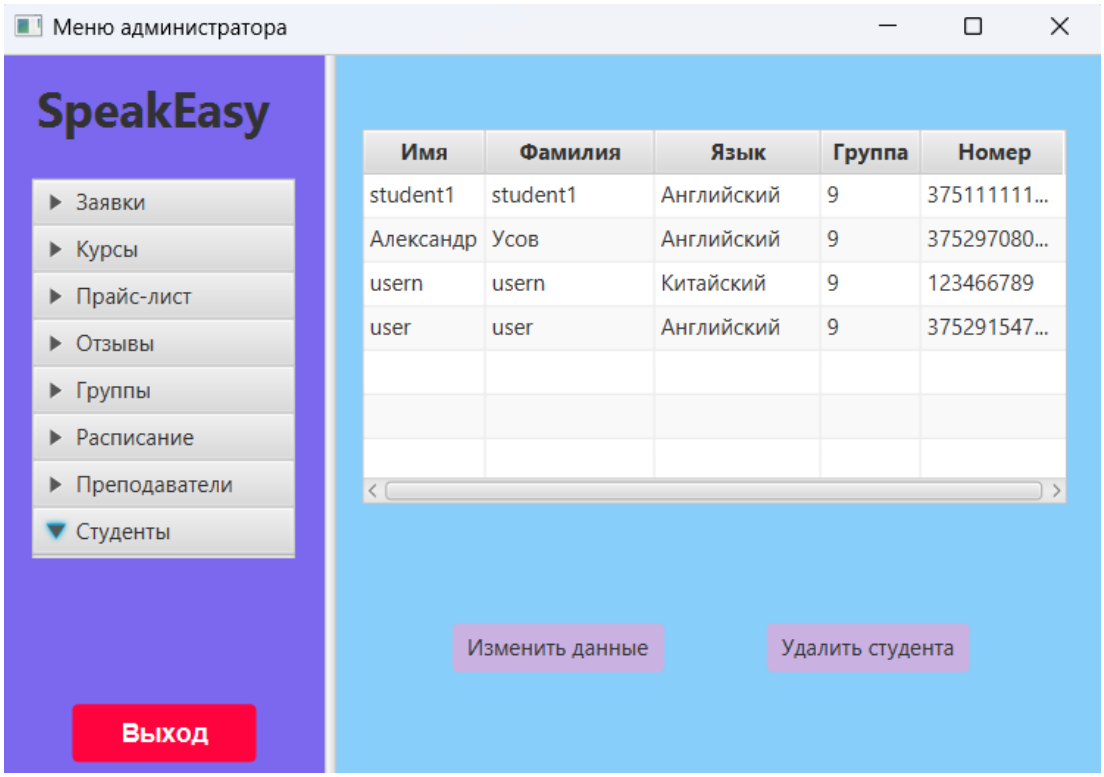


Рисунок 5.38 – Работа со студентами

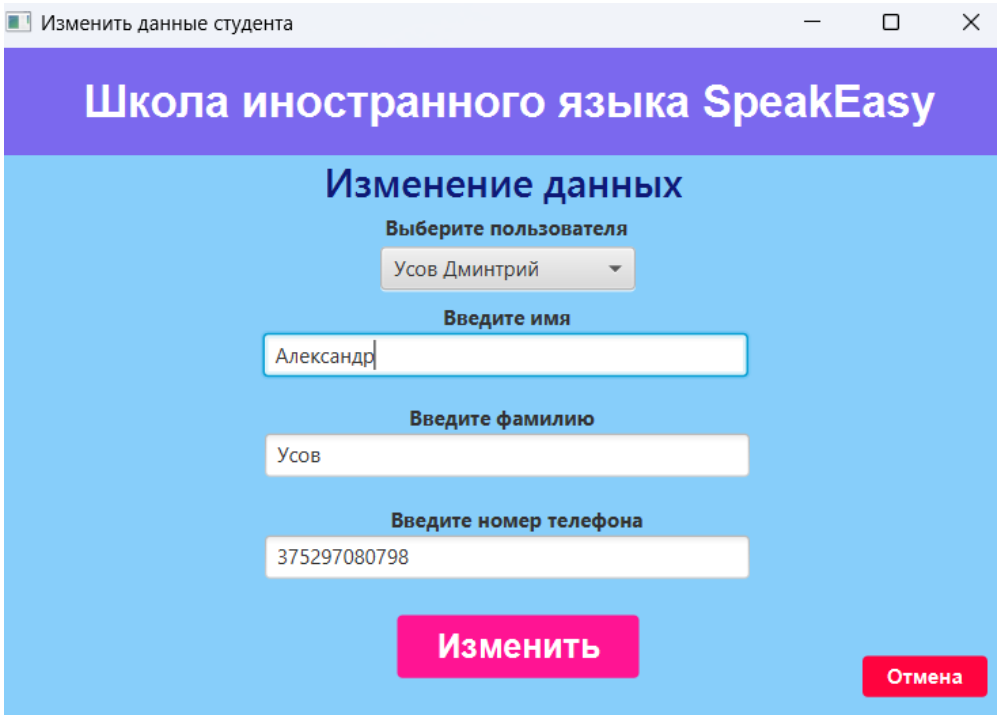


Рисунок 5.39 – Изменение данных студента

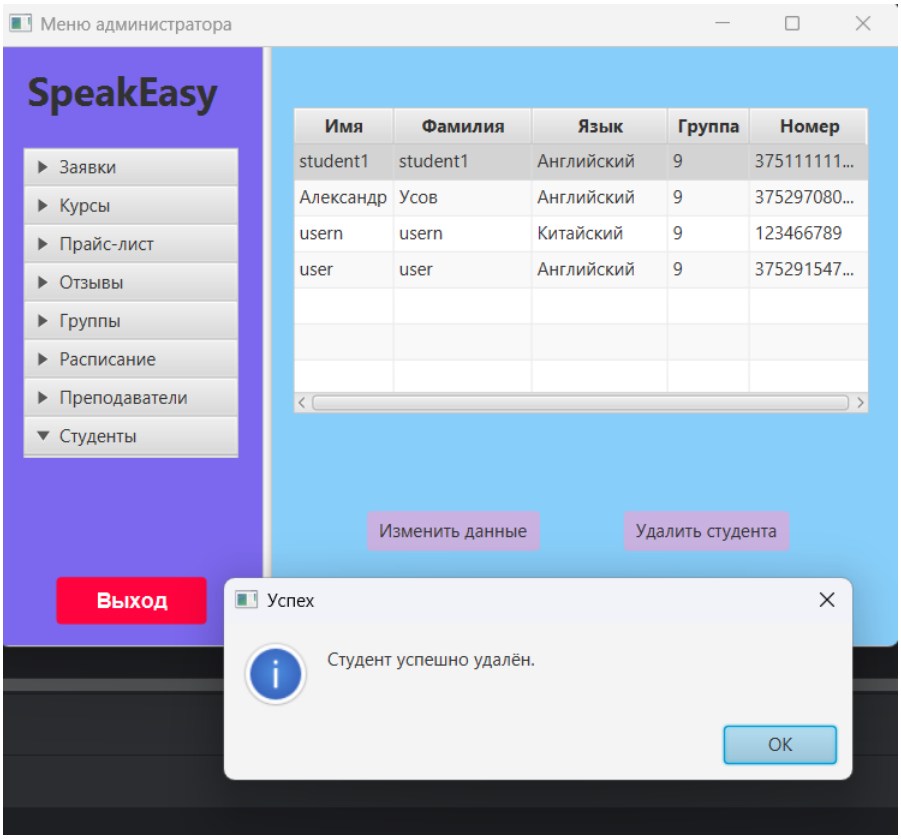


Рисунок 5.40 – Удаление студента

В данной главе представлена подробная инструкция, позволяющая любому пользователю установить и запустить приложение на персональном компьютере. Установка включает пошаговое руководство, начиная с загрузки установочного пакета с официального сайта или репозитория и заканчивая настройкой всех необходимых зависимостей, таких как база данных, драйверы и дополнительные библиотеки. Описывается процесс первого запуска, включая ввод учетных данных, проверку соединения с сервером и активацию основных модулей системы.

Кроме того, в главе раскрываются функциональные возможности системы для различных категорий пользователей. Обычные пользователи могут регистрироваться, авторизоваться, просматривать список доступных курсов, узнавать информацию о ценах и оставлять заявки на обучение. Для студентов предусмотрен расширенный функционал: они могут записываться на курсы, получать доступ к учебным материалам, выполнять задания и отслеживать личные результаты через удобный интерфейс.

Преподаватели получают инструменты для создания и редактирования курсов, загрузки материалов, проверки выполненных заданий студентов и выставления оценок. Для администраторов предусмотрен доступ к функциям управления пользователями, мониторинга активности системы, настройки глобальных параметров и обработки запросов на техническую поддержку.

Все разделы инструкции дополнены наглядными примерами интерфейсов, что обеспечивает интуитивное понимание работы с системой.

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсового проекта была разработана система автоматизации для школы иностранных языков, основная цель которой – оптимизация управления учебными процессами, повышение качества образовательных услуг и упрощение взаимодействия между всеми участниками образовательного процесса: администраторами, пользователями преподавателями и учениками.

Данная система была создана с учетом современных требований к образовательным учреждениям, где владение английским языком и использование информационных технологий являются важными составляющими успешного развития.

На начальном этапе проекта был проведен детальный анализ текущих процессов школы иностранных языков. Выявлены основные проблемы, включая сложности в управлении расписанием, регистрацией студентов на курсы, учете успеваемости и взаимодействии между преподавателями и администраторами. Результаты анализа легли в основу разработки функциональных требований к системе, которые включают автоматизацию записи студентов, ведение расписания, учет образовательной успеваемости и анализ ключевых показателей эффективности.

Была создана информационная модель, включающая базу данных для хранения информации о студентах, курсах, преподавателях и учебных материалах. Эта база данных отличается структурированностью, что делает ее удобной для использования. В процессе разработки приложения была уделена особая роль проектированию пользовательского интерфейса, чтобы система была понятной и удобной для всех категорий пользователей.

Функциональность приложения включает возможности регистрации студентов на курсы, автоматическое составление и редактирование расписания, учет посещаемости и успеваемости, а также возможность анализа данных для повышения эффективности учебного процесса. В системе реализованы гибкие инструменты управления курсами и отслеживания прогресса студентов, что позволяет не только облегчить административные задачи, но и улучшить качество образовательных услуг.

Особое внимание в проекте было уделено безопасности данных. Были реализованы современные методы шифрования и защиты информации, что минимизирует риски утечки данных и предотвращает несанкционированный доступ. Это позволяет обеспечить надежность работы системы и защиту персональных данных всех участников образовательного процесса.

В результате выполнения проекта была создана современная автоматизированная система, которая позволила значительно повысить эффективность работы школы иностранных языков. Реализация системы сделала обучение более удобным и доступным, улучшив качество образовательных услуг и повысив конкурентоспособность школы на рынке образовательных услуг.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Иностранные языки и развитие мозга человека [Электронный ресурс]. – Режим доступа: https://osvita.in/ru/inostrannye-jazyki-i-razvitie-mozga-cheloveka-porazitelnye-fakty/

[2] Школа иностранного языка ULC [Электронный ресурс]. – URL: https://ulc.by/

[3] Система Moodle [Электронный ресурс]. – Режим доступа: https://mts-link.ru/blog/moodle/

[4] Что такое CRM Битрикс 24 [Электронный ресурс]. – Режим доступа: https://www.aviant.ru/about/articles/chto-takoe-crm-bitriks-24/

[5] 1С:Образование [Электронный ресурс]. – Режим доступа: https://obr.1c.ru/educational/prepodavatelyam/1s-obrazovanie-5-shkola/

[6] Маклаков, С. В. BPwin и Erwin. CASE-средства разработки информационных систем /С. В. Маклаков. – М.: Диалог-МИФИ, 2001. – 295 с.

[7] Методология IDEF0 [Электронный ресурс]. – Режим доступа: https://sites.google.com/site/anisimovkhv/learning/pris/lecture/tema6/tema6_2

[8] Построение диаграммы вариантов использования [Электронный ресурс]. – Режим доступа: https://www.bsuir.by/m/12_100229_1_65759.pdf

[9] Принципы ООП [Электронный ресурс]. – Режим доступа: https://javarush.com/groups/posts/principy-oop

[10] Шпаргалка Java программиста 1: JPA и Hibernate в вопросах и ответах [Электронный ресурс]. – Режим доступа: https://habr.com/ru/articles/265061/

[11] Как надо хешировать пароли и как не надо системы [Электронный ресурс]. – Режим доступа: https://habr.com/ru/articles/210760/

[12] Учебник по JavaFX: начало работы [Электронный ресурс]. – Режим доступа: https://habr.com/ru/articles/474292/

[13] Зачем нам UML? Или как сохранить себе нервы и время [Электронный ресурс]. – Режим доступа: https://habr.com/ru/articles/458680/

[14] Описание алгоритмов, реализующих бизнес-логику серверной части проектируемой системы [Электронный ресурс]. – Режим доступа: https://cyberpedia.su/13x11c00.html

[15] JUnit / Тестирование в Java – JavaRush [Электронный ресурс]. – Режим доступа: https://javarush.com/groups/posts/605-junit

Приложение А
(обязательное)

Отчет о проверке на заимствования в системе «Антиплагиат»

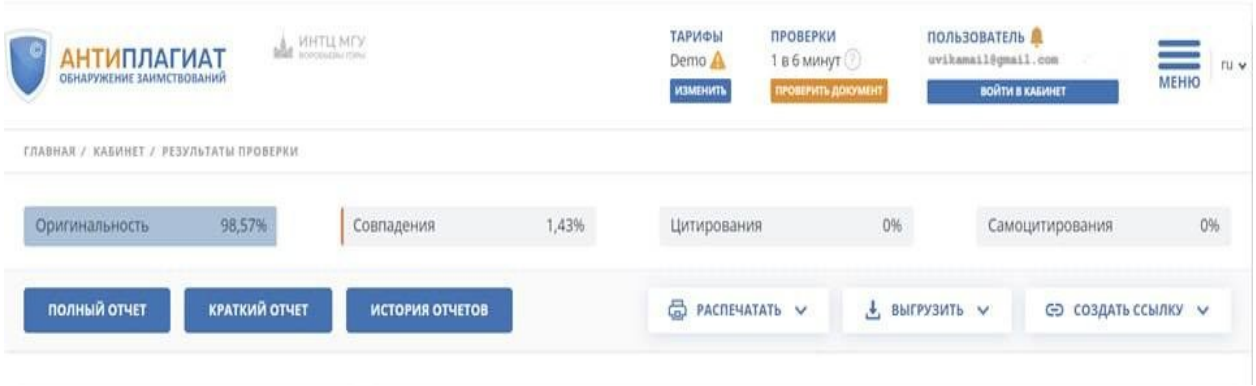


Рисунок А.1 – Результат проверки на антиплагиат

Приложение Б
(обязательное)

Листинг кода алгоритмов, реализующих основную бизнес-логику

```
public void handleStudyApplication(Request request) {
        List<Language> languages = languageService.findAll();
        String languagesJson = gson.toJson(languages);
        System.out.println(languagesJson);
        response.setMessage(languagesJson);
        response.setSuccess(true);
        response.setResponseType(RequestType.StudyApplication);
        writer.println(gson.toJson(response));
        writer.flush();
        System.out.println("Ответ отправлен на клиент");
    }
public void onSubmitApplication() {
    if (!languagesLoaded) {
        System.out.println("Языки еще не загружены.");
        return;
    }
    String selectedLanguage = languageChoiceBox.getValue();
    if (selectedLanguage == null) {
        System.out.println("Выберите язык.");
        return;
    }

    // Получаем ID языка
    int languageId = getLanguageIdByType(selectedLanguage);
    System.out.println(languageId);

    // Получаем currentProfileId из сессии
    int currentProfileId =
CurrentSession.getInstance().getCurrentProfileId();
    System.out.println(currentProfileId);


    Profile profile = new Profile(); // Заполните этот объект нужными
данными
    profile.setId(currentProfileId);

    Type_application typeApplication = new Type_application(); //
Заполните этот объект нужными данными
    typeApplication.setId(1); // Например, ID типа заявки

    org.example.Entities.Language language = new
org.example.Entities.Language(); // Заполните этот объект нужными
данными
    language.setId(languageId);
    System.out.println(languageId);

    // Создаем объект заявки
    org.example.Entities.Application application = new
org.example.Entities.Application();
```

```
application.setProfile(profile); // Заполняем профиль
    application.setType_application(typeApplication); // Заполняем
тип заявки
    application.setLanguage(language); // Заполняем язык
    application.setStatus("pending"); // Статус заявки
    System.out.println(application);
    // Формируем запрос
    Request request = new Request();
    request.setRequestType(RequestType.SubmitApplicationStudy);
    System.out.println(request);// Убедитесь, что этот тип
соответствует типу на сервере
    request.setMessage(new Gson().toJson(application));
    // Отправка запроса на сервер
    new Thread(() -> {
        try {
            // Отправляем запрос
            ClientSocket.getInstance().getWriter().println(new
Gson().toJson(request));
            ClientSocket.getInstance().getWriter().flush();
            System.out.println("данные отправлены");

            // Получаем ответ от сервера
            String responseLine = ClientSocket.getReader().readLine();
            Response response = new Gson().fromJson(responseLine,
Response.class);

            Platform.runLater(() -> {
                // Проверяем успех отправки заявки
                if (response.getSuccess()) {
                    showAlert("Заявка отправлена успешно!", "Успех");
                } else {
                    showAlert("Ошибка отправки заявки: " +
response.getMessage(), "Ошибка");
                }
            });
        } catch (IOException e) {
            System.out.println("Ошибка при отправке заявки: " +
e.getMessage());
        }
    }).start();
}
private void handleAddStudent(Request request) {
    try {
        // Десериализуем объект студента из тела запроса
        Student student = gson.fromJson(request.getMessage(),
Student.class);
        // Используем DAO для сохранения студента
        StudentService studentService = new StudentService();
        int studentCount =
studentService.getStudentCountInGroup(student.getGroup().getId());

        if (studentCount >= 10) {
            Response response = new Response();
```

```
response.setResponseType(RequestType.AddStudent);

response.setSuccess(false);
            response.setMessage("Группа уже заполнена (максимум 10
студентов).");
            writer.println(gson.toJson(response));
            writer.flush();
            return;
        }
        studentService.insert(student);
        int id
=studentService.findProfileIdByStudentId(student.getId());
        Profile profile = profileService.findById(id);
        int userid =profileService.findUserIdByProfileId(id);
        User user = userService.findById(userid);
        Role userRole = userService.getRoleById(3);
        user.setRole(userRole);
        profile.setUser(user);
        userService.update(user);
        applicationService.deleteByProfileId(id);
        // Отправляем успешный ответ
        Response response = new Response();
        response.setResponseType(RequestType.AddStudent);
        response.setSuccess(true);
        response.setMessage("Студент успешно добавлен.");
        writer.println(gson.toJson(response));
        writer.flush();

    } catch (Exception e) {

        e.printStackTrace();
        Response response = new Response();
        response.setSuccess(false);
        response.setMessage("Ошибка: " + e.getMessage());
        writer.println(gson.toJson(response));
        writer.flush();
    }
}

private void addStudentToDatabase() {
    // Получить выбранное заявление
    org.example.Entities.Application selectedApplication =
studentApplicationTable.getSelectionModel().getSelectedItem();
    Integer selectedGroupId = groupChoiceBox.getValue();

    if (selectedApplication == null || selectedGroupId == null) {
        System.out.println("Не выбрано заявление или группа!");
        return;
    }

    try {
        // Создать запрос на добавление студента
        Request request = new Request();
        request.setRequestType(RequestType.AddStudent);
```

```
// Создать объект для передачи

 Student student = new Student();
        student.setProfile(new
Profile(selectedApplication.getProfile().getId())); // ID профиля из
заявления
        student.setLanguage(new
Language(selectedApplication.getLanguage().getId())); // ID языка из
заявления
        student.setGroup(new Group(selectedGroupId)); // ID выбранной
группы

        // Добавить данные в запрос
        request.setMessage(gson.toJson(student));

        // Отправить запрос

ClientSocket.getInstance().getWriter().println(gson.toJson(request));
        ClientSocket.getInstance().getWriter().flush();

        // Получить ответ
        String responseLine =
ClientSocket.getInstance().getReader().readLine();
        Response response = gson.fromJson(responseLine,
Response.class);

        if (response.getSuccess()) {
            System.out.println("Студент успешно добавлен!");
        } else {
            System.out.println("Ошибка при добавлении студента: " +
response.getMessage());
        }

    } catch (IOException e) {
        System.out.println("Ошибка соединения с сервером: " +
e.getMessage());
    }
}
```

Приложение В
(обязательное)

Листинг скрипта генерации базы данных

```
CREATE DATABASE SchoolDB;
USE SchoolDB;

-- Создание таблицы Role
CREATE TABLE Role (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    roleType VARCHAR(50) NOT NULL
);

-- Создание таблицы Language
CREATE TABLE Language (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    languageType VARCHAR(50) NOT NULL
);

-- Создание таблицы User
CREATE TABLE User (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(200) NOT NULL,
    role_id INT ,
    FOREIGN KEY (role_id) REFERENCES Role(id) ON DELETE SET NULL
);
CREATE TABLE Profile (
       id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
       name VARCHAR(50) NOT NULL,
       lastname VARCHAR(50) NOT NULL,
    phonenumber VARCHAR(20),
    user_id INT UNIQUE,
       FOREIGN KEY (user_id) REFERENCES User(id) ON DELETE CASCADE
);

-- Создание таблицы Teacher
CREATE TABLE Teacher (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    language_id INT UNIQUE,
    profile_id INT UNIQUE,
    FOREIGN KEY (language_id) REFERENCES Language(id) ON DELETE SET
NULL,
    FOREIGN KEY (profile_id) REFERENCES Profile(id) ON DELETE CASCADE
);

-- Создание таблицы Course
CREATE TABLE Course (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    course_type VARCHAR(50) NOT NULL
);

-- Создание таблицы Prise_list
CREATE TABLE Prise_list (
```

```
 id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
course_id INT UNIQUE,
    price DECIMAL(10, 2) NOT NULL,

 FOREIGN KEY (course_id) REFERENCES Course(id) ON DELETE CASCADE
);

-- Создание таблицы Schedule
CREATE TABLE Schedule (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    date DATE NOT NULL,
    time TIME NOT NULL,
    class VARCHAR(50) NOT NULL
);
-- Создание таблицы Group
CREATE TABLE `Group` (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    teacher_id INT ,
    language_id INT ,
    schedule_id INT UNIQUE,
    course_id INT ,
    FOREIGN KEY (teacher_id) REFERENCES Teacher(id) ON DELETE SET NULL,
    FOREIGN KEY (language_id) REFERENCES Language(id) ON DELETE CASCADE,
    FOREIGN KEY (schedule_id) REFERENCES Schedule(id) ON DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES Course(id) ON DELETE CASCADE
);
-- Создание таблицы Student
CREATE TABLE Student (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    language_id INT,
    group_id INT ,
    profile_id INT UNIQUE,
    FOREIGN KEY (language_id) REFERENCES Language(id) ON DELETE SET
NULL,
    FOREIGN KEY (group_id) REFERENCES `Group`(id) ON DELETE SET NULL,
    FOREIGN KEY (profile_id) REFERENCES Profile(id) ON DELETE SET NULL
);

-- Создание таблицы Test
CREATE TABLE Test (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    Test_result INT,
    student_id INT,
    language_skill VARCHAR(50),
    FOREIGN KEY (student_id) REFERENCES Student(id) ON DELETE SET NULL
);
CREATE TABLE Type_application(
 id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
 type VARCHAR(50) NOT NULL
);
-- Создание таблицы Application
CREATE TABLE Application (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
```

```
profile_id INT ,
    status VARCHAR(50) NOT NULL,

 type_application_id INT,
    FOREIGN KEY (profile_id) REFERENCES Profile(id) ON DELETE SET NULL,
FOREIGN KEY (type_application_id) REFERENCES Type_application(id) ON
DELETE SET NULL
);

-- Создание таблицы Feedback
CREATE TABLE Feedback (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    text TEXT,
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES User(id) ON DELETE SET NULL
);
```

ПРИЛОЖЕНИЕ Г
(справочное)

Листинг кода JUnit тестов

```
// Тестирование регистрации пользователя

    @Test
    void testUserRegistration() {
        User user = new User("testuser", "test@example.com",
"password123");
        boolean registrationResult = userService.register(user);
        assertTrue(registrationResult);

        assertNotNull(userService.getUserByUsername("testuser"));
    }

    // Тестирование добавления курса

    @Test
    void testAddCourse() {
        Course course = new Course("English for Beginners", "Beginner");
        boolean addCourseResult = courseService.addCourse(course);
        assertTrue(addCourseResult);
        assertNotNull(courseService.getCourseByName("English for
Beginners"));
    }

    // Тестирование проверки существования пользователя

    @Test
    void testUserExists() {
        User user = new User("testuser", "test@example.com",
"password123");
        userService.register(user);
        assertDoesNotThrow(() ->
userService.checkUserExists("testuser"));

        assertThrows(UserNotFoundException.class, () -> {
            userService.checkUserExists("nonexistentuser");
        });
    }

    // Тестирование проверки роли пользователя

    @Test
    void testUserRole() {
        User user = new User("studentuser", "password123");
        user.setRole("USER");
        userService.register(user);

        assertDoesNotThrow(() ->
userService.checkUserRole("studentuser", "USER"));
        user.setRole("ADMIN");
        assertThrows(InvalidUserRoleException.class, () -> {
            userService.checkUserRole("studentuser", "USER");
        });
```

```
}

 // Тестирование проверки наличия свободных мест на курсе

    @Test
    void testCheckAvailableSeats() {

        Course course = new Course("English for Beginners", "Beginner");
        course.setAvailableSeats(5);
        courseService.addCourse(course);

        assertDoesNotThrow(()                                          ->
courseService.checkAvailableSeats("English for Beginners", 3));


        assertThrows(InsufficientSeatsException.class, () -> {
            courseService.checkAvailableSeats("English  for  Beginners",
10);
        });
    }
```

Ведомость документов курсового проекта